04820KA001 Mono Canada Slitter 903_V12_1104

Author: Michou
Created: 11/4/2026 7:16:15
Last Modified: 11/4/2026 9:58:53
Comment: Modified to Connect 2nd Diameter Sens

Sysmac Studio Module Version: 1.60.0.64010

Table of Contents

04820KA001 Mono Canada Slitter 903_V12_1104	3
1.PLC	3
1-8.POUs	3
1-8-1.Programs	3
1-8-1-1.TaglMonoTRM	3
1-8-1-1-1.Variables	3
1-8-1-1-2.General	14
1-8-1-1-3.Motor_Manager	21
1-8-1-1-4.OverRider_Manager	35
1-8-1-1-5.Console	38
1-8-1-1-6.Alarms	43
1-8-1-1-7.Output_Manager	48
1-8-1-1-8.AutoStop_Control	54
1-8-1-1-9.Tension_Helper	76
1-9.Data	77
1-9-1.Data Types	77
1-9-2.Global Variables	78

1.PLC

1-8.POUs

1-8-1.Programs

1-8-1-1.TaglMonoTRM

1-8-1-1-1.Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Comment
VAR						
SpdDelta	REAL			False	False	Minimum or maximum speed SpdDeltaerence
UwSlowDiamCalc	REAL			False	False	V7c: speed-dependent unwinder slow-down diameter (mm)
SonarRunMem	bool			False	False	
EndStopReq	BOOL			False	False	V12: end-of-roll stop requested (AutoStopCmd or unwinder diameter)
EndRampOK	BOOL			False	False	V12: end-of-roll ramp usable (FinalStopDecel, FF active, running)
EndRampHard	BOOL			False	False	V12: safety net during the ramp -> immediate stop
EndStopNow	BOOL			False	False	V12: immediate end-of-roll stop (Motor_Manager rung 4)
SonarInit	bool			False	False	
UwSlowDiamLatch	BOOL			False	False	V7c: unwinder diameter slow-down reached (frozen)
FFRate	REAL			False	False	V7f: ramp rate (reference per 4 ms scan)
FFRateSet	REAL			False	False	V7f: rate allowed to still stop at the target
FFJerk	REAL			False	False	V7f: max rate change per scan
SpdScanCnt	INT			False	False	V9: scan counter for the 100 ms speed window
HornRun	BOOL			False	False	V9: slow-down / final speed running
HornRunMem	BOOL			False	False	V9: edge memory
HornStartMem	BOOL			False	False	V9: start edge memory
HornTicks	INT			False	False	V9: scans since slow-down start
HornCnt	INT			False	False	V9: scans in the beep period
HornPer	INT			False	False	V9: beep period in scans
HornStartScans	INT			False	False	V9: start beep in scans
HornLenScans	INT			False	False	V9: short beep in scans
HornSlowBeep	BOOL			False	False	V9: slow-down beep
HornStopPend	BOOL			False	False	V9: stopped after a slow-down, waiting for standstill
HornStopTicks	INT			False	False	V9: stop beep counter
LongActive	BOOL			False	False	V9: long-baseline learning running
LongValid	BOOL			False	False	V9: long baseline >= 20 mm growth, replaces the windows
LongStartPos	DINT			False	False	V9: encoder at the start of the long baseline
LongStartD	REAL			False	False	V9: rewinder diameter at the start (mm)
LongGrow_mm	REAL			False	False	V9: growth since the start (mm)
LongPls	REAL			False	False	V9: pulses since the start
LongK	REAL			False	False	V9: K from the long baseline
RevW	REAL			False	False	V9: encoder pulses summed in th

						e current revolution
RevDP	DINT			False	False	V9: encoder pulses this scan
RevPrevPos	DINT			False	False	V9: encoder last scan
RevSum	REAL			False	False	V9: sum of raw DT35 distance x pulses
RevMin	REAL			False	False	V9: min raw distance in the revolution
RevMax	REAL			False	False	V9: max raw distance in the revolution
RwDiamRevOk	BOOL			False	False	V9: one full revolution averaged since the last fault
UwEmptyTicks	INT			False	False	V9: unwinder empty time (100 ms ticks)
UwFullTicks	INT			False	False	V9: new roll seen time (100 ms ticks)
GsmArmed	BOOL			False	False	V9: roll was empty -> next new roll asks the GSM question
T_JogStartBlk	TOF			False	False	V9: START blocked while JOG held + 2 s
StartHeldBlk	BOOL			False	False	V9: START pressed during the block, wait for release
StartPB_OK	BOOL			False	False	V9: START push-button accepted (replaces I_Start_PB in General 7/8/10)
FFSoftMem	BOOL			False	False	V9: soft stop last scan (rung B)
FFSlowAcc	BOOL			False	False	V9: gentle ramp-up after a cancelled soft stop
FFStartEdge	BOOL			False	False	V9: START push-button edge (rung B)
FFStartMem	BOOL			False	False	V9: START last scan (rung B)
SpdDeltaMin	REAL			False	False	Minimum speed difference
FFSqrtArg	REAL			False	False	SQRT argument (REAL)
SpdDeltaMax	REAL			False	False	Maximum speed SpdDeltaerence
SoftStopMem	BOOL			False	False	v6r: soft stop edge memory
SoftStopRatio	REAL			False	False	v6r: InvRef counts per m/min, frozen at the soft stop start
MemPos	DINT			False	False	Previous encoder count
MatChangeCnt	INT			False	False	v6s: consecutive learning windows far from RwMat_K
DeltaPulse	DINT			False	False	Encoder count difference
SlowMem	BOOL			False	False	SlowDown edge memory
DecelDiam	REAL			False	False	Rewinder diameter used by the decel model (mm, min 100)
TeachSwap	REAL			False	False	Temporary for UNDO swap
SlowStartDiam	REAL			False	False	Diameter at slow-down start (mm)
BrakeHold	BOOL			False	False	Keep T-one in auto during a normal stop (O_Brake)
StopNow	BOOL			False	False	Immediate stop request (replaces I_Stop_PB in General rung 9)
StopPB_Mem	BOOL			False	False	STOP button edge memory
T_StopLong	TON			False	False	STOP held 1 s = immediate stop
T_SoftStop	TON			False	False	Soft stop timeout (120 s)
BrakeHoldTicks	INT			False	False	BrakeHold duration after stop (100 ms ticks, max 150)
StopStartDiam	REAL			False	False	Diameter at stop command (mm)
DecelK_Meas	REAL			False	False	Measured decel per mm of diameter
SlowMeasActive	BOOL			False	False	Slow-down measurement running

TeachTicks	INT			False	False	TEACH CORE stable time (100 ms ticks)
SlowStartPos	DINT			False	False	Encoder at slow-down start
SlowStartSpd_Pps	REAL			False	False	Speed at slow-down start (pulses/s)
SlowMeas_Pls	REAL			False	False	Measured slow-down length (pulses)
SlowDecelMeas_mm	REAL			False	False	Measured slow-down decel (mm/s2)
SlowOneRamp	BOOL			False	False	Slow-down uses one ramp (manual or learned)
Velocity	REAL			False	False	Speed in [mm/s]
Alarm_Array	ARRAY[0..32] OF BOOL			False	False	Alarms
T_PlcOn	TON			False	False	PLC active timeout
ControlVoltage	BOOL			False	False	Auxiliaries OK
StartMotor_Cmd	BOOL			False	False	Motor start command
HubExtractEnable	BOOL			False	False	Hub extraction enable command
Brake_Cmd	BOOL			False	False	Brake command
T_StopVent	TOF			False	False	Fans stop timer
TimStopVen	TIME			False	False	Fans stop timeout
MotorCooling	BOOL			False	False	Fans command
OverRider_Auto	BOOL			False	False	OverRider in automatic
OverRider_Cont	BOOL			False	False	OverRider in continuous
OverRider_Lock	BOOL			False	False	OverRider lock command
GSM_LearnFault	BOOL			False	False	Paper [Grams / Square Meter] LearnFailCnt ≥ RwLearnFailMax
TickCnt	INT			False	False	100 ms tick scan counter (25 x 4 ms)
RunWasJog	BOOL			False	False	Run used jog / manual / slow button (2nd ramp)
StopLearnValid	BOOL			False	False	This stop may be learned
SpdBand_Pps	REAL			False	False	Speed band of the trim ramp (pulses/s)
TrimDecel_Pps2	REAL			False	False	Trim ramp decel (pulses/s2)
SpdKnee_Pps	REAL			False	False	Speed where coarse ramp hands over to trim ramp (pulses/s)
OverRider_RaiseUp	BOOL			False	False	OverRider raise command
Pot1_Mem	INT			False	False	Work potentiometer value
VelSet_mms	REAL			False	False	Set speed in mm/s
Vfd_SP_Modif	REAL			False	False	Inverter speed modifier
RwAutoLimAct	BOOL			False	False	v6y: rewinder AUTO speed limit active (taken over at standstill)
UwTeachTicks	INT			False	False	Unwinder TEACH CORE samples (100 ms)
T_PosPresOff	TON			False	False	OverRider photocell OFF timer
T_PosPresOn	TON			False	False	OverRider photocell ON timer
TimPosPresOff	TIME			False	False	OverRider photocell OFF timeout
TimPosPresOn	TIME			False	False	OverRider photocell ON timeout
RollEnd_Active	BOOL			False	False	Roll diameter photocell intercepted
Output_Ratio	REAL			False	False	Analog output ratio
Input_Ratio	REAL			False	False	Analog input ratio
T_StartLow	TON			False	False	Slow start timer
T_UwDiamDebounce	TON			False	False	Roll diameter photocell Debounce timer 500mS
UwDiamReache	BOOL			False	False	Roll diameter reached

d						
Index_A	INT			False	False	
AlarmStrobe	BOOL			False	False	Alarm message display strobe
T_Display	TON			False	False	Alarm display timer
RangeOK	BOOL			False	False	Range reached
Vel_Max_mms	REAL			False	False	Maximum speed in mm/s
Slow_Inhibit	BOOL			False	False	Inhibit slow button
Decel_Ramp	REAL			False	False	Deceleration ramp
Accel_Ramp	REAL			False	False	Acceleration ramp
Pot2_Mem	INT			False	False	Final potentiometer value
Current_mm	REAL			False	False	Current millimetres
EncoderPulse_to_mm	REAL			False	False	mm per encoder pulse
Stop	BOOL			False	False	Stop
T_EncoderStill	TON			False	False	(?) Encoder stopped timer
Analog_Res	UINT			True	False	Analog board resolution
Mt_To_mm	REAL			False	False	m/min to mm/s factor
Init_Ramp	BOOL			True	False	(?) Ramp initialisation
SlowByDiam	BOOL			False	False	(?) Slow down by diameter
UwStopByDiam	BOOL			False	False	Stop by diameter
T_Screen_Delay	TON			False	False	(?) Start page timer
T_PulStop	TON			False	False	(?) Stop button timer
Reading_Max	REAL	2000		True	False	Maximum sensor reading (full scale)
DeadZone	REAL	60		True	False	Sensor dead zone
In_UwDiameter	REAL			True	False	UnWinder Diameter Analog input voltage
Tick	BOOL			False	False	100 ms tick
MmPerPulse	REAL			False	False	mm per encoder pulse
RwDistRaw_mm	REAL			False	False	Rewinder Unfiltered distance (mm)
RwFiltInit	BOOL			False	False	Rewinder Filter initialised
SpdInit	BOOL			False	False	Speed init
EncPrev	DINT			False	False	Encoder at previous tick
SpdAct_Pps	REAL			False	False	Actual Speed (pulses/s)
LearnInit	BOOL			False	False	Learn initialised
LearnWinRunning	BOOL			False	False	Motor ran whole window
LearnWinTicks	INT			False	False	Learn window counter
LearnWin_Pls	DINT			False	False	Pulses in window
LearnWinDiamDelta	REAL			False	False	Diameter change in window
MatK_Window	REAL			False	False	Material factor this window
MotorRunMem	BOOL			False	False	Motor start edge memory
StopMeasActive	BOOL			False	False	Stop measurement running
StopStartPos	DINT			False	False	Encoder at stop command
StopStartSpd_Pps	REAL			False	False	Speed at stop command (pulses/s)
StandstillTicks	INT			False	False	Standstill tick counter
LearnWinOk	BOOL			False	False	Learning window gave a valid result
ThickWin_mm	REAL			False	False	Paper thickness this window (mm)
LearnFailCnt	INT			False	False	Consecutive failed learning windows
StopLearnOk	BOOL			False	False	Stop measurement gave a valid result
StopDecelMeas_mm	REAL			False	False	Measured stop decel (mm/s2)
StopMeas_Pls	REAL			False	False	Measured stop length (pulses)

StopDecelMeas	REAL			False	False	Measured stop decel (pulses/s2)
StopLen_Pls	REAL			False	False	Stop length (pulses)
StopLenFinal_Pls	REAL			False	False	Stop length from slow speed (pulses)
RwSlowLen_Pls	REAL			False	False	Rewinder Slow-down length (pulses)
SlowDecel_Pps2	REAL			False	False	Slow-down decel (pulses/s2)
SpdFinal_Pps	REAL			False	False	Final speed (pulses/s)
RwRemain_Pls	REAL			False	False	Rewinder Remaining length to target (pulses)
LearnWinStartPos	DINT			False	False	Rewinder Position Window 0
LearnWinStartDiam	REAL			False	False	Rewinder Diameter Window 0
T_ByPassProtOff	TOF			False	False	Delay Off Timer
UwEnMem	BOOL			False	False	Edge memory UwStopEnable
RwEnMem	BOOL			False	False	Edge memory RwEnable
SpdRef_Pps	REAL			False	False	Reference speed MAX (actual,setpoint) (pulses/s)
SpdWork_Pps	REAL			False	False	Work speed setpoint (pulses/s)
SlowLenBase_Pls	REAL			False	False	Slow-down length without margin (pulses)
UwSlowLen_Pls	REAL			False	False	Unwinder slow-down length (pulses)
UwMarginUsed_m	REAL			False	False	Unwinder margin at slow speed (m)
SqrtArg	REAL			False	False	Square root argument
AutoSlowCmd	BOOL			False	False	Automatic Slow Command
AutoStopCmd	BOOL			False	False	Automatic Stop Command
RwLastValidDiam	REAL			False	False	Rewinder Last valid diameter (mm)
RwRemainMax_Pls	REAL			False	False	Rewinder Remaining length to max diameter (pulses)
UnWinder_Empty	BOOL			False	False	UnWinder Empty
StopFailCnt	INT			False	False	Consecutive failed stop measurements
UwTeachSum	REAL			False	False	Unwinder TEACH CORE sonar sum (mm)
T_MatReset	TON			False	False	v6s: RwMatReset stays ON 3 s
RwAutoMaxSpdAct	REAL			False	False	v6y: active rewinder AUTO max speed (taken over at standstill)
UwEmptyMem	BOOL			False	False	v6x: UnWinder_Empty edge memory
LearnWinLen_Pls	REAL			False	False	v6w: learning window length in encoder pulses
WebRatio	REAL			False	False	V7: drive counts per m/min of web at the current diameter
FFActive	BOOL			False	False	V7: feed-forward loop active
FFWasActive	BOOL			False	False	V7: edge memory
FFTarget	REAL			False	False	V7: reference for the set speed (mm/s of the 0-10 V scale)
RampRef	REAL			False	False	V7: ramped reference (mm/s of the 0-10 V scale)
StepAcc	REAL			False	False	V7: accel step per scan
StepDec	REAL			False	False	V7: decel step per scan
FFSteadyTicks	INT			False	False	V7: scans at target
FFErr	REAL			False	False	V7: relative speed error
Clock_100ms	bool			False	False	Clock100ms

	AlarmCnt	INT			False	False	
	AlarmCntMem	INT			False	False	
	RstEdge	BOOL			False	False	
	RstMem	BOOL			False	False	
	SilenceTicks	INT			False	False	
	WarnCnt	INT			False	False	
	WarnCntMem	INT			False	False	
	WarnBeepTicks	INT			False	False	
	TensUnitMem	BOOL			False	False	
	TensIdeal_N	REAL			False	False	
	TensionSeen	BOOL	False		False	False	Tension reached once since START
	T_TensOn	TON			False	False	Tension ON debounce 0.5 s
	T_TensOff	TON			False	False	Tension OFF debounce 2 s
	T_UwSensFlt	TON			False	False	Unwinder sonar fault debounce 1 s
VAR_EXTERNAL							
	VelSet	REAL				False	Set speed
	OverSpeed	BOOL				False	Speed too high
	SpdDelta_Percent	REAL				False	Speed difference percentage
	Encoder_Pos	DINT				False	Encoder counter
	FinalStopDecel	REAL				False	
	FinalStopRamp	BOOL				False	
	Velocity_M_min	REAL				False	Speed in m/min
	Current_Metre	REAL				False	Current metres
	I_RsetCnt	BOOL				False	Metre counter reset button
	SonarAsk	BOOL				False	
	SonarAskYes	BOOL				False	
	WorkSpd_Max_Set	REAL				False	HMI request: work pot full scale (m/min), applied only with motor stopped
	FinalSpd_Max_Set	REAL				False	HMI request: final pot full scale (m/min), applied only with motor stopped
	SpdMax_Pending	BOOL				False	New pot range waiting for the motor to stop
	RwAxisDist_Prev	REAL				False	Rewinder: RwAxisDist before the last TEACH (for UNDO)
	UwCoreDist_Prev	REAL				False	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
	RwTeachUndo	BOOL				False	HMI: UNDO rewinder TEACH (PLC resets it)
	UwTeachUndo	BOOL				False	HMI: UNDO unwinder TEACH (PLC resets it)
	RwLearnReset	BOOL				False	HMI / Watch: clear learned decels RwStopDecelK, RwStopDecel, RwSlowDecelK (PLC resets it)
	UnderSpeed	BOOL				False	Speed too low
	I_ControlVoltage	BOOL				False	Auxiliaries ON
	I_OverRider_PH	BOOL				False	Roll diameter photocell
	RwCoreDiameter	REAL				False	Rewinder core outside diameter for TEACH CORE (mm)
	UwTeachState	INT				False	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
	SoftStop	BOOL				False	Soft stop in progress (HMI)
	SoftStopEnable	BOOL				False	STOP button = soft stop (False = immediate as before)

SoftStopEndSpd	REAL				False	Soft stop: run command drops below this speed (m/min)
SoftStopDecel	REAL				False	Soft stop decel (mm/s2), 0 = DecelCoarse
RwTeachCore	BOOL				False	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT				False	TEACH CORE: 0 idle 1 teaching 2 done 3 refused
O_MotorStart	BOOL				False	Motor start
RwSlowDecelK	REAL				False	Learned slow-down decel per mm of rewinder diameter (mm/s2 per mm)
RwStopDecelK	REAL				False	Learned stop decel per mm of rewinder diameter (mm/s2 per mm)
I_OR_Position_Sens	BOOL				False	Rider roll position sensor
WorkSpd_Max	REAL				False	Work speed pot full scale (m/min), 50..VelMax
FinalSpd_Max	REAL				False	Final speed pot full scale (m/min), 5..WorkSpd_Max
I_OR_Pressure_Sens	BOOL				False	Rider roll pressure switch
I_OR_Auto_OS	BOOL				False	Rider roll in automatic
I_OR_Cont_OS	BOOL				False	Rider roll in continuous
I_Start_PB	BOOL				False	Start button
I_SafeGuard_Closed	BOOL				False	Side doors closed
I_Motor_Thermal	BOOL				False	Motor thermal switch
I_FanTemp_Ok	BOOL				False	Motor (fan) thermal OK
I_Stop_PB	BOOL				False	Stop button
I_Hub_Position	BOOL				False	Hub/chuck in extraction position
I_FrontGuard_Closed	BOOL				False	Front guard closed
RwMaxSlowFallback	REAL				False	Rewinder max: slow-down diameter while K not learned (mm)
RwMaxSlowAtDiam	REAL				False	Rewinder max: slow-down starts at diameter (mm)
O_Alarm_Array	BOOL				False	Alarm lamp
O_RwUnload	BOOL				False	Unloaders enable - opening
I_Prot_ByPass	BOOL				False	Guards-bypassed selector switch
O_Prot_ByPass_Lamp	BOOL				False	Guards bypassed lamp
O_Brake	BOOL				False	Brake command
Fan_Timeout	REAL				False	Fans stop timeout (from panel)
O_CoolingFan	BOOL				False	Brake and motor fans
O_OR_Auto_Lamp	BOOL				False	Rider roll auto/continuous lamp
O_OverRider_Up	BOOL				False	Raise rider roll
O_OverRider_Hold	BOOL				False	Lock rider roll
Fix10Vdc	INT				False	Fixed 10 Vdc generator
InvRef	INT				False	Inverter speed reference
Encoder_On	BOOL				False	Encoder enable
Encoder_Reset	BOOL				False	Encoder reset
OverRider_TimeOFF	REAL				False	Rider roll photocell OFF timeout (panel)
OverRider_Time	REAL				False	Rider roll photocell ON timeout (

ON						panel)
VelMax	REAL				False	Maximum speed
I_Jog_PB	BOOL				False	Slow speed button
O_Motor_Lamp	BOOL				False	Motor ON lamp
AlarmOn	BOOL				False	Alarm active
BuzzerOn	BOOL				False	Panel buzzer command
I_VFD_Alarm	BOOL				False	Inverter in alarm
Msg	INT				False	Message
Index_B	INT				False	Active alarm index
MotStop	BOOL				False	Motor stopped
Clock1s	BOOL				False	1 s clock
ManMode	BOOL				False	Manual mode
Counter_Reset	BOOL				False	Panel counter reset button
I_FrontGuard_Open	BOOL				False	Front guard open
O_RwLoading_Enable	BOOL				False	Unloaders enable - closing
JogSpd_SP	REAL				False	Speed in manual
O_AccDec2	BOOL				False	Second acc/dec ramp
AccelTrim	REAL				False	Acceleration coefficient in run
DecelTrim	REAL				False	Deceleration coefficient in run
DecelCoarse	REAL				False	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL				False	Deceleration coefficient at start (orig. comment; name suggests accel)
Speed_Display	REAL				False	Set speed (displayed)
O_AcousticSignal	BOOL				False	Horn / acoustic signal
Pot1_Work_Velocity	INT				False	Potentiometer 1 value - Work speed -
O_FrontGuard_ByPass	BOOL				False	Front guard bypass
Pot2_Slow_Velocity	INT				False	Potentiometer 2 value - Final speed -
LowSpd_Display	REAL				False	Slow-down speed (displayed)
Encoder_Pulse	INT				False	Encoder pulses
RwSlowDecelLrn	REAL				False	Learned real slow-down decel (mm/s2) - 0 = not learned
UwSlowMargin_m	REAL				False	Slow-down metres
UwStop_at_M_Val	REAL				False	Total metres
I_AutoStop_OS	BOOL				False	Stop on metre counter
I_Input_11_Spare_1255	BOOL				False	Slow down at end of roll
I_T_one_TensionOK	BOOL				False	Slow-down photocell
I_WebBreak_Monitor	BOOL				False	Paper break monitoring ON
I_WebBreak_PH	BOOL				False	Paper break photocell
I_Vacuum_OK	BOOL				False	Extractor fan thermal OK
CountReset_Req	BOOL				False	Metre counter reset request
Psw	INT				False	Password
I_RsetAlarm	BOOL				False	Alarm reset
EncoderWheel_Circumf	REAL				False	Encoder wheel diameter
AlarmCodeSV	ARRAY[0..32] OF BOOL				False	Panel alarms

SafeGuard_ByPass	BOOL				False	Guards bypassed
FrontGuard_Closed	BOOL				False	Front guard closed
Core_Measure	BOOL				False	(?) Stored core value
UwDiam_Measure	REAL				False	Measured distance
UwCoreDistance	REAL				False	(?) Core measurement
RwSlowFallbackMm	REAL				False	
AI_Diam_Unwinder	INT				False	Analog value - parent roll measurement
RwMatChangePct	REAL				False	
UwCoreDiameter	REAL				False	(?) Shaft diameter
Measure_ON	BOOL				False	(?) Measurement enabled
UwDiam_Act	REAL				False	(?) Measured roll diameter
UwStopDiam_Limit	REAL				False	(?) Stop diameter
UwSlow_at_D_Val	REAL				False	(?) Slow-down diameter
Alarm_Reset	BOOL				False	(?) Alarm reset from panel
HMI_Screen	ARRAY[0..1] OF DINT				False	HMI panel page
Psw22	BOOL				False	(?) Password 22
RwMatReset	BOOL				False	
Psw44	BOOL				False	(?) Password 44
I_Roll_OK	BOOL				False	(?) Roll OK
AI_Diam_Rewinder	INT				False	Rewinder DT35 raw - I/O Map Unit2 Ch2
RwStopEnable	BOOL				False	Rewinder diameter stop enabled
RwTargetDiam	REAL				False	Rewinder final diameter (mm)
RwUseSlow	BOOL				False	Slow down before target
RwSlowMargin_m	REAL				False	Metres at slow speed before stop
UwRawMin	INT				False	
UwRawMax	INT				False	
RwSlowDecel	REAL				False	Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL				False	Reset requests below target - this (mm)
RwDist0V	REAL				False	DT35 distance at 0 V (mm)
RwDist10V	REAL				False	DT35 distance at 10 V (mm)
RwRawMin	INT				False	Raw below = sensor fault
RwRawMax	INT				False	Raw above = sensor fault
RwStopOnFault	BOOL				False	Stop on sensor fault
RwAxisDist	REAL				False	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL				False	Distance filter per 100 ms (0..1)
RwMinPulses	DINT				False	Min pulses in window to learn
RwLearn_K	REAL				False	Material factor filter weight
RwMat_K	REAL				False	Learned material factor
RwLearnStop	BOOL				False	Learn stop decel on every stop
RwMinLearnSpd	REAL				False	Min speed to learn a stop (m/min)
RwStopDecel	REAL				False	Learned stop decel (pulses/s2)
RwTreact	REAL				False	Stop reaction time (s)
RwTstopDef	REAL				False	Stop time until decel learned (s)
RwTrim	REAL				False	Stop length trim (>1 = earlier)

RwDist	REAL				False	Filtered distance (mm)
RwDiam	REAL				False	Rewinder diameter (mm)
RwSpeed	REAL				False	Line speed from encoder (m/min)
RwRemain_m	REAL				False	Metres to target diameter
RwStopLen_m	REAL				False	Stopping length now (m)
RwSlowLen_m	REAL				False	Slow-down length now (m)
RwStop_at_D_Req	BOOL				False	Rewinder diameter STOP request
RwSlow_at_D_Req	BOOL				False	Rewinder diameter SLOW request
RwSensFault	BOOL				False	Rewinder DT35 sensor fault
UwStopEnable	BOOL				False	Unwinder metre-counter stop ON (HMI)
SlowAtDisp	REAL				False	HMI: slow-down starts at (m or mm, see SlowAtMode)
UwSlow_at_D_Req	BOOL				False	
UwSlowDiamAct	REAL				False	
RwJogMaxDiam	REAL				False	
JogInhibit	BOOL				False	
SCurveTime	REAL				False	
TOneDecel	BOOL				False	
HornStartBeep_s	REAL				False	
HornBeepLen_s	REAL				False	
HornBeepPeriod	REAL				False	
HornStopBeep_s	REAL				False	
HornTest	BOOL				False	
RwLockDiam	REAL				False	
UwSensTest	BOOL				False	
RwLockMinGrow_mm	REAL				False	
RwMatLocked	BOOL				False	
LongThick_mm	REAL				False	
RwDiamRev	REAL				False	
RwWave_mm	REAL				False	
UwStopLenShow	BOOL				False	
UwSlowDiamShow	BOOL				False	
SoftCancelAccel	REAL				False	
SpdHold	BOOL				False	
SlowAtMode	INT				False	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL				False	HMI: show slow-down display
UwSlow_at_M_Val	REAL				False	Unwinder: slow-down starts at (m)
UwSlow_at_M_Req	BOOL				False	Unwinder metre slow-down request (latched)
RwSlow_at_D_Val	REAL				False	Rewinder: slow-down starts at diameter (mm)
RwStop_at_D_Val	REAL				False	Rewinder: stop command at diameter (mm)
RwMaxDiam	REAL				False	Rewinder max diameter protection (mm)
RwMaxStop	BOOL				False	Rewinder max diameter STOP (latched)
RwMaxSlow	BOOL				False	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL				False	

	RwThickMin_mm	REAL				False	Learning: min plausible paper thickness (mm)
	RwThickMax_mm	REAL				False	Learning: max plausible paper thickness (mm)
	RwDecelMin_mm	REAL				False	Learning: min plausible stop decel (mm/s2)
	RwDecelMax_mm	REAL				False	Learning: max plausible stop decel (mm/s2)
	RwLearnFailMax	INT				False	Learning: failed windows before alarm (12 = 2 min)
	RwStopFailMax	INT				False	Learning: failed stops before alarm
	GSM_Learned_OK	BOOL				False	Rewinder learning fault (alarm 16)
	RwThick_mm	REAL				False	Learned paper thickness (mm)
	Psw33	BOOL				False	
	RwAutoMaxSpd	REAL				False	
	RwWinLen_m	REAL				False	
	RwWinGrow_mm	REAL				False	
	GsmAsk	BOOL				False	
	GsmAskYes	BOOL				False	
	DecelWeb	REAL				False	
	AccelWeb	REAL				False	
	RwRatioD	REAL				False	
	FFEnable	BOOL				False	
	UwStopDiamShow	BOOL				False	
	UwStop_at_D_Val	REAL				False	
	Clock10ms	BOOL				False	
	Sec_to_Min	REAL				False	
	Psw48	BOOL				False	
	JogRun	BOOL				False	
	GuardStopOK	BOOL				False	V11: stopped for real -> guard bypass allowed (Output 13)
	SlowDown	BOOL				False	Slow down
	WarningOn	bool				False	
	AlarmSilenced	BOOL				False	
	PaperGSM	REAL				False	
	WebWidth_mm	REAL				False	
	TensFactor	REAL				False	
	TensShowKg	BOOL				False	
	TensShowLb	BOOL				False	
	TensShowN	BOOL				False	
	TensIdeal_Disp	REAL				False	
	TensUnit	INT				False	
	TensUnitPB	BOOL				False	
	TensCheckEnable	BOOL				False	
	TensLostWarn	BOOL				False	
	TensWaitSpd	REAL				False	
	TensWait	BOOL				False	
	TensionOK	BOOL				False	
	UwSensFault	BOOL				False	
	Thickness_Fault	BOOL				False	

1-8-1-1-2.General

0 Remote 10.146.233.11 ... 1s clock.



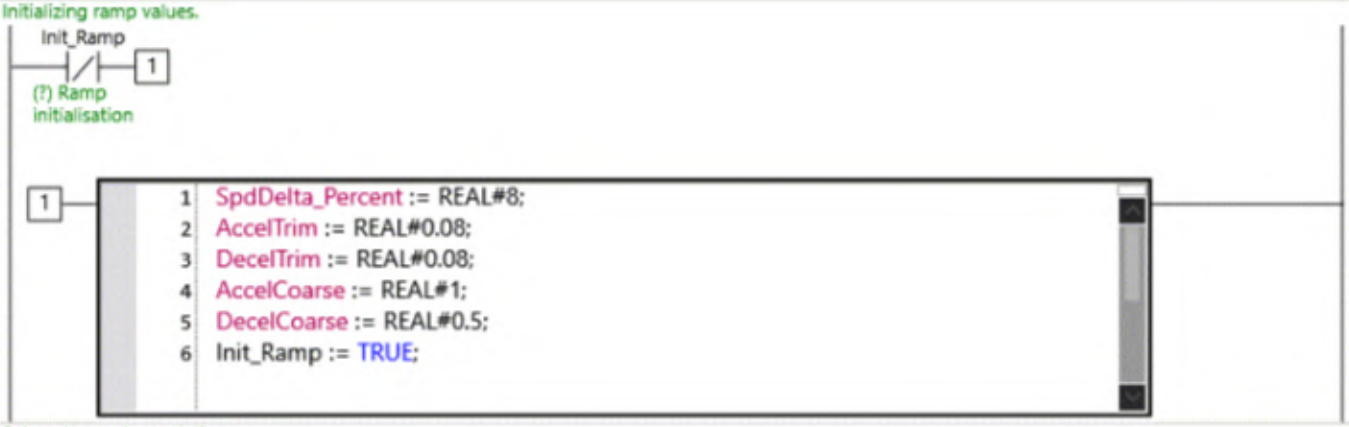
1 100ms clock.



2 10ms clock.



3 Initializing ramp values.



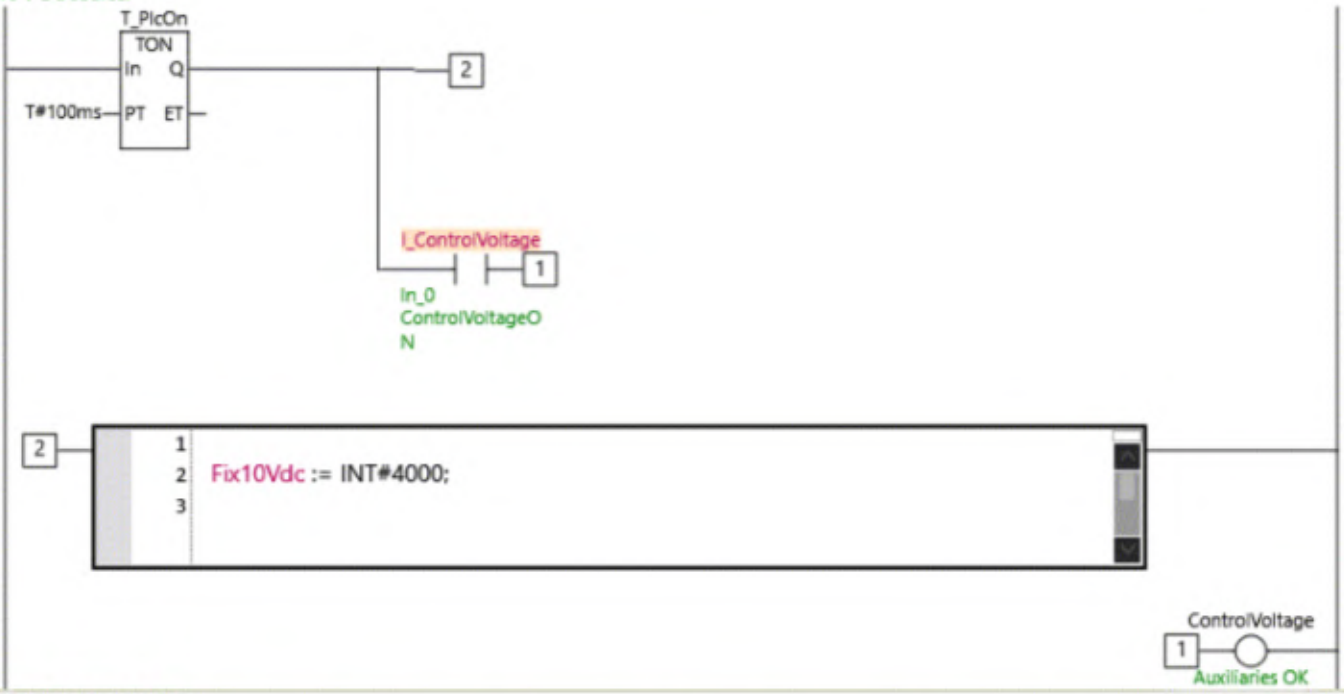
4 Set analog scale resolution.



5 Variable for converting m/min to mm/sec and vice versa.



6 10 V DC source.



7 START blocked by JOG

```
// ==== V11: REPLACE the code of General rung 7 ("START blocked by JOG", printed V9).
// START push-button is ignored while the JOG (slow speed) button is held and for
// 2 s after it is released. A START held during the block must be released and
// pressed again (no start by itself when the 2 s are over).
// Without this: START pressed during a jog sets Slow_Inhibit -> releasing the jog
// no longer stops (General rung 11) and VelSet jumps to the work speed.

T_JogStartBlk(In := I_Jog_PB, PT := T#2s);      // TOF: ON while held + 2 s
StartHeldBlk := (StartHeldBlk OR (I_Start_PB AND T_JogStartBlk.Q)) AND I_Start_PB;
StartPB_OK := I_Start_PB AND NOT T_JogStartBlk.Q AND NOT StartHeldBlk;

// V10: JogRun = a JOG really running: JOG button held, NOT a normal START run
// (Slow_Inhibit = normal run, set by START) and NOT a soft stop. Used instead of
// the raw I_Jog_PB for: guard bypass (Output 13), 2nd drive ramp (Output 12),
// pot reading (Motor_Manager 8), feed-forward (Motor_Manager 14/17), AutoStop learning.
// Without it, holding JOG during a 500 m/min run bypassed the front gate / lateral
// doors safety (23KA4 // 5SA1) and switched the drive to its 2nd ramp.

JogRun := I_Jog_PB AND NOT Slow_Inhibit AND NOT SoftStop;

// Guard bypass (Output_Manager rung 13, 23KA4 // 5SA1): ON when the run command is
// off (machine stopped by the operator).

GuardStopOK := NOT StartMotor_Cmd;
```

8

```
// ==== General: NEW ST rung (no contact), directly BELOW rung 7 (JogRun / GuardStopOK)
// ==== Comment: "T-one Alarm 0: tension established"
// T-one: F.9 = 0 (alarm when tension ABOVE threshold), F.10 = threshold (kg), F.11 = full scale
// Relay pins 15-16 (Alarm 0): +24 V -> pin 15, pin 16 -> Inline IB IL 24 DI 8 IN05, pin 1.3
// I_T_one_TensionOK = node#[1.0] _IN05 (Pin 1.3) (was I_SlowSens_PH, used in no rung)
// Relay closed = tension > F.10 (not latched). Broken wire / T-one off = "no tension" (fail-safe).

T_TensOn(In := I_T_one_TensionOK, PT := T#500ms);       // stable 0.5 s
T_TensOff(In := NOT I_T_one_TensionOK, PT := T#2s);     // ignore short dips (splice, wave, chatter)

IF T_TensOn.Q OR NOT TensCheckEnable THEN               // HMI switch OFF = old behaviour
    TensionOK := TRUE;
ELSIF T_TensOff.Q THEN
    TensionOK := FALSE;
END_IF;

// Latch: tension reached once since START

IF NOT StartMotor_Cmd THEN
    TensionSeen := FALSE;
ELSIF TensionOK THEN
    TensionSeen := TRUE;
END_IF;

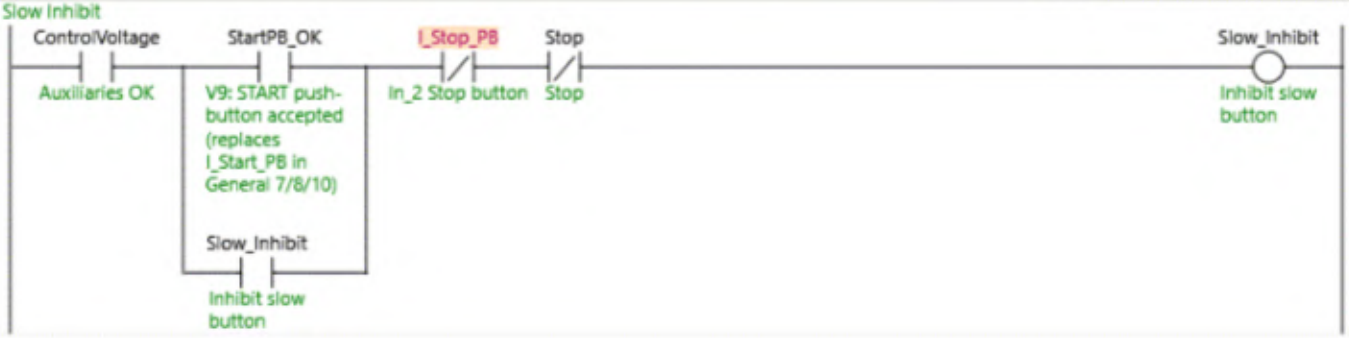
// After START: hold at TensWaitSpd until the tension is there (JOG not affected)

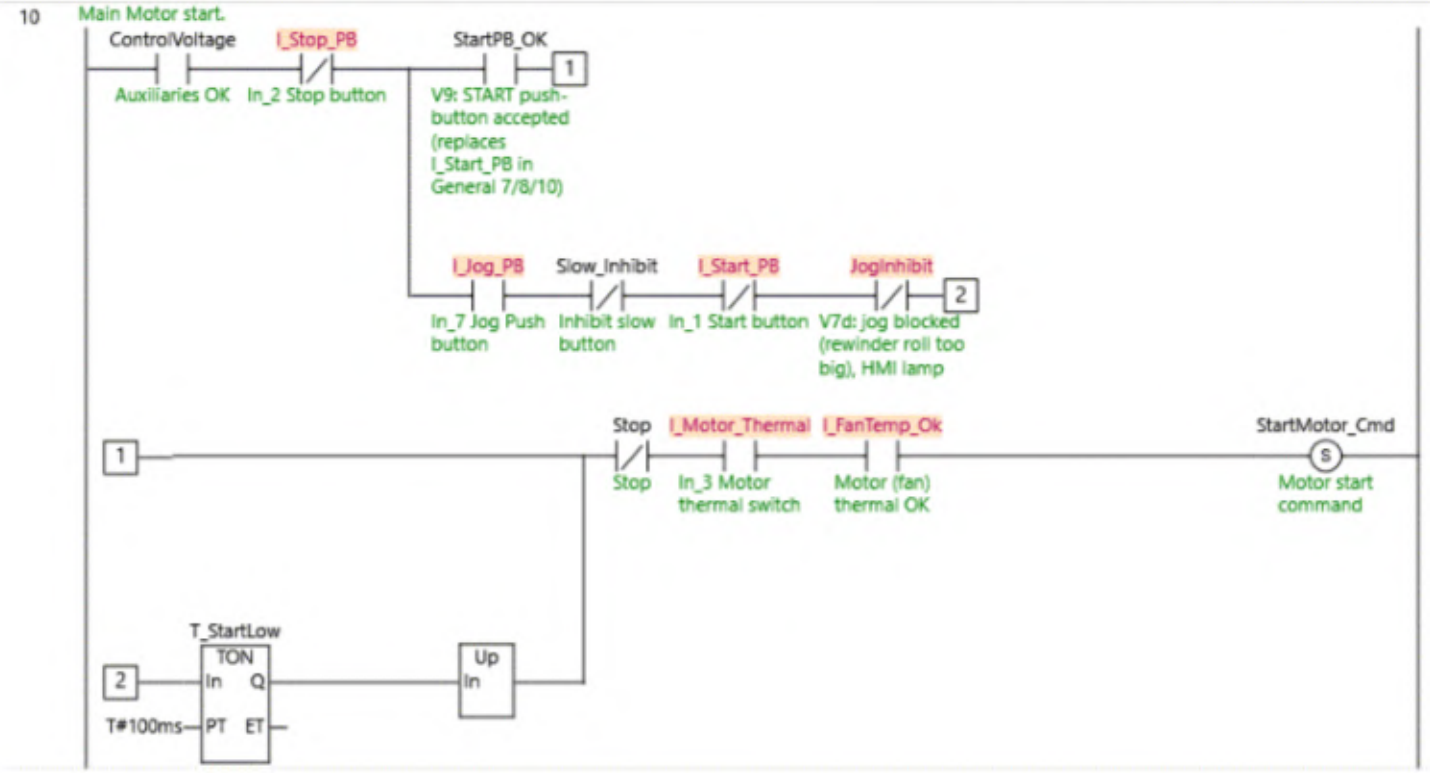
TensWait := StartMotor_Cmd AND NOT TensionSeen AND NOT JogRun;

// Tension lost later while running -> warning only (no slow-down)

TensLostWarn := StartMotor_Cmd AND TensionSeen AND NOT TensionOK;
```

9 Slow Inhibit




10
Main Motor start.
ControlVoltage
Auxiliaries OK
I_Stop_PB
In_2 Stop button
StartPB_OK
1
V9: START push-button accepted (replaces I_Start_PB in General 7/8/10)
I_Jog_PB
In_7 Jog Push button
Slow_Inhibit
Inhibit slow button
I_Start_PB
In_1 Start button
JogInhibit
2
V7d: jog blocked (rewinder roll too big), HMI lamp
1
Stop
Stop
I_Motor_Thermal
In_3 Motor thermal switch
I_FanTemp_Ok
Motor (fan) thermal OK
StartMotor_Cmd
S
Motor start command
T_StartLow
TON
2
In
Q
T#100ms
PT
ET
Up
In

11 Soft Stop Process

```
// ==== V12: REPLACE the code of General rung 10 ("Soft Stop Process", printed V9).
// v6m: STOP button = soft stop. StopNow replaces I_Stop_PB in rung 9.
// V12: an end-of-roll stop (FinalStopRamp, AutoStop section 11b) also starts the
// soft stop, at FinalStopDecel (Motor_Manager rung 17); START cannot cancel it.

StopNow := FALSE;
T_StopLong(In := I_Stop_PB, PT := T#1s);

IF NOT StartMotor_Cmd THEN
    SoftStop := FALSE;
ELSIF I_Stop_PB AND NOT StopPB_Mem THEN          // STOP pressed (edge)

    IF SoftStop OR NOT SoftStopEnable OR I_Jog_PB
      OR (Velocity_M_min <= SoftStopEndSpd) THEN
        StopNow := TRUE;                         // 2nd press / jog / slow / disabled
    ELSE
        SoftStop := TRUE;                        // start soft stop
    END_IF;
END_IF;

StopPB_Mem := I_Stop_PB;

IF T_StopLong.Q THEN
    StopNow := TRUE;                             // held 1 s = immediate
END_IF;

// V12: end-of-roll stop ramp (FinalStopRamp set by AutoStop section 11b)

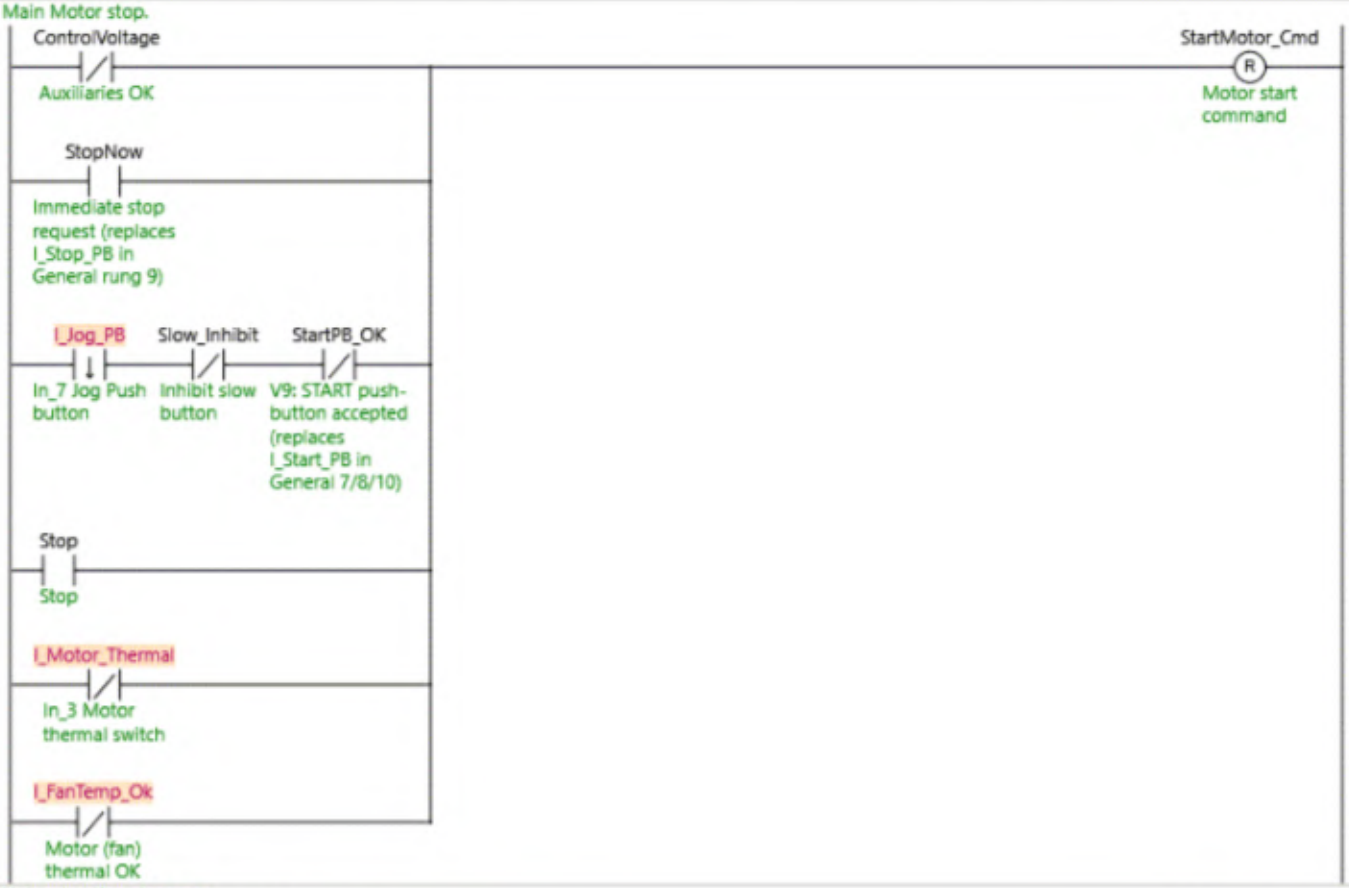
IF FinalStopRamp AND StartMotor_Cmd AND NOT SoftStop THEN
    SoftStop := TRUE;
END_IF;

IF SoftStop AND I_Start_PB AND NOT FinalStopRamp THEN
    SoftStop := FALSE;                           // START cancels the soft stop
END_IF;

T_SoftStop(In := SoftStop, PT := T#120s);

IF SoftStop AND ((Velocity_M_min <= SoftStopEndSpd) OR T_SoftStop.Q) THEN
    StopNow := TRUE;                             // almost stopped (or timeout)
END_IF;
```

12 Main Motor stop.



13 Hub extraction clearance.



14 Brake control.

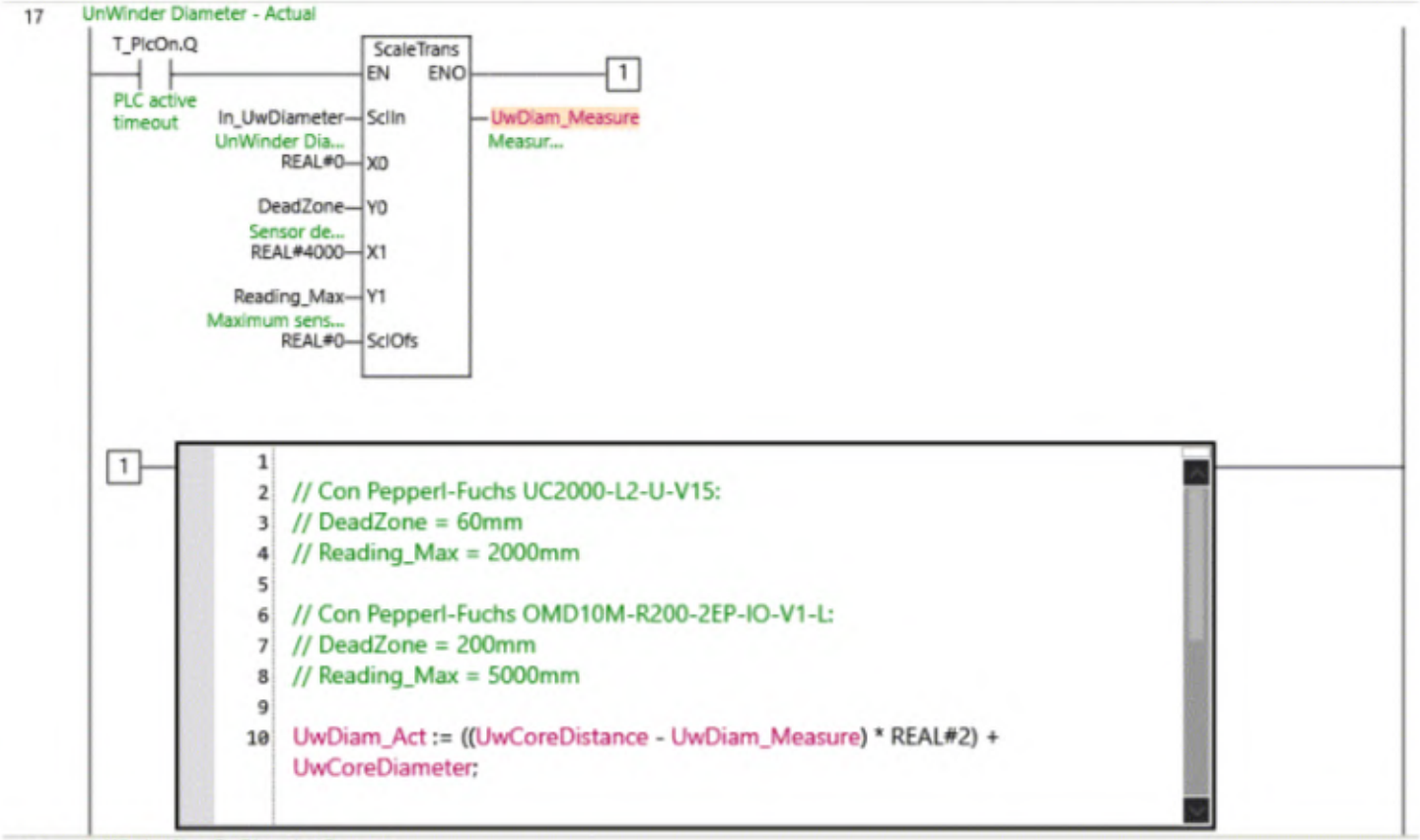


15 Cooling Fan control.



16 UnWinder Diameter - Input

```
1  In_UwDiameter := INT_TO_REAL(AI_Diam_Unwinder)/REAL#1;
```


17
UnWinder Diameter - Actual
T_PlcOn.Q
PLC active timeout
ScaleTrans
EN
ENO
1
In_UwDiameter
UnWinder Dia...
REAL#0
SclIn
X0
DeadZone
Sensor de...
REAL#4000
Y0
X1
Reading_Max
Maximum sens...
REAL#0
Y1
SclOfs
UwDiam_Measure
Measur...
1
// Con Pepperl-Fuchs UC2000-L2-U-V15:
// DeadZone = 60mm
// Reading_Max = 2000mm
// Con Pepperl-Fuchs OMD10M-R200-2EP-IO-V1-L:
// DeadZone = 200mm
// Reading_Max = 5000mm
UwDiam_Act := ((UwCoreDistance - UwDiam_Measure) * REAL#2) + UwCoreDiameter;
18
UnWinder Slow By Diameter Compare
I_ControlVoltage
Measure_ON
In_0 ControlVoltageON
Measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwSlowDiamAct
V7c: unwind...
In2
SlowByDiam
(?) Slow down by diameter
19
UnWinder Stop By Diameter Compare
I_ControlVoltage
Measure_ON
In_0 ControlVoltageON
Measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwStop_at_D_Val
V11: unwinder...
In2
UwStopByDiam
Stop by diameter

1-8-1-1-3.Motor_Manager

0 Enable encoder reading.



1 Calculation of mm per encoder pulse.

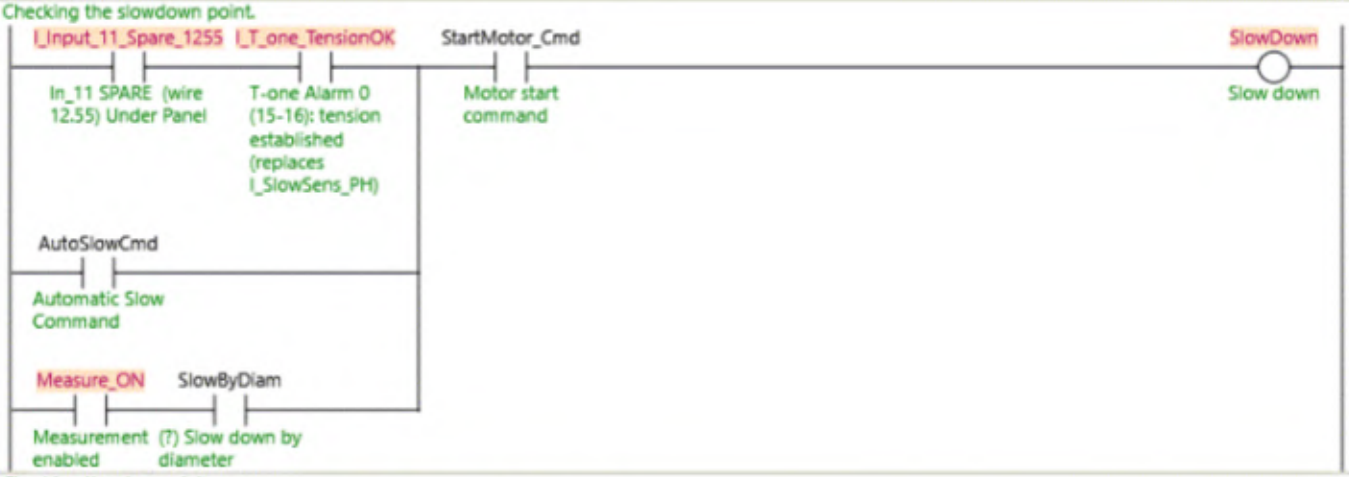
```
IF Encoder_Pulse = INT#0 THEN;
    Encoder_Pulse :=INT#1;
END_IF;
// Encoder Standard 400mm
IF  EncoderWheel_Circumf = REAL#0 THEN;
    EncoderWheel_Circumf := REAL#400;
END_IF;

EncoderPulse_to_mm := EncoderWheel_Circumf / (INT_TO_REAL(Encoder_Pulse) *
REAL#4);
```

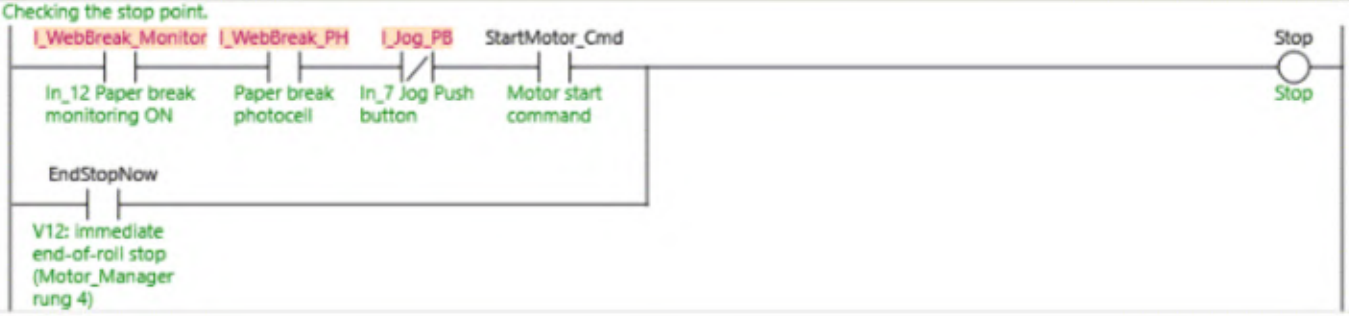
2 Reset encoder counter.



3 Checking the slowdown point.



4 Checking the stop point.



5 Check the maximum and minimum values for working speed and slow speed.

```
IF VelMax < REAL#600 THEN;
      VelMax := REAL#600;
   ELSIF VelMax > REAL#1000 THEN;
      VelMax := REAL#1000;
END_IF;
Vel_Max_mms := VelMax * REAL#1000 / REAL#60;

IF JogSpd_SP < REAL#10 THEN;
      JogSpd_SP := REAL#10;
ELSIF JogSpd_SP > REAL#70 THEN;
   JogSpd_SP := REAL#70;
END_IF;

// v6y: rewinder AUTO max speed, 0 = no limit

IF RwAutoMaxSpd > VelMax THEN
   RwAutoMaxSpd := VelMax;
ELSIF (RwAutoMaxSpd > REAL#0) AND (RwAutoMaxSpd < REAL#50) THEN
   RwAutoMaxSpd := REAL#50;
END_IF;

IF NOT StartMotor_Cmd THEN          // taken over at standstill only
   RwAutoLimAct := RwStopEnable AND (RwAutoMaxSpd > REAL#0);
   RwAutoMaxSpdAct := RwAutoMaxSpd;
END_IF;

// ---- V7: add at the END of Motor_Manager rung 5 ("Check the maximum and
minimum values...")
// ---- after the JogSpd_SP clamp, before Output_Ratio. Same style as the VelMax /
JogSpd clamps.
// V7: limits on settings that stop the machine or break the auto stop if 0 / absurd

IF SpdDelta_Percent < REAL#2 THEN              // fine band [m/min]: 0 = speed
hunting
   SpdDelta_Percent := REAL#2;
ELSIF SpdDelta_Percent > REAL#15 THEN
   SpdDelta_Percent := REAL#15;
END_IF;

IF AccelTrim < REAL#0.02 THEN                  // trim ramps: 0 = no speed correction
   AccelTrim := REAL#0.02;
ELSIF AccelTrim > REAL#0.5 THEN
   AccelTrim := REAL#0.5;
END_IF;

IF DecelTrim < REAL#0.02 THEN
   DecelTrim := REAL#0.02;
ELSIF DecelTrim > REAL#0.5 THEN
   DecelTrim := REAL#0.5;
END_IF;

IF AccelCoarse < REAL#0.2 THEN                 // coarse ramps: 0 = never reaches
speed
   AccelCoarse := REAL#0.2;
ELSIF AccelCoarse > REAL#4 THEN
   AccelCoarse := REAL#4;
END_IF;
```

```
IF DecelCoarse < REAL#0.2 THEN
    DecelCoarse := REAL#0.2;
ELSIF DecelCoarse > REAL#4 THEN
    DecelCoarse := REAL#4;
END_IF;

IF DecelWeb > REAL#0 THEN                    // V7 web ramps: 0 = old coarse ramp (allowed)
    IF DecelWeb < REAL#50 THEN
        DecelWeb := REAL#50;
    ELSIF DecelWeb > REAL#1000 THEN
        DecelWeb := REAL#1000;
    END_IF;
END_IF;

IF AccelWeb > REAL#0 THEN
    IF AccelWeb < REAL#50 THEN
        AccelWeb := REAL#50;
    ELSIF AccelWeb > REAL#1000 THEN
        AccelWeb := REAL#1000;
    END_IF;
END_IF;

IF SoftStopDecel > REAL#1500 THEN            // 0 = DecelCoarse (allowed)
    SoftStopDecel := REAL#1500;
END_IF;

IF RwMaxDiam > REAL#1450 THEN                // rewinder MECHANICAL limit [mm]
    RwMaxDiam := REAL#1450;
ELSIF RwMaxDiam < REAL#300 THEN
    RwMaxDiam := REAL#300;
END_IF;

IF RwMaxSlowFallback >= RwMaxDiam THEN       // fallback slow-down below the max
    RwMaxSlowFallback := RwMaxDiam - REAL#30;
END_IF;

// ---- V9 (V7d): add at the END of Motor_Manager rung 5, after the V7 clamps
// ---- (last lines now: RwMaxSlowFallback clamp), before Output_Ratio / Input_Ratio.
// V7d: jog / slow start button blocked on a big rewinder roll.
// RwJogMaxDiam (mm, HMI, Retain): 0 = no limit, else min 150.
// DT35 fault -> jog allowed (needed to thread / clear the machine).

IF (RwJogMaxDiam > REAL#0) AND (RwJogMaxDiam < REAL#150) THEN
    RwJogMaxDiam := REAL#150;
END_IF;

JogInhibit := (RwJogMaxDiam > REAL#0) AND (RwDiam > RwJogMaxDiam) AND NOT RwSensFault;

Output_Ratio := (VelMax * Mt_To_mm) / UINT_TO_REAL(Analog_Res): //REAL#4000;
Input_Ratio := UINT_TO_REAL(Analog_Res) / VelMax ;
```

6 Reading the current value of the potentiometers. v6n: requested pot ranges, limited, applied only with the motor stopped !

```
IF WorkSpd_Max_Set < 50.0 THEN
    WorkSpd_Max_Set := 50.0;
ELSIF WorkSpd_Max_Set > VelMax THEN
    WorkSpd_Max_Set := VelMax;
END_IF;

IF FinalSpd_Max_Set < 5.0 THEN
    FinalSpd_Max_Set := 5.0;
ELSIF FinalSpd_Max_Set > WorkSpd_Max_Set THEN
    FinalSpd_Max_Set := WorkSpd_Max_Set;
END_IF;

IF NOT StartMotor_Cmd THEN
    WorkSpd_Max := WorkSpd_Max_Set;          // motor stopped: apply
    FinalSpd_Max := FinalSpd_Max_Set;
END_IF;

SpdMax_Pending := (WorkSpd_Max <> WorkSpd_Max_Set)
            OR (FinalSpd_Max <> FinalSpd_Max_Set)
            OR (RwAutoLimAct <> (RwStopEnable AND (RwAutoMaxSpd > REAL#0)))
            OR (RwAutoLimAct AND (RwAutoMaxSpdAct <> RwAutoMaxSpd));

// v6i: pots scaled to their own full scale (m/min)

IF Pot1_Work_Velocity > INT#5 THEN
    Pot1_Mem := REAL_TO_INT(INT_TO_REAL(Pot1_Work_Velocity) * WorkSpd_Max /
VelMax);
ELSE
    Pot1_Mem := INT#0;
END_IF;

IF Pot2_Slow_Velocity > INT#5 THEN
    Pot2_Mem := REAL_TO_INT(INT_TO_REAL(Pot2_Slow_Velocity) * FinalSpd_Max /
VelMax);
ELSE
    Pot2_Mem := INT#0;
END_IF;
```

7 Updating the set speed.

```
IF ManMode = FALSE THEN;

    Speed_Display := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;
    LowSpd_Display := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;

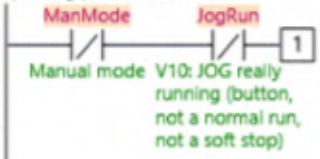
  // v6y: rewinder AUTO stop -> work speed limited

  IF RwAutoLimAct AND (Speed_Display > RwAutoMaxSpdAct) THEN
     Speed_Display := RwAutoMaxSpdAct;
  END_IF;

ELSE;
  Speed_Display := Pot1_Work_Velocity / (Analog_Res / UINT#40); // v6i: raw pot. %
  LowSpd_Display := Pot2_Slow_Velocity / (Analog_Res / UINT#40);

END_IF;
```

8 Updating potentiometer value.



1

```
IF SoftStop THEN
   VelSet := 0.0;                    // v6m: soft stop
ELSIF SlowDown = FALSE  THEN
   VelSet := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;

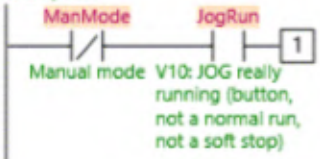
   // v6y: rewinder AUTO stop -> work speed limited

   IF RwAutoLimAct AND (VelSet > RwAutoMaxSpdAct) THEN
      VelSet := RwAutoMaxSpdAct;
   END_IF;

ELSE
   VelSet := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;
END_IF;

VelSet_mms := VelSet * Mt_To_mm;
```

9 Slow speed.



1

```
VelSet := JogSpd_SP;
VelSet_mms := VelSet * Mt_To_mm;
```

10 Check the difference between the set speed and the detected speed.

```
//SpdDelta := VelSet_mms * SpdDelta_Percent / REAL#100;

SpdDelta := SpdDelta_Percent;
SpdDeltaMax := VelSet + SpdDelta;
SpdDeltaMin := VelSet - SpdDelta;

// Too fast

IF Velocity_M_min > SpdDeltaMax THEN;
    OverSpeed := TRUE;
ELSE
    OverSpeed := FALSE;
END_IF;

// Too slow

IF Velocity_M_min < SpdDeltaMin THEN;
    UnderSpeed := TRUE;
ELSE
    UnderSpeed := FALSE;
END_IF;
```

11 Speed calculation.

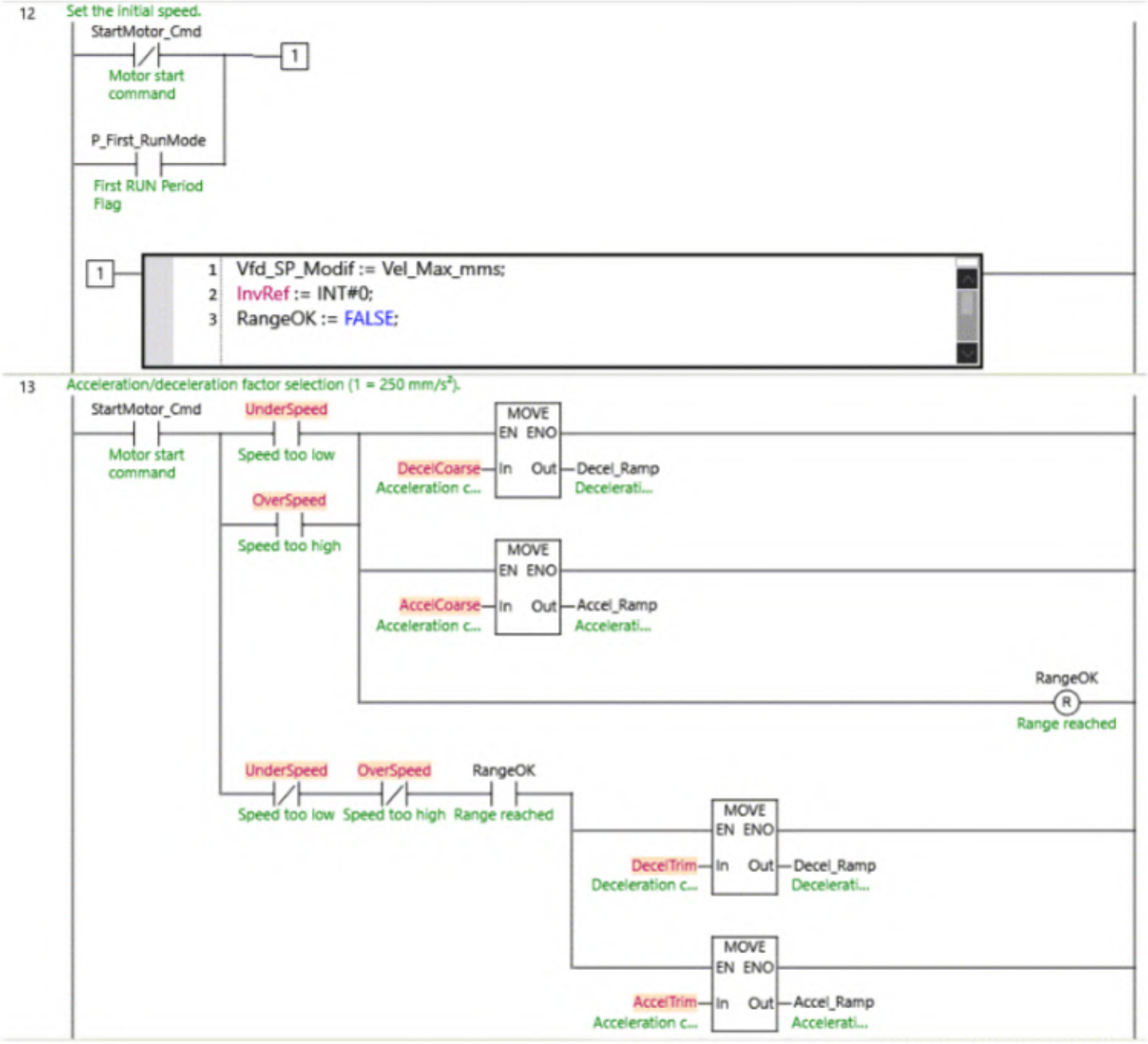
```
// ==== V9: REPLACE Motor_Manager rungs 11 ("Read the current value of the encoder", MOVE on
// ==== Clock100ms rising edge) and 12 ("Speed calculation", Clock100ms falling edge) by ONE
// ==== ST rung with this code (no contact in front). Comment: "V9: speed over exactly 100 ms".
// Old: window = time between the rising and falling edge of Clock100ms, assumed 50 ms,
// but with the 4 ms PrimaryTask it is 12 or 13 scans (48 / 52 ms) -> the speed jumps
// -4 % / +4 % (418 set: display 397 / 431).
// New: exactly 25 scans = 100 ms (same as AutoStop SpdAct_Pps), updated 10 times/s.

SpdScanCnt := SpdScanCnt + 1;

IF SpdScanCnt >= 25 THEN
    SpdScanCnt := 0;
    DeltaPulse := Encoder_Pos - MemPos;
    MemPos := Encoder_Pos;

    IF ABS(DeltaPulse) > 100000 THEN            // counter reset (Encoder_Reset)
        DeltaPulse := 0;
    END_IF;

    Velocity := DINT_TO_REAL(DeltaPulse) * EncoderPulse_to_mm * REAL#10;   // mm/s
    Velocity_M_min := Velocity / Mt_To_mm;
END_IF;
```


12
Set the initial speed.
StartMotor_Cmd
Motor start command
1
P_First_RunMode
First RUN Period Flag
1
1 Vfd_SP_Modif := Vel_Max_mms;
2 InvRef := INT#0;
3 RangeOK := FALSE;
13
Acceleration/deceleration factor selection (1 = 250 mm/s²).
StartMotor_Cmd
Motor start command
UnderSpeed
Speed too low
OverSpeed
Speed too high
MOVE
EN ENO
DecelCoarse
Acceleration c...
In Out
Decel_Ramp
Decelerati...
MOVE
EN ENO
AccelCoarse
Acceleration c...
In Out
Accel_Ramp
Accelerati...
RangeOK
R
Range reached
UnderSpeed
Speed too low
OverSpeed
Speed too high
RangeOK
Range reached
MOVE
EN ENO
DecelTrim
Deceleration c...
In Out
Decel_Ramp
Decelerati...
MOVE
EN ENO
AccelTrim
Acceleration c...
In Out
Accel_Ramp
Accelerati...

14 Ratio drive counts per m/min of web at the current rewinder diameter.

```
// ==== V10: REPLACE the code of Motor_Manager rung 14 (printed V9) ("Ratio drive counts per m/min of web
// ==== at the current rewinder diameter." = V7 rung A) with this file. (V7b jog line inside.)
// Ratio drive counts per m/min of web at the current rewinder diameter.
// The motor drives the rewinder: web speed per Hz grows with the diameter, so
// counts per m/min = RwRatioD / diameter. RwRatioD (Retain) is adapted by rung B.

IF (RwRatioD > 0.0) AND (DecelDiam >= 100.0) THEN
    WebRatio := RwRatioD / DecelDiam;
ELSE
    WebRatio := 0.0;
END_IF;

// Fallback only (FFEnable = FALSE): old integrating loop (rung 16) with the coarse
// ramps in real web mm/s2. With FFEnable = TRUE, rung B drives the reference.
// V7b: never in jog -> jog keeps the rung 13 ramps exactly as v6y. V10: JogRun (jog really running).

IF NOT FFEnable AND NOT JogRun AND StartMotor_Cmd AND (UnderSpeed OR OverSpeed)
   AND (WebRatio > 0.0) THEN

    IF DecelWeb > 0.0 THEN
        Decel_Ramp := DecelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
        IF Decel_Ramp > 8.0 THEN
            Decel_Ramp := 8.0;
        END_IF;
    END_IF;

    IF AccelWeb > 0.0 THEN
        Accel_Ramp := AccelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;

        IF Accel_Ramp > 8.0 THEN
            Accel_Ramp := 8.0;
        END_IF;
    END_IF;
END_IF;
```

15 Soft Stop - SoftStopDecel (mm/s2), same for every roll.

```
// v6r: soft stop at a REAL web decel SoftStopDecel (mm/s2), same for every roll.
// Ratio drive counts / web m/min frozen at the soft stop start (encoder = exact).

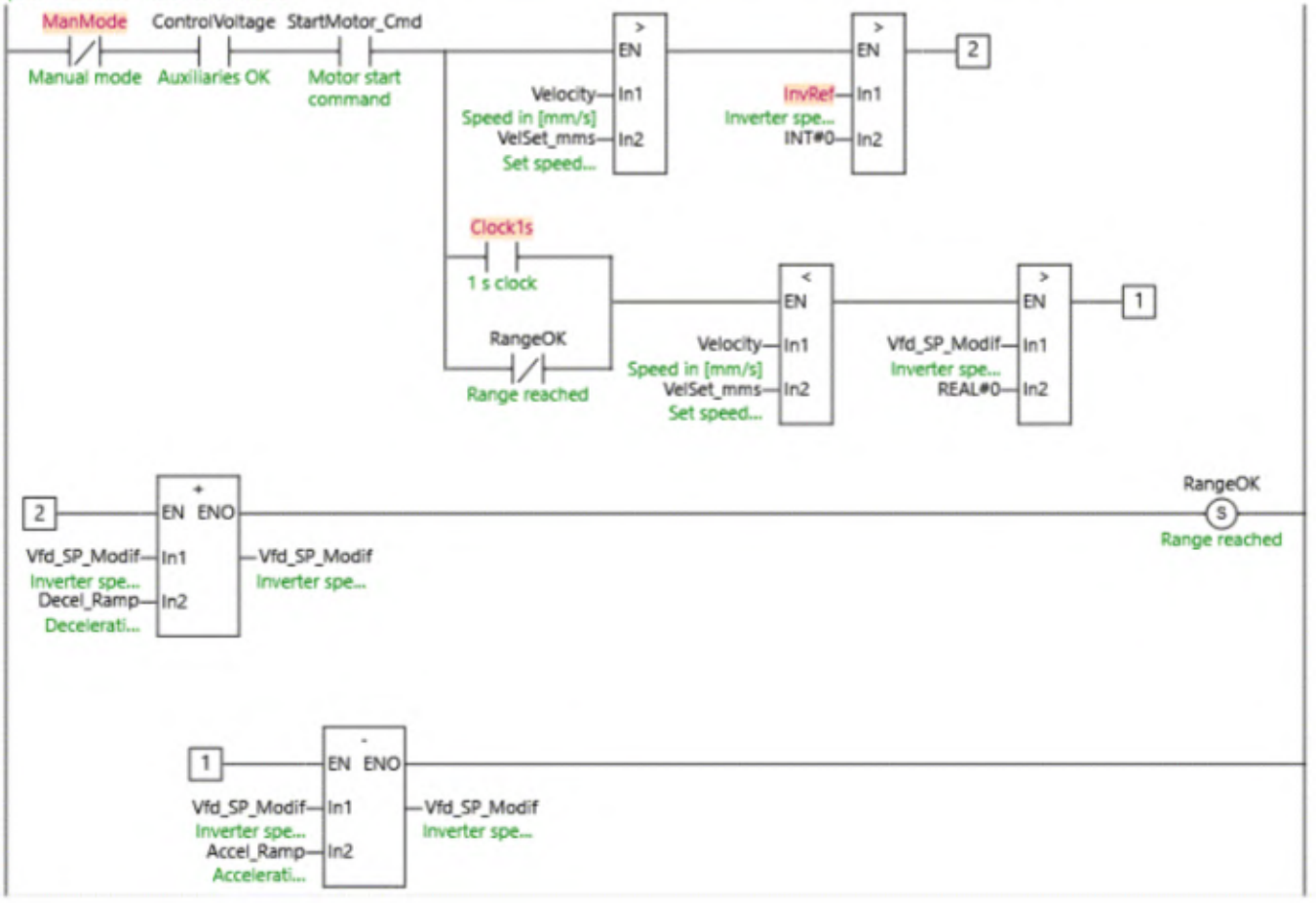
IF SoftStop AND NOT SoftStopMem THEN
    IF Velocity_M_min > 1.0 THEN
        SoftStopRatio := INT_TO_REAL(InvRef) / Velocity_M_min;
    ELSE
        SoftStopRatio := 0.0;
    END_IF;
END_IF;

SoftStopMem := SoftStop;

IF SoftStop AND (SoftStopDecel > 0.0) THEN
    IF SoftStopRatio > 0.0 THEN
        Decel_Ramp := SoftStopDecel * 0.06 * SoftStopRatio * Vel_Max_mms /
1000000.0;
    ELSE
        Decel_Ramp := SoftStopDecel / 250.0;          // fallback (old v6m behaviour)
    END_IF;

    IF Decel_Ramp > 8.0 THEN
        Decel_Ramp := 8.0;                    // max 9 Hz/s asked from the PLC
    END_IF;
END_IF;
```

16 Speed correction factor calculation.



17 Slow adaptation of RwRatioD on the relative speed error.

```
// ==== V12: REPLACE the code of Motor_Manager rung 17 (printed V9) ("Slow adaptation of RwRatioD on the
// ==== relative speed error." = V7 rung B) with this file. (V7a + V7b + V7f + V9 + V10 + V12 inside.)
// The PLC sends the drive STRAIGHT to the frequency of the set speed
// (VelSet x WebRatio), along a ramp in REAL web mm/s2 that does not depend on the
// measured speed -> no hunting, whatever the roll inertia / web delay.
// The measured speed only adapts RwRatioD slowly (~10 s), once at target.
// It overwrites Vfd_SP_Modif after rung 16, so rung 16 has no effect in AUTO.
// FFEnable = FALSE -> old loop (rung 16) exactly as before.
// V7b: not in jog (V10: JogRun = jog really running): at 10 m/min the feed-forward gives
// only ~0.4 Hz, too little to move the web -> the old loop (rung 16) runs the jog.

FFActive := FFEnable AND NOT ManMode AND NOT JogRun AND StartMotor_Cmd
        AND ControlVoltage AND (WebRatio > 0.0) AND (Vel_Max_mms > 0.0);

IF NOT StartMotor_Cmd THEN
    RampRef := 0.0;
    FFRate := 0.0;
    FFSteadyTicks := 0;
    FFSlowAcc := FALSE;
END_IF;

// V9: START during a soft stop: the decel flattens and the speed is HELD
// (SpdHold, HMI lamp) until START is pushed AGAIN -> gentle ramp up at
// SoftCancelAccel (mm/s2) to the set speed. The hold only blocks speeding up:
// a lower target (pot down, auto slow-down, RwAutoMaxSpd) still slows down.
// STOP during the hold = new soft stop (General rung 9).

IF (SoftCancelAccel > 0.0) AND (SoftCancelAccel < 20.0) THEN
    SoftCancelAccel := 20.0;
ELSIF SoftCancelAccel > 1000.0 THEN
    SoftCancelAccel := 1000.0;
END_IF;

FFStartEdge := I_Start_PB AND NOT FFStartMem;
FFStartMem := I_Start_PB;

IF FFSoftMem AND NOT SoftStop AND StartMotor_Cmd AND FFActive THEN   // soft stop cancelled
    SpdHold := TRUE;
    FFSlowAcc := FALSE;
ELSIF SpdHold AND FFStartEdge THEN                  // 2nd START: resume
    SpdHold := FALSE;
    FFSlowAcc := TRUE;
END_IF;

IF SoftStop OR NOT StartMotor_Cmd OR NOT FFActive THEN    // FFEnable OFF: old loop
    SpdHold := FALSE;
    FFSlowAcc := FALSE;
END_IF;

FFSoftMem := SoftStop;

// V7f: S-curve time (s). 0 = off. max 2 s
```

```
53
54  IF SCurveTime < 0.0 THEN
55      SCurveTime := 0.0;
56  ELSIF SCurveTime > 2.0 THEN
57      SCurveTime := 2.0;
58  END_IF;
59
60  IF FFActive THEN
61      IF NOT FFWasActive THEN
62          RampRef := Vel_Max_mms - Vfd_SP_Modif;          // bumpless start / switch-
    over
63          FFRate := 0.0;
64      END_IF;
65
66      // Target reference (mm/s of the 0..10 V scale) for the set speed
67      FFTarget := VelSet * WebRatio * Output_Ratio;
68
69      IF FFTarget > Vel_Max_mms THEN
70          FFTarget := Vel_Max_mms;
71      ELSIF FFTarget < 0.0 THEN
72          FFTarget := 0.0;
73      END_IF;
74
75      // Tension: hold at TensWaitSpd until the T-one reports tension
    (I_T_one_TensionOK)
76
77      IF TensWait AND (VelSet > TensWaitSpd) THEN
78          FFTarget := TensWaitSpd * WebRatio * Output_Ratio;
79      END_IF;
80
81      // V9: hold after a cancelled soft stop (the S-curve flattens it smoothly):
82      // never above the present reference, lower targets still allowed
83
84      IF SpdHold THEN
85          IF FFTarget > RampRef THEN
86              FFTarget := RampRef;
87          END_IF;
88
89      ELSIF FFSlowAcc AND (ABS(FFTarget - RampRef) <= FFTarget * 0.005 + 2.0) THEN
90          FFSlowAcc := FALSE;                          // back at the set speed
91      END_IF;
92
93      // Ramp steps per 4 ms scan: web mm/s2 -> reference mm/s per scan
94
95      IF FFSlowAcc AND (SoftCancelAccel > 0.0) THEN
96          StepAcc := SoftCancelAccel * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;   //
    V9
97      ELSIF AccelWeb > 0.0 THEN
98          StepAcc := AccelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
99      ELSE
100         StepAcc := AccelCoarse;
101     END_IF;
102
103     IF SoftStop AND FinalStopRamp AND (FinalStopDecel > 0.0) THEN
104         StepDec := FinalStopDecel * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;   //
    V12 end-of-roll
105     ELSIF SoftStop AND (SoftStopDecel > 0.0) THEN
106         StepDec := SoftStopDecel * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
107     ELSIF DecelWeb > 0.0 THEN
```

```
108     StepDec := DecelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
109   ELSE
110     StepDec := DecelCoarse;
111   END_IF;
112
113   IF StepAcc > 8.0 THEN
114     StepAcc := 8.0;
115   END_IF;
116
117   IF StepDec > 8.0 THEN
118     StepDec := 8.0;
119   END_IF;
120
121   // Ramp generator
122   // V7f: S-curve. FFRate (reference per scan) changes at most FFJerk per scan,
123   // so the full ramp is reached after SCurveTime (s) and is rounded off the
124   // same way before the target -> the T-one / brake air can follow.
125   // SCurveTime = 0 -> plain linear ramp as V7a.
126
127   IF SCurveTime > 0.0 THEN
128     IF (FFRate > 0.0) OR ((FFRate = 0.0) AND (FFTarget > RampRef)) THEN
129       FFJerk := StepAcc * 0.004 / SCurveTime;
130     ELSE
131       FFJerk := StepDec * 0.004 / SCurveTime;
132     END_IF;
133
134     // rate allowed so that it can still fall to 0 at the target
135     // (no MIN(), argument in a REAL variable first: Sysmac data type rules.
136     //  same pattern as SqrtArg / SQRT(SqrtArg) in AutoStop)
137
138     IF FFTarget >= RampRef THEN
139       FFSqrtArg := REAL#2.0 * FFJerk * (FFTarget - RampRef);
140       FFRateSet := SQRT(FFSqrtArg);
141
142       IF FFRateSet > StepAcc THEN
143         FFRateSet := StepAcc;
144       END_IF;
145     ELSE
146       FFSqrtArg := REAL#2.0 * FFJerk * (RampRef - FFTarget);
147       FFRateSet := SQRT(FFSqrtArg);
148
149       IF FFRateSet > StepDec THEN
150         FFRateSet := StepDec;
151       END_IF;
152
153       FFRateSet := -FFRateSet;
154     END_IF;
155
156     IF FFRateSet > FFRate + FFJerk THEN
157       FFRate := FFRate + FFJerk;
158     ELSIF FFRateSet < FFRate - FFJerk THEN
159       FFRate := FFRate - FFJerk;
160     ELSE
161       FFRate := FFRateSet;
162     END_IF;
163
164     RampRef := RampRef + FFRate;
165
166     IF (ABS(FFTarget - RampRef) <= 3.0) AND (ABS(FFRate) <= 2.0 * FFJerk) THEN
```

```
167         RampRef := FFTarget;                    // at target (pot noise)
168         FFRate := 0.0;
169       END_IF;
170     ELSE
171       FFRate := 0.0;
172
173       IF RampRef < FFTarget - StepAcc THEN
174         RampRef := RampRef + StepAcc;
175       ELSIF RampRef > FFTarget + StepDec THEN
176         RampRef := RampRef - StepDec;
177       ELSE
178         RampRef := FFTarget;                    // at target
179       END_IF;
180     END_IF;
181
182     // V7a: "at target" = within 0.5 % of the target (the pot and RwDiam noise
183     // move FFTarget by a few counts every scan -> the counter never reached 500)
184
185     IF (ABS(FFTarget - RampRef) <= FFTarget * 0.005 + 2.0)
186       AND NOT SpdHold AND NOT FFSlowAcc THEN          // V9: not during the
    hold
187
188       IF FFSteadyTicks < 30000 THEN
189         FFSteadyTicks := FFSteadyTicks + 1;
190       END_IF;
191     ELSE
192       FFSteadyTicks := 0;
193     END_IF;
194
195     // Slow adaptation of RwRatioD on the relative speed error (2 s after the
196     // target is reached, not at the 60 Hz limit, not during soft stop / jog)
197
198     IF (FFSteadyTicks >= 500) AND NOT SoftStop AND NOT JogRun
199       AND (VelSet > 15.0) AND (Velocity_M_min > 10.0) AND NOT RwSensFault
200       AND (FFTarget < Vel_Max_mms * 0.98) THEN
201       FFErr := (VelSet - Velocity_M_min) / VelSet;
202
203       IF FFErr > 0.05 THEN
204         FFErr := 0.05;
205       ELSIF FFErr < -0.05 THEN
206         FFErr := -0.05;
207       END_IF;
208
209       IF ABS(FFErr) > 0.003 THEN                  // dead band 0.3 % (encoder noise)
210         RwRatioD := RwRatioD * (1.0 + 0.0004 * FFErr);  // ~10 s time constant
211       END_IF;
212
213       IF RwRatioD < 1000.0 THEN
214         RwRatioD := 1000.0;
215       ELSIF RwRatioD > 8000.0 THEN
216         RwRatioD := 8000.0;
217       END_IF;
218     END_IF;
219
220     Vfd_SP_Modif := Vel_Max_mms - RampRef;          // rung 17 -> InvRef
221   END_IF;
222
223   FFWasActive := FFActive;
```

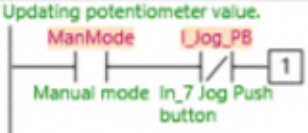
```
225 // V7f: T-one decel signal (for a T-one "stop / boost" input, spare PLC output):
226 // ON while the feed-forward ramps down (more than 1 % above the target, so the
227 // slow drift of the target with the diameter does not pulse it) or soft stop.
228
229 TOneDecel := (FFActive AND (RampRef - FFTarget > FFTarget * 0.01 + 5.0))
```

18 Management of the inverter speed reference in mm/s and conversion to analog points (0–4000 points = 0–10 VDC = 0–10,000 mm/s).



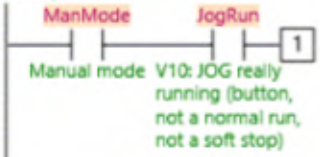

```
1 //InvRef2 := REAL_TO_UINT((Vel_Max_mms - Vfd_SP_Modif ) / Output_Ratio);
2 //InvRefWW := UINT_TO_WORD(InvRef2);
3 //InvRefSW :=Swap(UINT_TO_WORD(InvRef2));
4 //InvRef := WORD_TO_UINT(InvRefSW);
5
6 InvRef := REAL_TO_INT((Vel_Max_mms - Vfd_SP_Modif ) / Output_Ratio);
```

19 Updating potentiometer value.



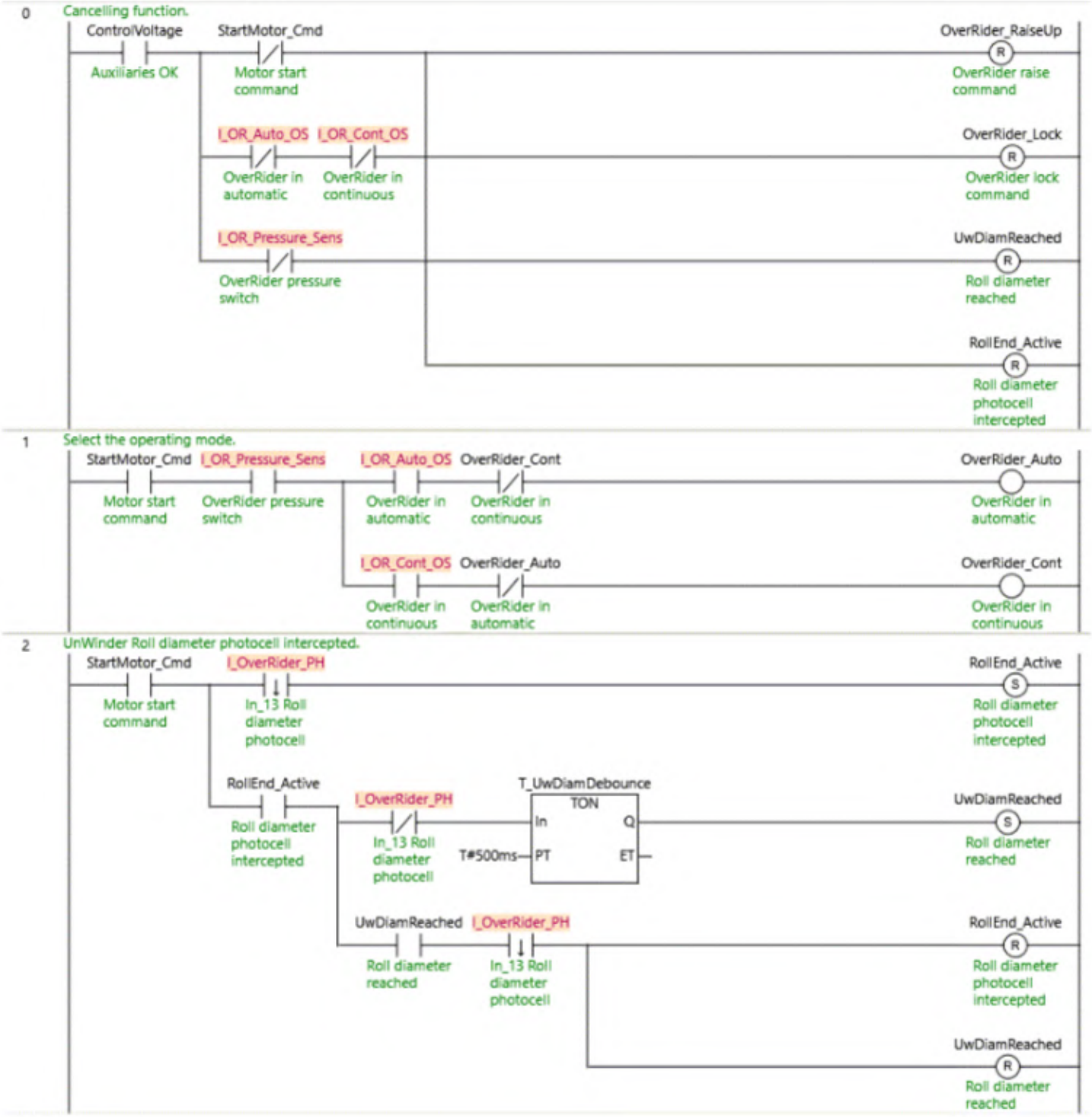

```
1 IF SoftStop THEN
2     InvRef := INT#0;                    // v6m: drive ramps down on its own
3 ELSIF SlowDown = FALSE THEN
4     InvRef := Pot1_Work_Velocity;
5 ELSE
6     InvRef := Pot2_Slow_Velocity;
7 END_IF;
```

20 Slow speed.

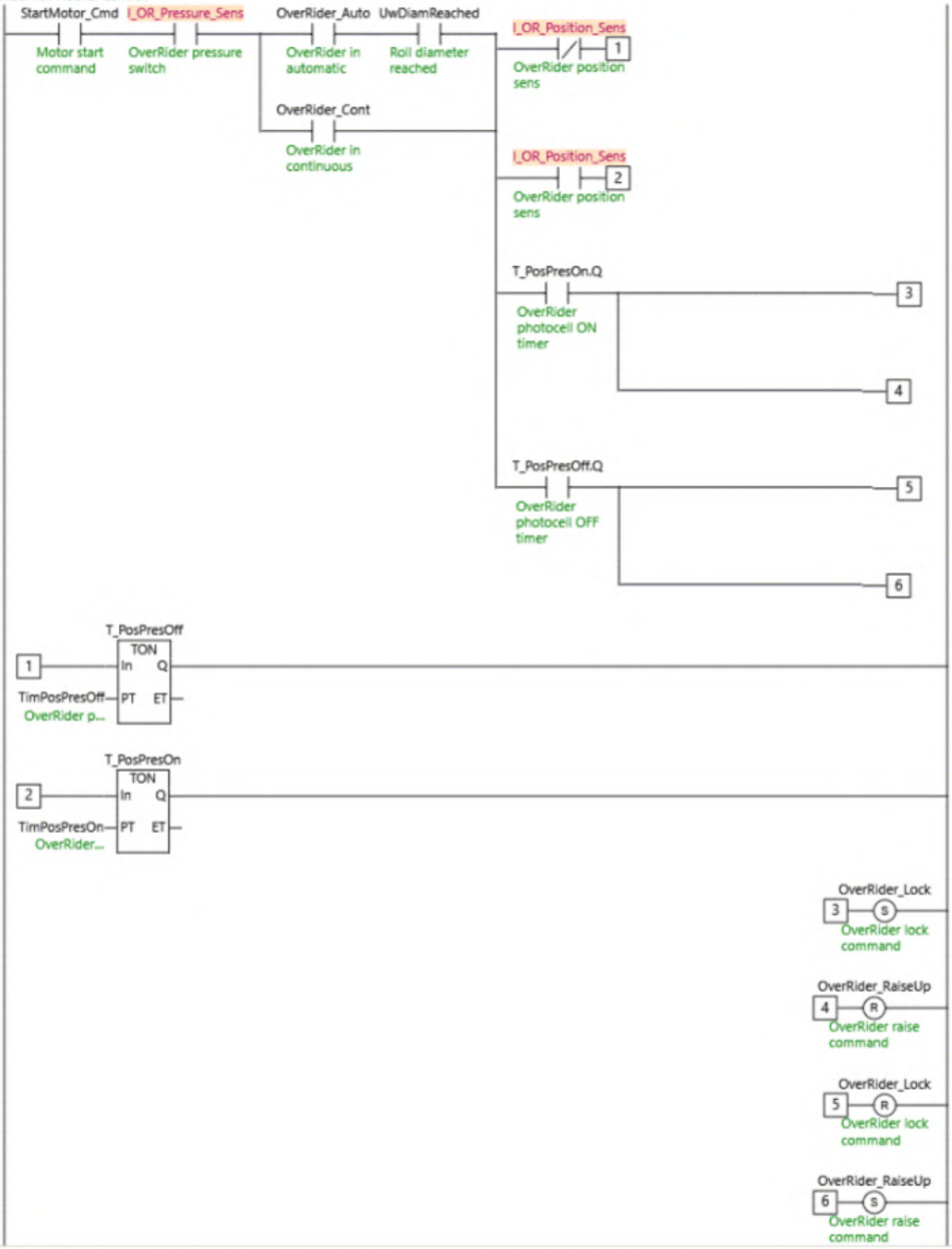



```
1 VelSet := JogSpd_SP;
2 VelSet_mms := VelSet * Mt_To_mm;
3
4 //InvRef2 := REAL_TO_UINT((VelSet_mms) / Output_Ratio);
5 //InvRefWW := UINT_TO_WORD(InvRef2);
6 //InvRefSW :=Swap(UINT_TO_WORD(InvRef2));
7 //InvRef := WORD_TO_UINT(InvRefSW);
8
9 InvRef := REAL_TO_INT(VelSet_mms / Output_Ratio);
```

1-8-1-1-4.OverRider_Manager

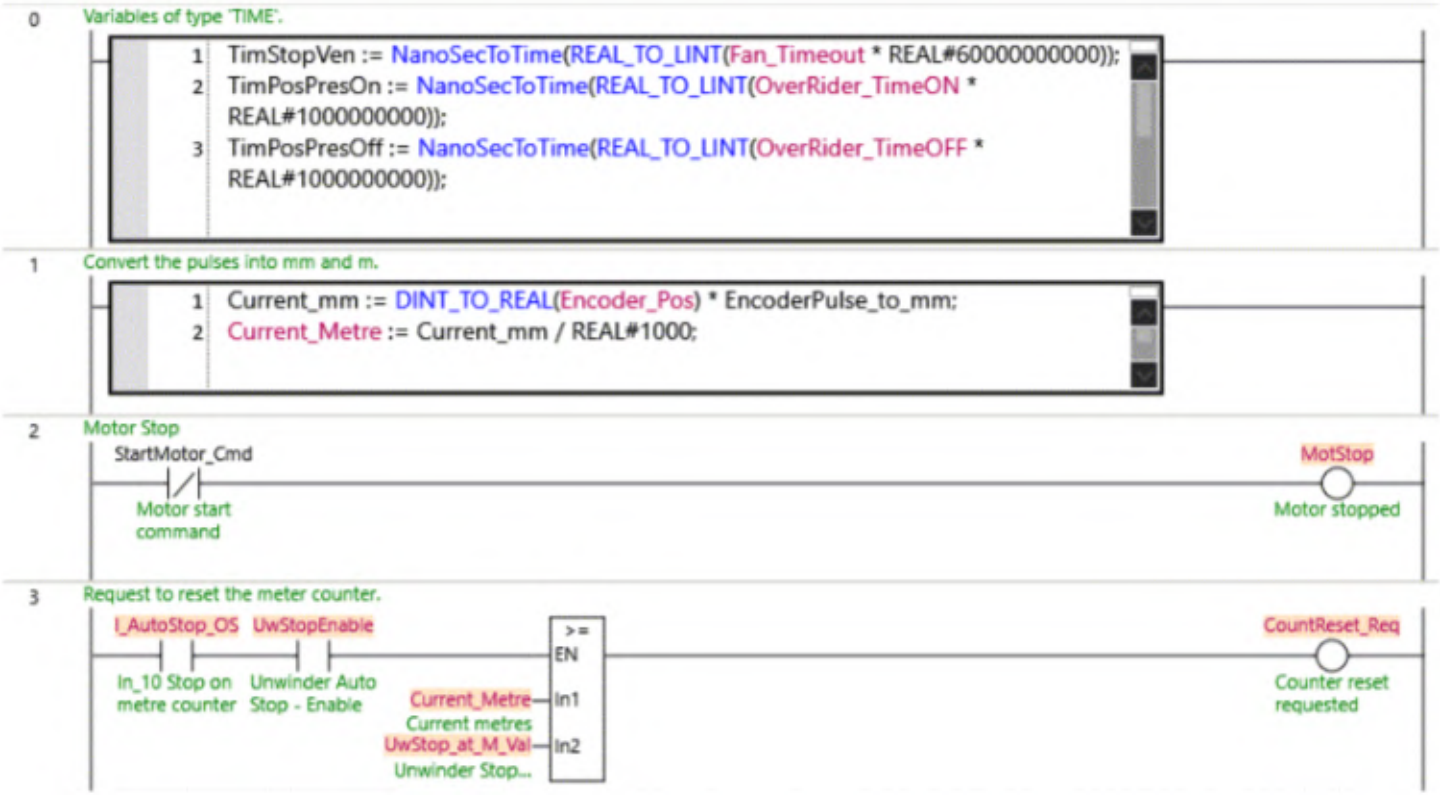


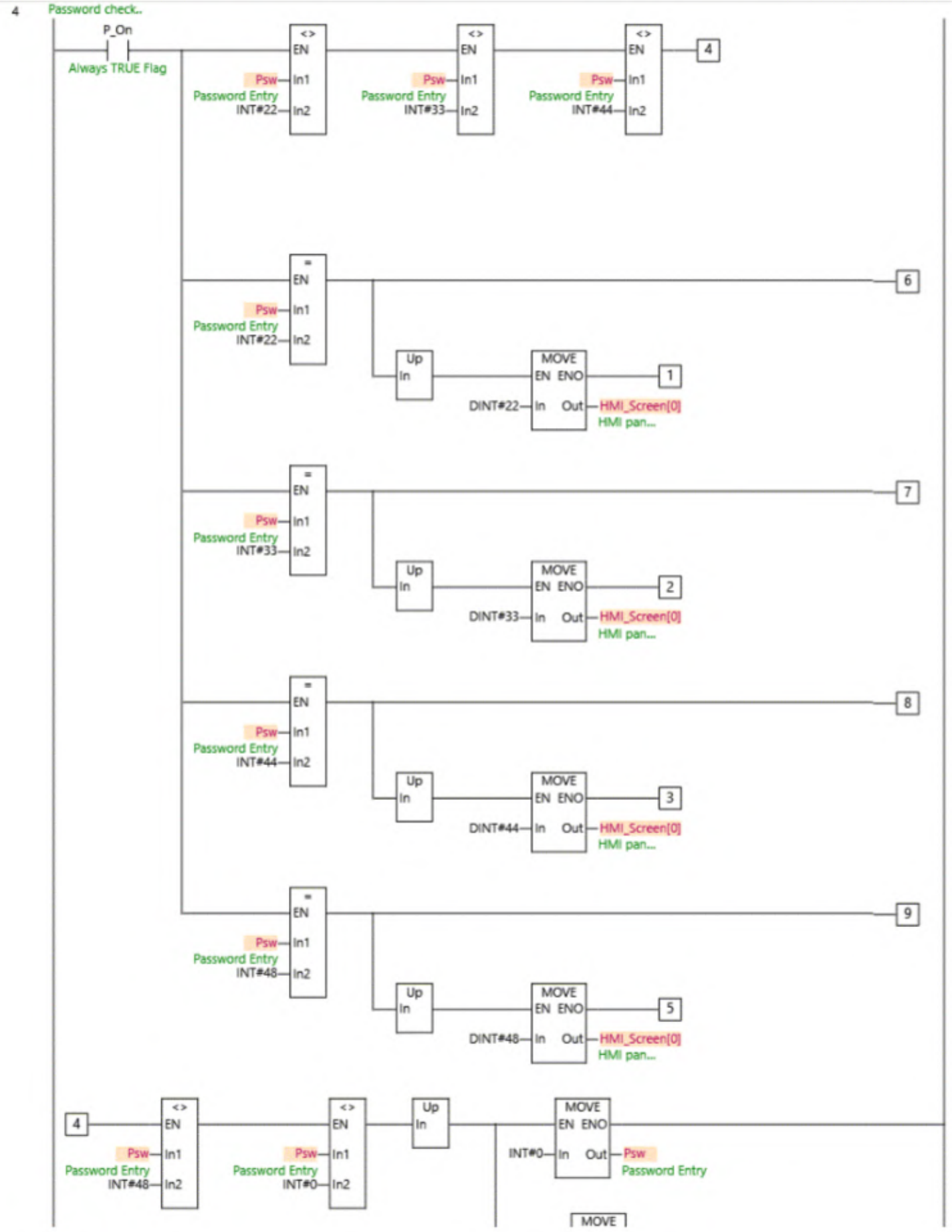
3 Rider roll Hoover control.

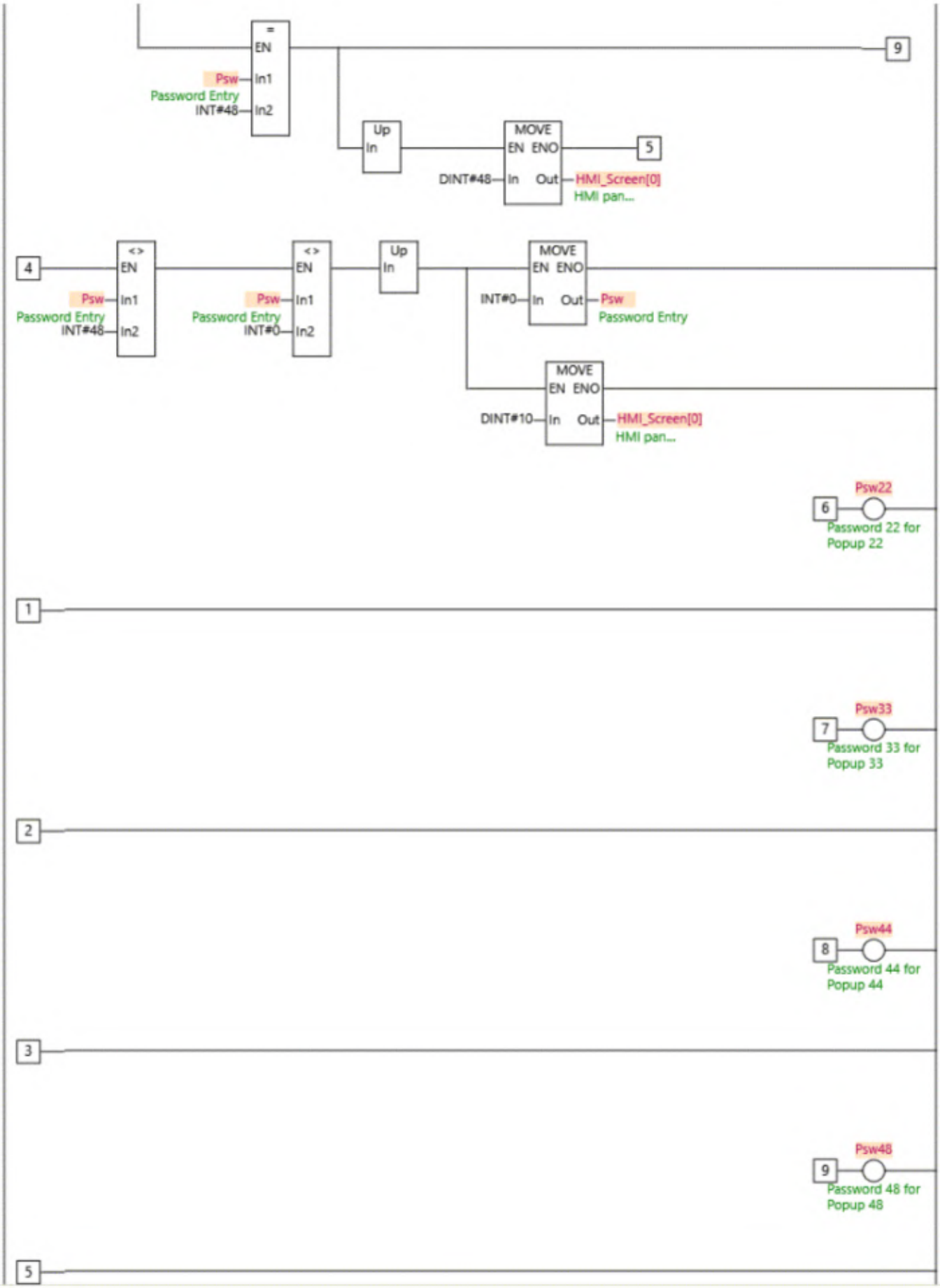


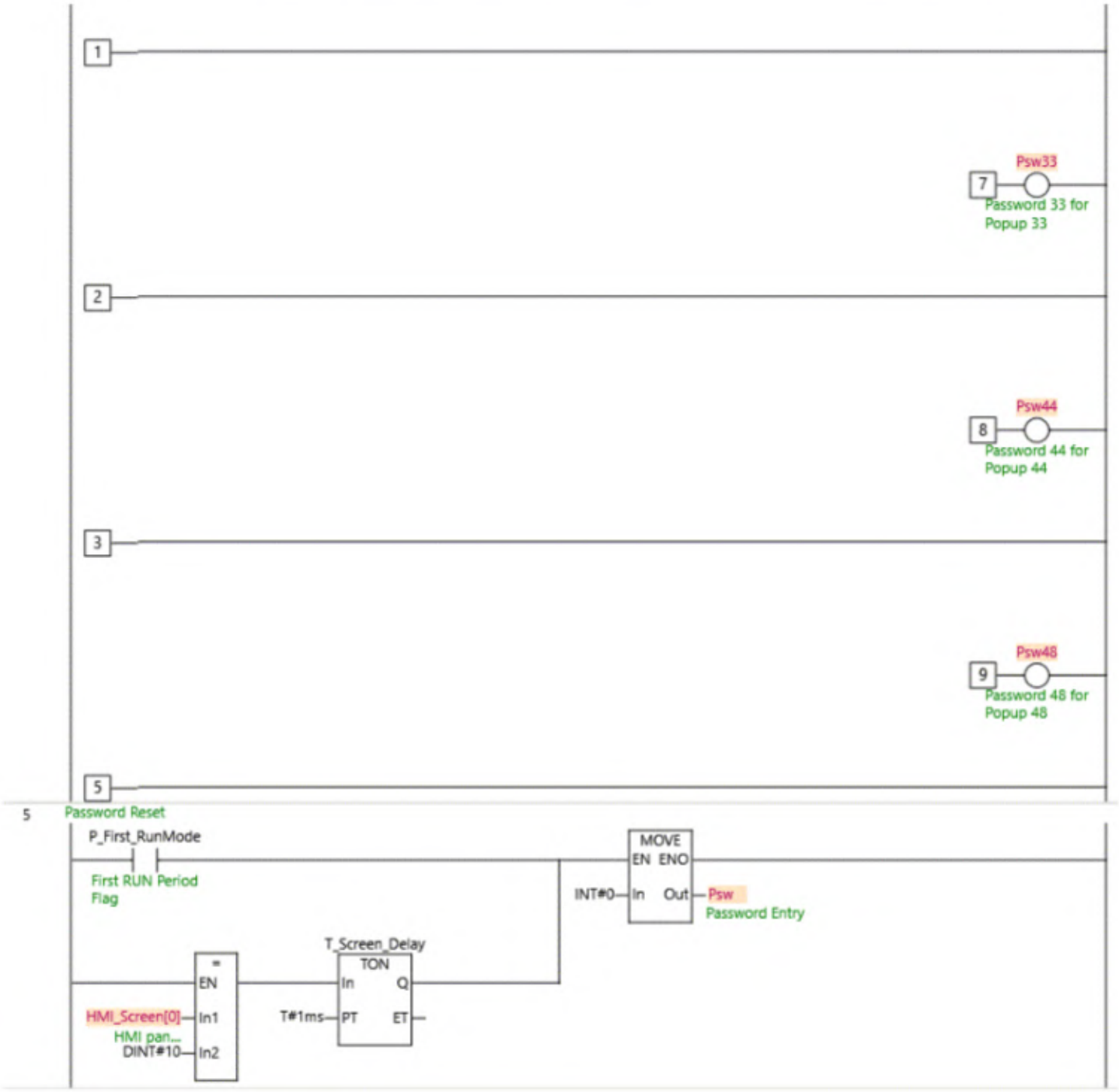

4
Rider roll raise command.
ControlVoltage
Auxiliaries OK
OverRider_RaiseUp
S
OverRider raise command
5
I_ControlVoltage
In_0
ControlVoltageON
OverRider_RaiseUp
R
OverRider raise command

1-8-1-1-5.Console

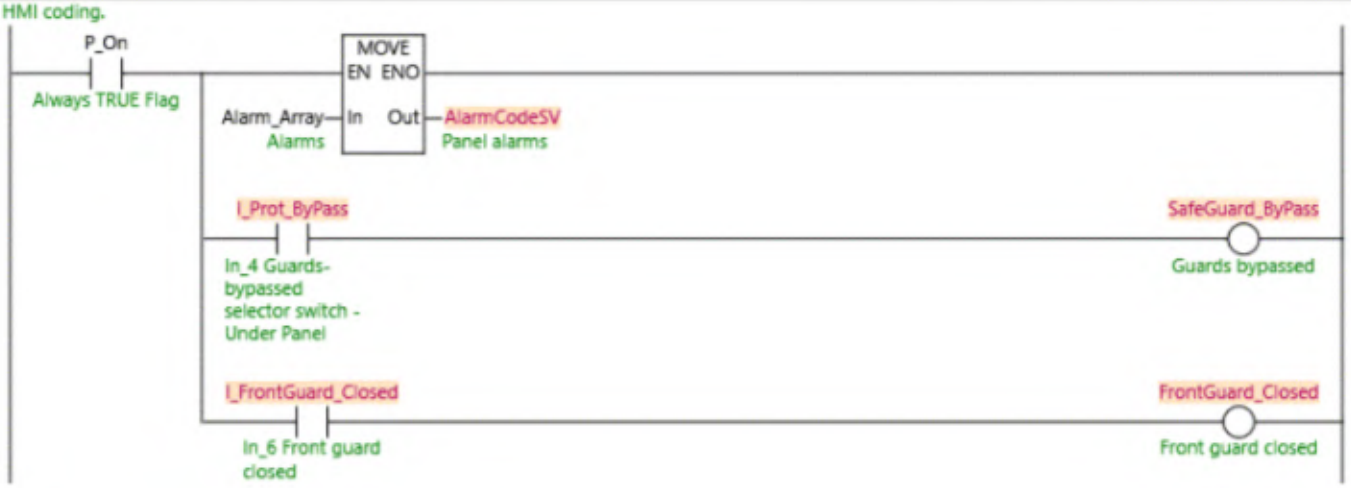



4
Password check..
P_On
Always TRUE Flag
<>
EN
In1
In2
Psw
Password Entry
INT#22
INT#33
INT#44
4
=
6
7
8
9
Up
In
MOVE
EN ENO
In Out
DINT#22
DINT#33
DINT#44
DINT#48
HMI_Screen[0]
HMI pan...
1
2
3
5
INT#48
INT#0
MOVE


=
EN
Psw
Password Entry
INT#48
In1
In2
9
Up
In
MOVE
EN ENO
DINT#48
In Out
5
HMI_Screen[0]
HMI pan...
4
<>
EN
Psw
Password Entry
INT#48
In1
In2
<>
EN
Psw
Password Entry
INT#0
In1
In2
Up
In
MOVE
EN ENO
INT#0
In Out
Psw
Password Entry
MOVE
EN ENO
DINT#10
In Out
HMI_Screen[0]
HMI pan...
Psw22
6
Password 22 for
Popup 22
1
Psw33
7
Password 33 for
Popup 33
2
Psw44
8
Password 44 for
Popup 44
3
Psw48
9
Password 48 for
Popup 48
5


1
Psw33
7
Password 33 for Popup 33
2
Psw44
8
Password 44 for Popup 44
3
Psw48
9
Password 48 for Popup 48
5
5 Password Reset
P_First_RunMode
First RUN Period Flag
MOVE
EN ENO
INT#0
In Out
Psw
Password Entry
T_Screen_Delay
TON
In Q
PT ET
T#1ms
=
EN
HMI_Screen[0]
In1
HMI pan...
DINT#10
In2

6 HMI coding.


P_On
Always TRUE Flag
MOVE
EN ENO
Alarm_Array
Alarms
In
Out
AlarmCodeSV
Panel alarms
I_Prot_ByPass
In_4 Guards-bypassed selector switch - Under Panel
SafeGuard_ByPass
Guards bypassed
I_FrontGuard_Closed
In_6 Front guard closed
FrontGuard_Closed
Front guard closed

1-8-1-1-6.Alarms

0 Check Present alarms.

```
// Alarm / warning handling + panel buzzer (BuzzerOn).
// Alarms: buzzer pulses (Clock1s). ALARM RESET (panel or HMI) silences it 1 min;
//         a NEW alarm during the silence sounds again at once.
// Warnings: ONE 1 s beep each time a new warning appears.
// PrimaryTask 4 ms: 250 scans = 1 s, 15000 scans = 60 s.

AlarmOn := FALSE;
WarningOn := FALSE;
AlarmCnt := 0;
WarnCnt := 0;

FOR Index_A := 0 TO 32 DO

   IF Alarm_Array[Index_A] THEN
      CASE Index_A OF
         5, 13, 15, 16, 17:              // warnings
            WarningOn := TRUE;
            WarnCnt := WarnCnt + 1;
      ELSE                               // alarms
         AlarmOn := TRUE;
         AlarmCnt := AlarmCnt + 1;
      END_CASE;
   END_IF;
END_FOR;

// ALARM RESET button (panel I_RsetAlarm or HMI Alarm_Reset): silence 1 min

RstEdge := (I_RsetAlarm OR Alarm_Reset) AND NOT RstMem;
RstMem := I_RsetAlarm OR Alarm_Reset;

IF RstEdge AND AlarmOn THEN
   SilenceTicks := 15000;
END_IF;

IF (AlarmCnt > AlarmCntMem) OR NOT AlarmOn THEN
   SilenceTicks := 0;                    // new alarm / no alarm: not silenced
END_IF;

IF SilenceTicks > 0 THEN
   SilenceTicks := SilenceTicks - 1;
END_IF;

// New warning: one 1 s beep

IF WarnCnt > WarnCntMem THEN
   WarnBeepTicks := 250;
END_IF;

IF WarnBeepTicks > 0 THEN
   WarnBeepTicks := WarnBeepTicks - 1;
END_IF;

AlarmCntMem := AlarmCnt;
WarnCntMem := WarnCnt;

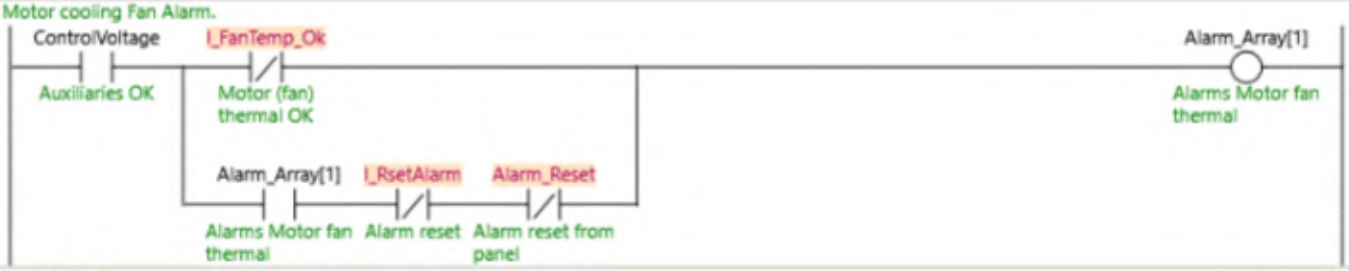
AlarmSilenced := AlarmOn AND (SilenceTicks > 0);
```

```
57
58 BuzzerOn := ControlVoltage
59         AND ((AlarmOn AND NOT AlarmSilenced AND Clock1s) OR (WarnBeepTicks
   > 0));
```

1

```
1  // ---- Alarm 14 (ALARM): unwinder sonar (NX-AD2603 Ch1, P+F UC2000) fault ----
2  // Raw value out of range for 1 s (no echo / broken wire / sensor off).
3  // Set when the measurement is used (unwinder AutoStop ON or motor running),
4  // latched, RESET only once the sensor is OK again (same as alarm 10, DT35).
5
6  T_UwSensFlt(In := (AI_Diam_Unwinder < UwRawMin) OR (AI_Diam_Unwinder >
   UwRawMax), PT := T#1s);
7  UwSensTest := T_UwSensFlt.Q;
8
9  IF ControlVoltage AND Measure_ON AND UwSensTest AND (UwStopEnable OR
   StartMotor_Cmd) THEN
10    UwSensFault := TRUE;
11 ELSIF NOT UwSensTest AND (I_RsetAlarm OR Alarm_Reset) THEN
12    UwSensFault := FALSE;
13 END_IF;
14
15 // ---- Alarm 17 (WARNING): AutoStop ON but paper thickness not learned ----
16 // RwMat_K = 0 -> diameter / stop-length targets use the fixed fallback.
17 // Only above 300 mm rewinder diameter (learning needs ~20 mm growth after
   NEW PAPER),
18 // so it does not beep at every new roll. Not latched.
19
20 Thickness_Fault := ControlVoltage AND StartMotor_Cmd
21             AND (RwMat_K <= 0.0) AND (RwDiam > 300.0);
```

2 Motor cooling fan Alarm.



3 Inverter fault.



4 Machine in emergency mode.



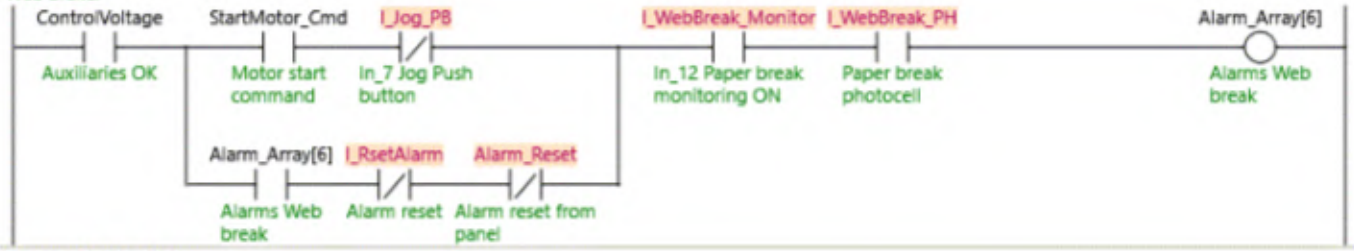
5 Main Motor thermal cutout.



6 Front Guard - Warning



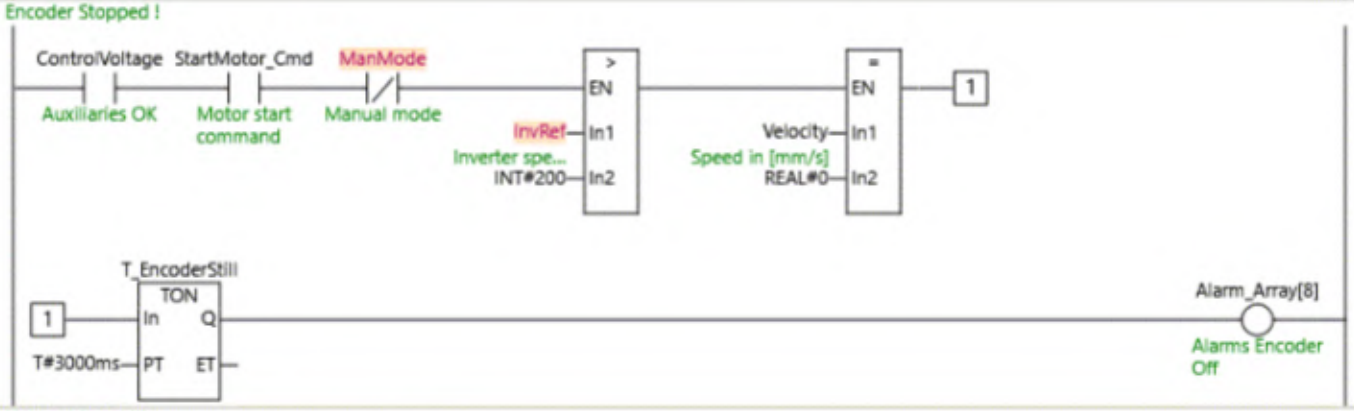
7 Web Brake.



8 Vacuum Cooling fan.



9 Encoder Stopped !



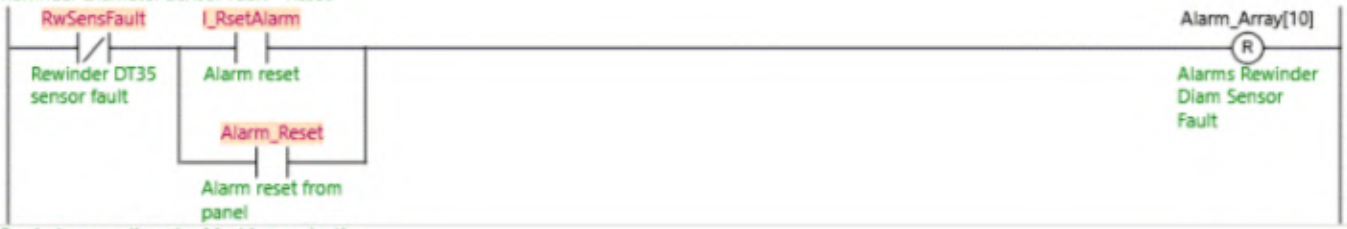
10 Hub not in position.



11 Rewinder Diameter Sensor Fault.



12 Rewinder Diameter Sensor Fault - Reset



13 Rewinder max diameter Machine protection.



14 Rewinder max diameter Machine protection - Reset



15 Emergency Stop by Holding Stop PB 2s



16 Web Tension Monitor - Warning



17 Unwinder Diameter Sensor Fault.



18 Unwinder Roll is Empty - Warning



19 Unwinder Roll is Empty - Reset



20 Rewinder GSM learning fault



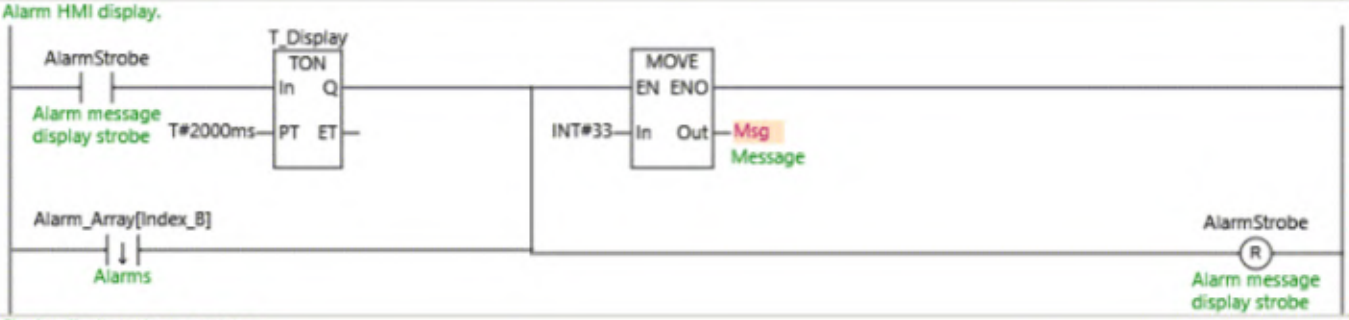
21 Rewinder learning fault - Reset



22 Paper Thickness Learn Process - Not Done ! - Warning



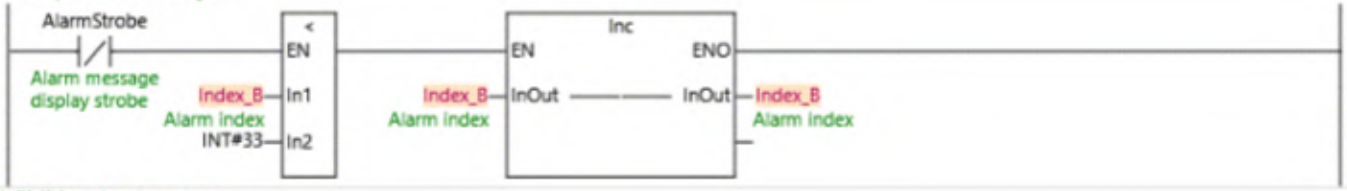
23 Alarm HMI display.



24 Strobe displays alarm message.



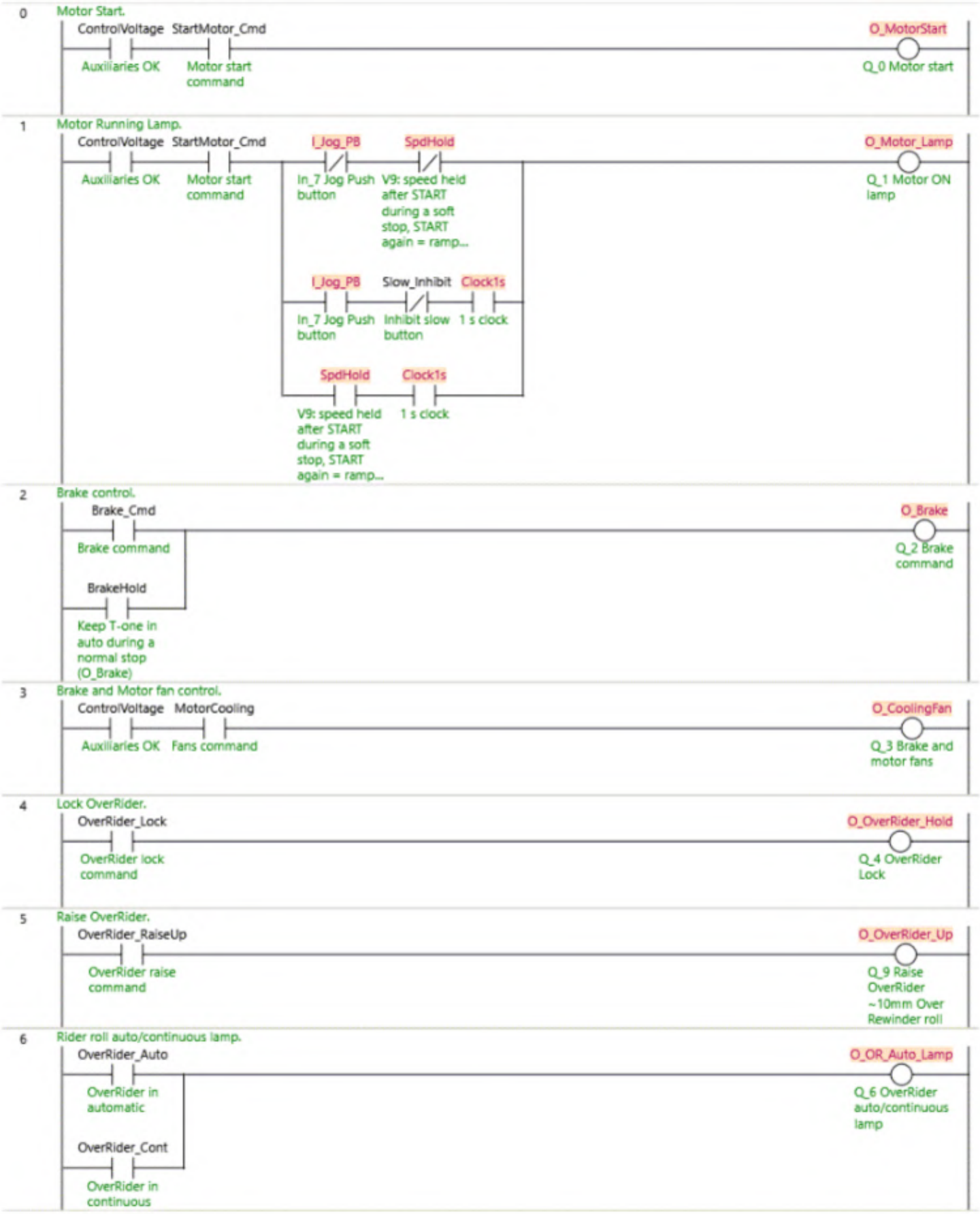
25 Alarm presence check cycle.

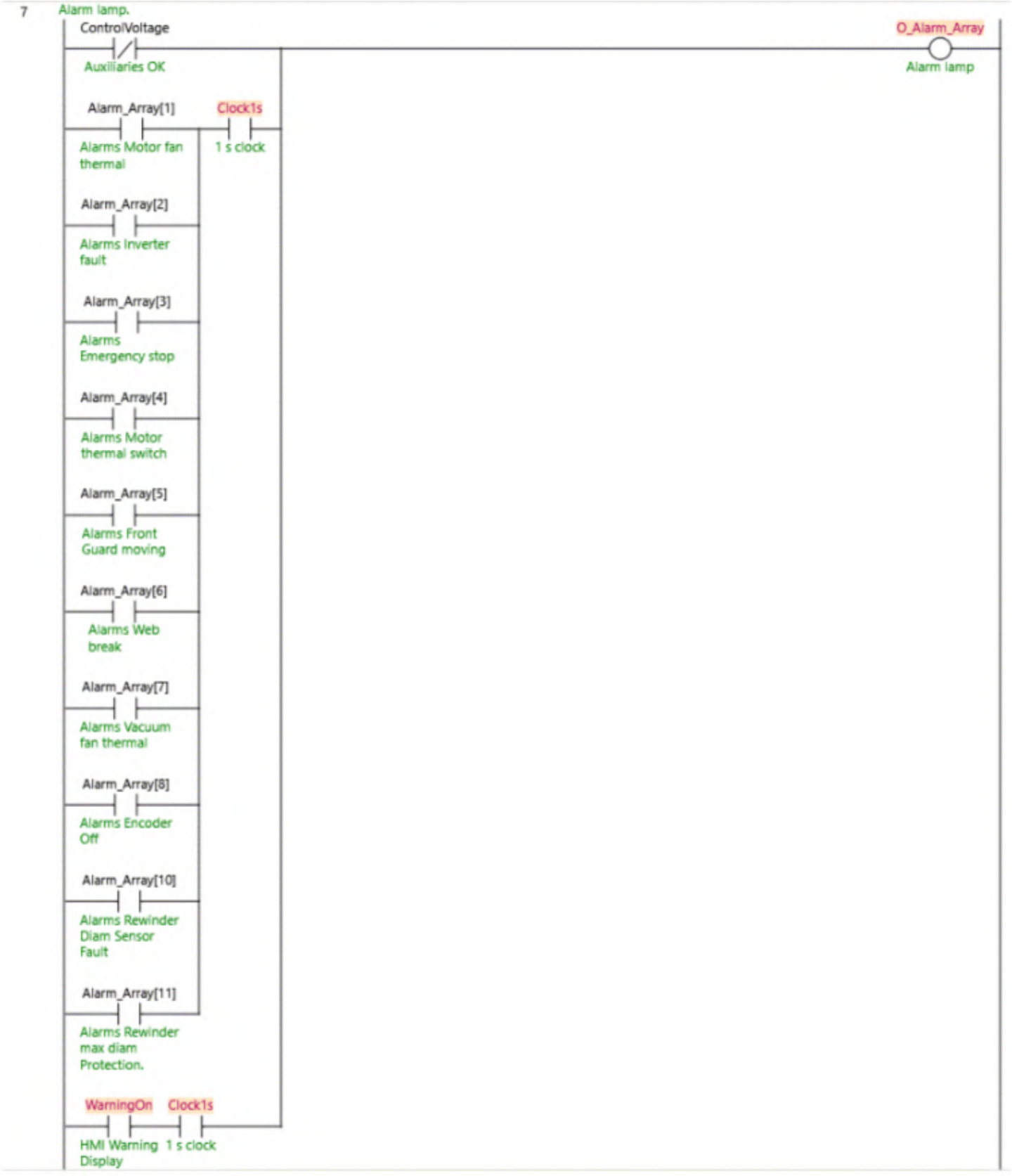


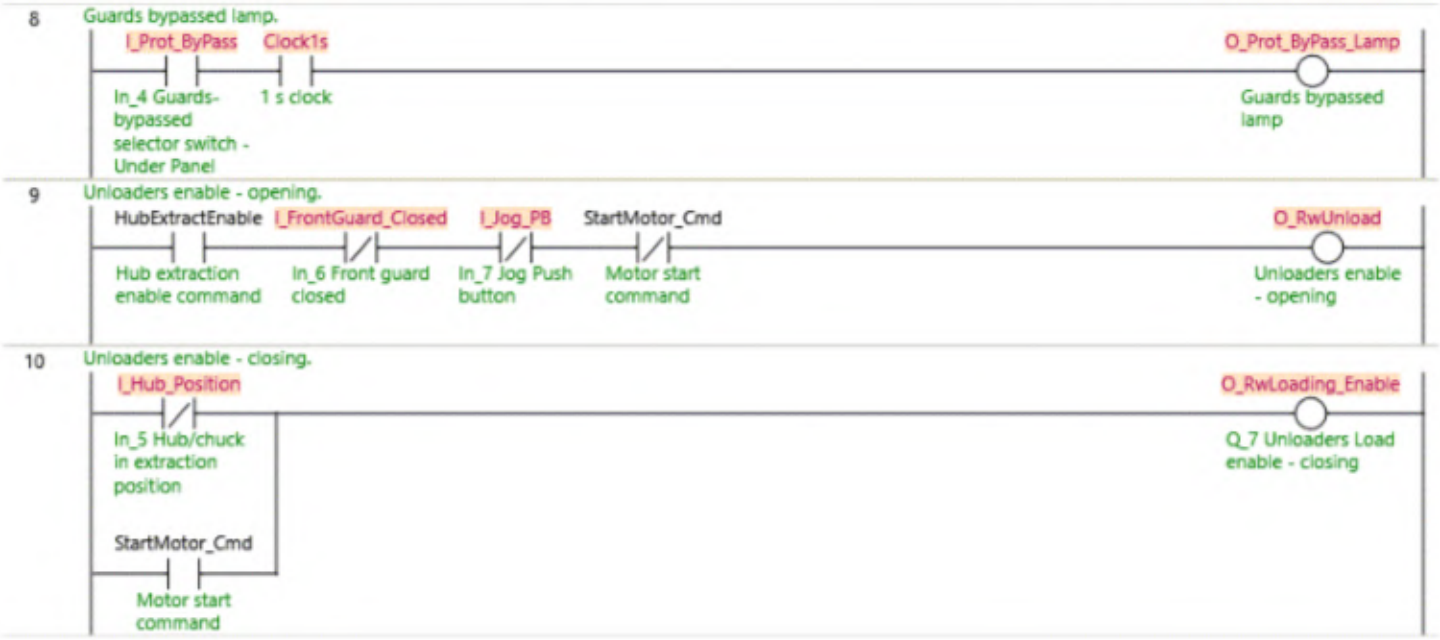
26 Initializing alarm message.



1-8-1-1-7.Output_Manager




7
Alarm lamp.
ControlVoltage
Auxiliaries OK
O_Alarm_Array
Alarm lamp
Alarm_Array[1]
Clock1s
Alarms Motor fan thermal
1 s clock
Alarm_Array[2]
Alarms Inverter fault
Alarm_Array[3]
Alarms Emergency stop
Alarm_Array[4]
Alarms Motor thermal switch
Alarm_Array[5]
Alarms Front Guard moving
Alarm_Array[6]
Alarms Web break
Alarm_Array[7]
Alarms Vacuum fan thermal
Alarm_Array[8]
Alarms Encoder Off
Alarm_Array[10]
Alarms Rewinder Diam Sensor Fault
Alarm_Array[11]
Alarms Rewinder max diam Protection.
WarningOn
Clock1s
HMI Warning Display
1 s clock


8
Guards bypassed lamp.
I_Prot_ByPass
Clock1s
O_Prot_ByPass_Lamp
In_4 Guards-bypassed selector switch - Under Panel
1 s clock
Guards bypassed lamp
9
Unloaders enable - opening.
HubExtractEnable
I_FrontGuard_Closed
I_Jog_PB
StartMotor_Cmd
O_RwUnload
Hub extraction enable command
In_6 Front guard closed
In_7 Jog Push button
Motor start command
Unloaders enable - opening
10
Unloaders enable - closing.
I_Hub_Position
O_RwLoading_Enable
In_5 Hub/chuck in extraction position
StartMotor_Cmd
Motor start command
Q_7 Unloaders Load enable - closing

11 Horn / acoustic signal.

```
// ==== V9: REPLACE Output_Manager rung 11 ("Horn / acoustic signal.", ladder) by ONE ST rung
// ==== with this code (no contact in front). Comment: "V9: horn, beeps set on the HMI".
// Old: horn blinking with Clock1s (0.5 s ON / 0.5 s OFF) during the whole slow-down
//      and the crawl at final speed -> very annoying.
// New, slow-down / final speed (StartMotor_Cmd AND SlowDown AND NOT Stop):
//      HornStartBeep_s beep when it starts (1 s).
//      then a HornBeepLen_s beep (0.2 s) every HornBeepPeriod (5 s).
//      then ONE HornStopBeep_s beep (3 s) when the machine has stopped (< 2 m/min).
//      Each setting 0 = that beep OFF. Front guard moving: unchanged (Clock1s blinking).
// PrimaryTask 4 ms: 250 scans = 1 s.

// HMI limits

IF HornStartBeep_s < REAL#0 THEN
    HornStartBeep_s := REAL#0;
ELSIF HornStartBeep_s > REAL#5 THEN
    HornStartBeep_s := REAL#5;
END_IF;

IF HornBeepLen_s < REAL#0 THEN
    HornBeepLen_s := REAL#0;
ELSIF HornBeepLen_s > REAL#2 THEN
    HornBeepLen_s := REAL#2;
END_IF;

IF HornBeepPeriod < REAL#0 THEN
    HornBeepPeriod := REAL#0;
ELSIF (HornBeepPeriod > REAL#0) AND (HornBeepPeriod < HornBeepLen_s + REAL#0.5) THEN
    HornBeepPeriod := HornBeepLen_s + REAL#0.5;    // always a pause between beeps
ELSIF HornBeepPeriod > REAL#30 THEN
    HornBeepPeriod := REAL#30;
END_IF;

IF HornStopBeep_s < REAL#0 THEN
    HornStopBeep_s := REAL#0;
ELSIF HornStopBeep_s > REAL#10 THEN
    HornStopBeep_s := REAL#10;
END_IF;

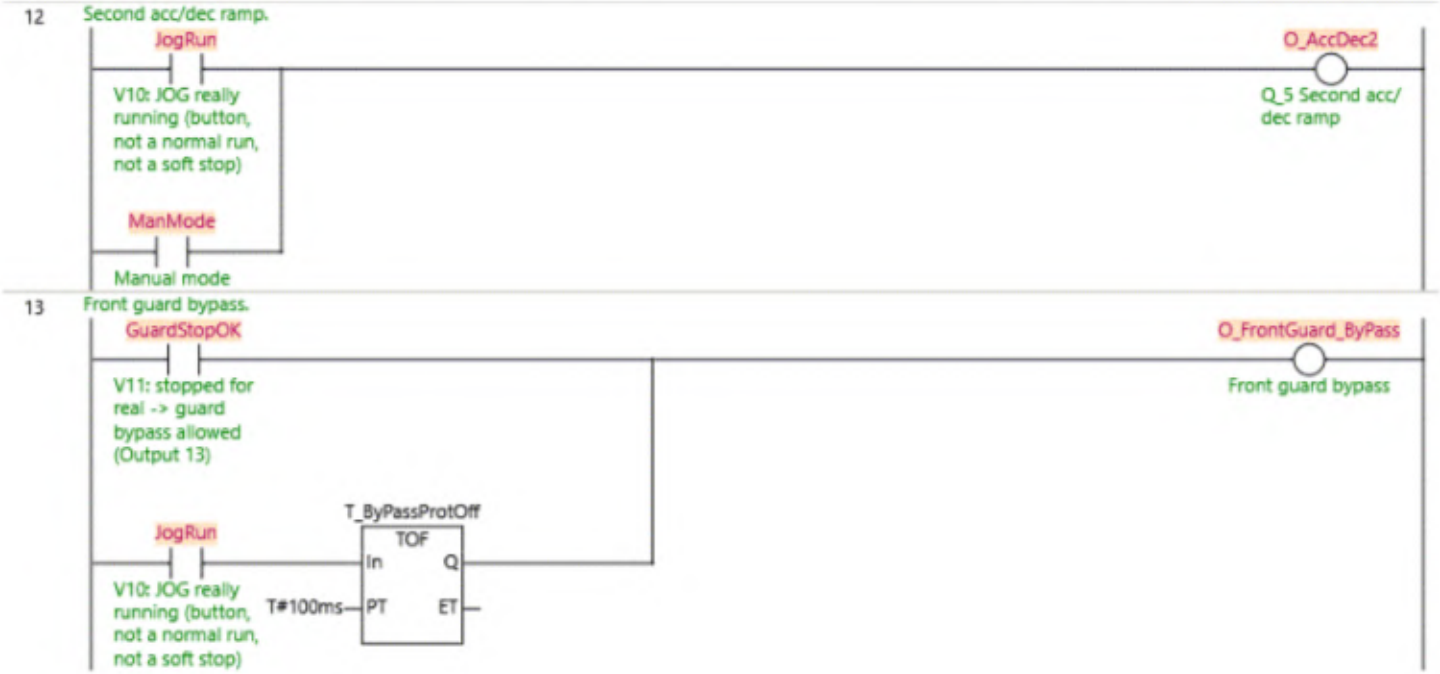
HornStartScans := REAL_TO_INT(HornStartBeep_s * REAL#250);
HornLenScans   := REAL_TO_INT(HornBeepLen_s * REAL#250);
HornPer        := REAL_TO_INT(HornBeepPeriod * REAL#250);

// Slow-down / final speed beeps

HornRun := StartMotor_Cmd AND SlowDown AND NOT Stop;

IF HornRun THEN
    IF NOT HornRunMem THEN
        HornTicks := 0;                    // slow-down starts
    END_IF;
```

```
52
53      IF HornTicks < 30000 THEN
54          HornTicks := HornTicks + 1;
55      END_IF;
56
57      IF HornTicks <= HornStartScans THEN
58          HornSlowBeep := TRUE;                   // start beep
59          HornCnt := 0;
60      ELSIF (HornPer > 0) AND (HornLenScans > 0) THEN
61          HornCnt := HornCnt + 1;
62          IF HornCnt >= HornPer THEN
63              HornCnt := 0;
64          END_IF;
65
66          HornSlowBeep := (HornCnt >= HornPer - HornLenScans);   // beep at the end
    of each period
67      ELSE
68          HornSlowBeep := FALSE;                  // periodic beeps OFF
69      END_IF;
70  ELSE
71      HornTicks := 0;
72      HornCnt := 0;
73      HornSlowBeep := FALSE;
74  END_IF;
75
76  // End of a slow-down by a stop (auto stop, STOP button): one beep at standstill
77  // (Stop is set in Motor_Manager before StartMotor_Cmd drops on the next scan)
78
79  IF HornRunMem AND NOT HornRun AND (Stop OR NOT StartMotor_Cmd) THEN
80      HornStopPend := TRUE;
81  END_IF;
82
83  IF StartMotor_Cmd AND NOT HornStartMem THEN
84      HornStopPend := FALSE;                      // restarted: no stop beep
85      HornStopTicks := 0;
86  END_IF;
87
88  HornStartMem := StartMotor_Cmd;
89
90  IF HornStopPend AND (Velocity_M_min < REAL#2) THEN
91      HornStopPend := FALSE;
92      HornStopTicks := REAL_TO_INT(HornStopBeep_s * REAL#250);
93  END_IF;
94
95  IF HornStopTicks > 0 THEN
96      HornStopTicks := HornStopTicks - 1;
97  END_IF;
98
99  HornRunMem := HornRun;
100
101 // HMI test button: horn while pressed (machine stopped only)
102
103 O_AcousticSignal := (NOT I_FrontGuard_Closed AND NOT I_FrontGuard_Open AND
    Clock1s)
104                 OR HornSlowBeep OR (HornStopTicks > 0)
105                 OR (HornTest AND NOT StartMotor_Cmd);
```


12
Second acc/dec ramp.
JogRun
V10: JOG really running (button, not a normal run, not a soft stop)
ManMode
Manual mode
O_AccDec2
Q_5 Second acc/ dec ramp
13
Front guard bypass.
GuardStopOK
V11: stopped for real -> guard bypass allowed (Output 13)
JogRun
V10: JOG really running (button, not a normal run, not a soft stop)
T_ByPassProtOff
TOF
In
Q
T#100ms
PT
ET
O_FrontGuard_ByPass
Front guard bypass

1-8-1-1-8.AutoStop_Control

0 Auto Stop Process 1-5

```
(*
================================================================
====================
  AutoStop_Control  V10  -  PART 1 of 2 (rung 0): sections 1 - 5c
  Project : 04820KA001 Mono Canada II  (NX1P2, PrimaryTask 4 ms)
  Rewinder (Rw) : SICK DT35 0-10 V -> AI_Diam_Rewinder (AD2603 Ch2)
  Unwinder (Uw) : meter counter mode + sonar diameter (UwDiam_Act)

  NAMING  Uw_ = unwinder   Rw_ = rewinder
        _mm = millimeters   _m = meters
        _Pls = encoder pulses (counts)   _Pps = pulses per second
        _Pps2 = pulses per second squared

  V10: JogRun (jog really running) instead of the raw jog button (5, 5b):
       long baseline also restarted if the encoder goes 2 m back (4b).
  V9 : RwDiamRev = DT35 averaged over exactly one revolution (2b), used by
       the thickness learning (removes the out-of-round wave), RwWave_mm.
     GSM popup: once per real new roll (empty 3 s, then > stop + 100 mm
       for 3 s); NEW PAPER closes it (section 10).
     paper thickness: long-baseline learning (D^2 growth / length) from the
       start point, LOCKED at RwLockDiam (300 mm): no more change, no
       paper-change detection, until NEW PAPER / GSM popup YES (4b).
  V8 : = V7 + V7c unwinder DIAMETER slow-down follows the speed (6c).
       V7e ManMode slow-down model as before V7 (section 6),
       V7f S-curve extra length in the slow-down (section 6)
  V7 : FEED-FORWARD speed loop (Motor_Manager rung B): reference = set speed
     x RwRatioD / diameter along a ramp in real web mm/s2 (DecelWeb /
     AccelWeb), RwRatioD adapted slowly -> no hunting on big rolls.
     Section 6: slow-down length = one ramp at DecelWeb (FFEnable), else
     web coarse ramp + trim tail; learning only as fallback.
  v6y: rewinder AUTO stop (RwStopEnable) limits the work speed to
     RwAutoMaxSpd (500 m/min), taken over at standstill only; section 6
     uses the limited speed.
  v6x: new parent roll on the unwinder -> GsmAsk (HMI popup "did you change
     the paper?"); GsmAskYes forgets the learned thickness.
  v6w: paper thickness window is LENGTH-based instead of 10 s, modulated by
     the diameter (RwWinGrow_mm 2.5 mm of growth, 4 m .. RwWinLen_m 30 m):
     same accuracy at every speed and roll size; stopped window = dropped.
     ALARM RESET also clears LearnFailCnt / StopFailCnt (alarm 16).
  v6v: BrakeHold released on the SIGNED web speed (Velocity_M_min < 5),
     so a web pulled back after the stop releases the T-one at once.
  v6u: stop learning: a stop shorter than the reaction distance or faster
     than RwDecelMax_mm is ignored (not a fault). Only a too slow stop
     (< RwDecelMin_mm) counts toward alarm 16.
  v6t: rewinder slow-down fallback while the paper thickness is not
     learned: RwSlowReq at RwTargetDiam - RwSlowFallbackMm (30 mm).
  v6s: paper change: a learning window more than RwMatChangePct (25 %) away
     from RwMat_K is ignored once, taken directly the 2nd time in a row
     (automatic). RwMatReset (HMI "NEW PAPER") forgets RwMat_K.
  v6p: section 5c: RwLearnReset clears RwStopDecelK, RwStopDecel and
     RwSlowDecelK (the v6f seed that refilled RwStopDecelK is removed).
  v6l: previous value saved before every TEACH CORE (RwAxisDist_Prev,
       UwCoreDist_Prev, Retain) + UNDO buttons (RwTeachUndo,
       UwTeachUndo): swap current <-> previous. State 4 = RESTORED.
  v6k: unwinder TEACH CORE (Core_Measure) works like the rewinder one:
       same checks, 2 s stable with sonar AVERAGE, status UwTeachState.
```

```
55          Replaces General rung 15 (one immediate sonar reading).
56      v6j: TEACH CORE (section 14): HMI button RwTeachCore, empty core on the
57          rewinder, machine stopped -> after 2 s stable:
58          RwAxisDist := RwDist + RwCoreDiameter / 2. Status in RwTeachState.
59      v6i: auto: pots scaled to WorkSpd_Max / FinalSpd_Max (Motor_Manager rung 6,
60          Pot1_Mem / Pot2_Mem). ManMode keeps the raw pots (percent),
61          so section 6 uses the raw pots in ManMode.
62      v6h: BrakeHold keeps O_Brake (T-one "reel brake automatic") ON during a
63          normal stop until the web is almost stopped (< 5 m/min, max 15 s).
64          Before: T-one switched to "stop" with the run command, while the
65          rewinder was decelerating hard -> unwinder overran, web on the
66          floor (field 10/20/2026). Used in Output_Manager rung 2.
67      v6g: slow-down AND stop decel proportional to the rewinder diameter.
68          The ATV930 ramp is in Hz/s (motor = rewind shaft), so the web
69          decel [mm/s2] = shaft decel x radius. Field data 10/20/2026:
70          24 mm/s2 at 120 mm -> 90 mm/s2 at 600 mm.
71          Learned: RwSlowDecelK, RwStopDecelK [mm/s2 per mm of diameter].
72          Live:    RwSlowDecelLrn [mm/s2], RwStopDecel [pulses/s2] = K x DecelDiam.
73      v6f: learn the REAL slow-down deceleration (section 5b) -> RwSlowDecelLrn
74          (mm/s2). The ATV930 stretches its ramp with a heavy roll (no
75          braking resistor), so the PLC ramp (DecelCoarse) was too
76          optimistic: the stop came before final speed was reached.
77          Priority in section 6: RwSlowDecel (manual) > RwSlowDecelLrn
78          (learned) > DecelCoarse/DecelTrim model.
79      v6e: GSM_Learned_OK (replaces global RwLearnFault): TRUE only when
80          RwMat_K is learned, RwStopDecel is learned (if RwLearnStop) and
81          there is no learning fault. Alarm 16 uses GSM_LearnFault.
82      v6d: exact 100 ms tick (25 x 4 ms scan counter, the self-resetting
83          TON ran ~104-108 ms -> speed ~4-8 % too high)
84         standstill = <= 2 counts / 100 ms (encoder jitter)
85         stop learning skipped after jog / manual / slow-button runs and
86          when the auxiliaries drop (E-stop): RunWasJog, StopLearnValid
87         work / final speed from the potentiometers in BOTH modes
88          (Speed_Display / LowSpd_Display are not m/min in ManMode)
89         slow-down length includes the DecelTrim tail inside the
90          SpdDelta_Percent band (Motor_Manager rungs 14/15)
91      v6c: learning supervision -> RwLearnFault (alarm 16), learned paper
92          thickness RwThick_mm, plausibility limits for K and stop decel
93      v6b: RwMaxSlowAtDiam display + RwMaxSlowFallback (slow-down warning
94          while RwMat_K is not learned yet)
95      v6 : consistent internal names, units in names.
96         Globals renamed: RwRemain_m, RwStopLen_m, RwSlowLen_m,
97                     RwSlowMargin_m, UwSlowMargin_m
98      v5 : max diameter protection (always active), sensor plausibility,
99         12SA5 master (section 9), alarms in section Alarms (ladder)
100     v4b: AUTO interlock + single HMI display (SlowAtDisp / SlowAtMode)
101
    ================================================================
    ==================== *)
102
103 (* ---- 1. 100 ms tick, mm per encoder pulse ------------------------------ *)
104 (* v6d: 25 scans x 4 ms PrimaryTask = exactly 100 ms.                    *)
105 (*      Change 25 if the PrimaryTask period is changed.                  *)
106   TickCnt := TickCnt + 1;
107 IF TickCnt >= 25 THEN
108     TickCnt := 0;
109     Tick := TRUE;
110 ELSE
111     Tick := FALSE;
```

```
112 END_IF;
113
114 IF (Encoder_Pulse > 0) AND (EncoderWheel_Circumf > 0.0) THEN
115     MmPerPulse := EncoderWheel_Circumf / (INT_TO_REAL(Encoder_Pulse) * 4.0);
116 ELSE
117     MmPerPulse := 0.0;
118 END_IF;
119
120 (* ---- 2. Rewinder DT35 -> filtered distance -> diameter ---------- *)
121 RwDistRaw_mm := RwDist0V + (INT_TO_REAL(AI_Diam_Rewinder) / 4000.0)
122             * (RwDist10V - RwDist0V);
123
124 IF Tick THEN
125     IF NOT RwFiltInit THEN
126         RwDist := RwDistRaw_mm;
127         RwFiltInit := TRUE;
128     ELSE
129         RwDist := RwDist + RwFilter_K * (RwDistRaw_mm - RwDist);
130     END_IF;
131 END_IF;
132
133 RwSensFault := (AI_Diam_Rewinder < RwRawMin) OR (AI_Diam_Rewinder >
    RwRawMax)
134             OR (RwDist > RwAxisDist);
135
136 RwDiam := 2.0 * (RwAxisDist - RwDist);
137 IF RwDiam < 0.0 THEN
138     RwDiam := 0.0;
139 END_IF;
140
141 (* ---- 2b. V9: diameter averaged over exactly ONE revolution ---------- *)
142 (* The out-of-round roll gives a wave at the rotation frequency (seen in   *)
143 (* the 10/29 traces: 2.4 Hz at 444 m/min / 973 mm). Averaging the raw     *)
144 (* DT35 distance over exactly one revolution cancels it (and all its      *)
145 (* harmonics), whatever the phase -> no shaft position sensor needed:     *)
146 (* one revolution = PI x D of web on the encoder. Each scan is weighted   *)
147 (* by its encoder pulses = by roll ANGLE, so ramps and standstill do not  *)
148 (* bias it (stopped: nothing added). Used only by the thickness learning. *)
149 (* RwWave_mm = peak-peak diameter wave in the last revolution (HMI).      *)
150 RevDP := ABS(Encoder_Pos - RevPrevPos);
151 RevPrevPos := Encoder_Pos;
152 IF RwSensFault OR (MmPerPulse <= 0.0) OR (RwDiam < 50.0) OR (RevDP > 100000)
    THEN
153     RevW := 0.0;
154     RevSum := 0.0;
155     RwDiamRevOk := RwDiamRevOk AND NOT RwSensFault;
156 ELSIF RevDP > 0 THEN
157     IF RevW <= 0.0 THEN
158         RevMin := RwDistRaw_mm;
159         RevMax := RwDistRaw_mm;
160     END_IF;
161     RevSum := RevSum + RwDistRaw_mm * DINT_TO_REAL(RevDP);
162     RevW := RevW + DINT_TO_REAL(RevDP);
163     IF RwDistRaw_mm < RevMin THEN
164         RevMin := RwDistRaw_mm;
165     END_IF;
166     IF RwDistRaw_mm > RevMax THEN
167         RevMax := RwDistRaw_mm;
168     END_IF;
```

```
    IF RevW >= 3.14159 * RwDiam / MmPerPulse THEN
        RwDiamRev := 2.0 * (RwAxisDist - RevSum / RevW);
        RwWave_mm := 2.0 * (RevMax - RevMin);
        RwDiamRevOk := TRUE;
        RevW := 0.0;
        RevSum := 0.0;
    END_IF;
END_IF;

(* v6g: diameter used by the decel model (last valid on sensor fault)      *)
IF NOT RwSensFault THEN
    DecelDiam := RwDiam;
ELSE
    DecelDiam := RwLastValidDiam;
END_IF;
IF DecelDiam < 100.0 THEN
    DecelDiam := 100.0;                         (* core *)
END_IF;

(* ---- 3. Actual speed from encoder, 100 ms window ---------------------- *)
IF Tick THEN
    IF NOT SpdInit THEN
        EncPrev := Encoder_Pos;
        SpdInit := TRUE;
    END_IF;
    SpdAct_Pps := DINT_TO_REAL(ABS(Encoder_Pos - EncPrev)) * 10.0;
    EncPrev := Encoder_Pos;
END_IF;
RwSpeed := SpdAct_Pps * MmPerPulse * 0.06;          (* m/min *)

(* ---- 4a. v6s: NEW PAPER button: forget the learned thickness ---------- *)
(* RwMat_K = 0 -> the first valid 10 s window sets it directly.            *)
(* HMI momentary button: must be HELD 3 s, then the reset is done once and *)
(* the PLC clears the bit (HMI lamp goes off = done).                      *)
T_MatReset(In := RwMatReset, PT := T#3s);
IF T_MatReset.Q THEN
    RwMat_K := 0.0;
    LearnFailCnt := 0;
    MatChangeCnt := 0;
    RwMatLocked := FALSE;                       (* V9 *)
    LongActive := FALSE;
    GsmAsk := FALSE;                            (* V9: NEW PAPER answers the popup *)
    GsmArmed := FALSE;
    RwMatReset := FALSE;
END_IF;

(* ---- 4b. V9: long-baseline thickness learning + LOCK ------------------ *)
(* From the start point (first run after NEW PAPER / GSM YES / power-on    *)
(* while not locked): K = (D^2 - D0^2) / (2 x pulses). The waving of the    *)
(* roll (+/-0.4 mm) only matters at the two ends, so the error falls with  *)
(* the growth (50 mm growth: ~1 %). After 20 mm growth it replaces the     *)
(* 30 m windows. At RwLockDiam (and >= RwLockMinGrow_mm growth) the value
*)
(* is LOCKED (Retain): windows and paper-change detection stop.            *)
(* RwLockDiam = 0 -> never locked (windows + long baseline as learning).   *)
IF Tick AND NOT RwMatLocked THEN
    IF LongActive AND (RwDiamRev < LongStartD - 30.0) AND RwDiamRevOk THEN
        LongActive := FALSE;                    (* new core: restart *)
    END_IF;
```

```
227    IF Encoder_Reset OR (Encoder_Pos < LongStartPos - 20000) THEN
228      LongActive := FALSE;                    (* metre counter reset / 2 m back *)
229    END_IF;
230    IF NOT LongActive AND O_MotorStart AND RwDiamRevOk
231      AND (RwDiamRev > 0.0) THEN
232      LongStartPos := Encoder_Pos;
233      LongStartD := RwDiamRev;
234      LongActive := TRUE;
235    END_IF;
236    LongValid := FALSE;
237    IF LongActive AND RwDiamRevOk AND (MmPerPulse > 0.0) THEN
238      LongGrow_mm := RwDiamRev - LongStartD;
239      LongPls := DINT_TO_REAL(ABS(Encoder_Pos - LongStartPos));
240      IF (LongGrow_mm >= 20.0) AND (LongPls > 0.0) THEN
241        LongK := (RwDiamRev * RwDiamRev - LongStartD * LongStartD) / (2.0 *
   LongPls);
242        LongThick_mm := LongK * 3.14159 / (2.0 * MmPerPulse);
243        IF (LongThick_mm >= RwThickMin_mm) AND (LongThick_mm <=
   RwThickMax_mm) THEN
244          LongValid := TRUE;
245          RwMat_K := LongK;
246          MatChangeCnt := 0;
247          LearnFailCnt := 0;
248          IF (RwLockDiam > 0.0) AND (RwDiamRev >= RwLockDiam)
249            AND (LongGrow_mm >= RwLockMinGrow_mm) THEN
250            RwMatLocked := TRUE;          (* solid from now on *)
251            LongActive := FALSE;
252          END_IF;
253        END_IF;
254      END_IF;
255    END_IF;
256  END_IF;
257  IF RwMatLocked THEN
258    LongValid := FALSE;
259    LearnFailCnt := 0;                    (* no thickness alarm *)
260  END_IF;
261
262  (* ---- 4. Learn material factor  K = dDiam/dPulse * Diam ---------- *)
263  IF Tick THEN
264    LearnWinTicks := LearnWinTicks + 1;
265    IF NOT LearnInit THEN
266      LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiamRev;  (* V9 *)
267      LearnWinTicks := 0;
268      LearnWinRunning := O_MotorStart;
269      LearnInit := TRUE;
270    ELSE
271      LearnWinRunning := LearnWinRunning AND O_MotorStart;
272      (* v6w: LENGTH-based window, modulated by the diameter: closes    *)
273      (* when the roll should have grown RwWinGrow_mm (2.5 mm), so the  *)
274      (* accuracy vs DT35 noise is the same at every speed and size.    *)
275      (* Pulses = growth x diameter / K (K = learned RwMat_K).          *)
276      (* Limits: min 4 m, max RwWinLen_m (30 m). K not learned: max.    *)
277      (* Motor stopped during the window -> window dropped (no fault).  *)
278      IF MmPerPulse > 0.0 THEN
279        IF (RwMat_K > 0.0) AND (RwWinGrow_mm > 0.0) THEN
280          LearnWinLen_Pls := RwWinGrow_mm * MAX(RwDiam, REAL#100.0) /
   RwMat_K;
281        ELSE
282          LearnWinLen_Pls := RwWinLen_m * 1000.0 / MmPerPulse;
```

```
283             END_IF;
284             IF LearnWinLen_Pls < 4000.0 / MmPerPulse THEN
285                 LearnWinLen_Pls := 4000.0 / MmPerPulse;          (* 4 m *)
286             END_IF;
287             IF (RwWinLen_m > 0.0)
288               AND (LearnWinLen_Pls > RwWinLen_m * 1000.0 / MmPerPulse) THEN
289                 LearnWinLen_Pls := RwWinLen_m * 1000.0 / MmPerPulse;
290             END_IF;
291         ELSE
292             LearnWinLen_Pls := 300000.0;
293         END_IF;
294         IF (DINT_TO_REAL(ABS(Encoder_Pos - LearnWinStartPos)) >= LearnWinLen_Pls)
295           OR NOT LearnWinRunning THEN
296             LearnWin_Pls := ABS(Encoder_Pos - LearnWinStartPos);
297             LearnWinDiamDelta := RwDiamRev - LearnWinStartDiam;  (* V9 *)
298             (* A window SHOULD learn: ran all the time, enough length.  *)
299             (* DT35 OK. Then it must give a plausible paper thickness. *)
300             IF LearnWinRunning AND (LearnWin_Pls >= RwMinPulses)
301               AND NOT RwSensFault
302               AND NOT RwMatLocked AND NOT LongValid     (* V9 *)
303               AND RwDiamRevOk AND (LearnWinStartDiam > 0.0) THEN
304                 LearnWinOk := FALSE;
305                 IF (LearnWinDiamDelta > 0.0) AND (MmPerPulse > 0.0) THEN
306                     MatK_Window := (LearnWinDiamDelta / DINT_TO_REAL(LearnWin_Pls))
307                                 * ((RwDiamRev + LearnWinStartDiam) / 2.0);
308                     (* K = 2 * thickness * mm-per-pulse / PI         *)
309                     ThickWin_mm := MatK_Window * 3.14159 / (2.0 * MmPerPulse);
310                     IF (ThickWin_mm >= RwThickMin_mm)
311                       AND (ThickWin_mm <= RwThickMax_mm) THEN
312                         LearnWinOk := TRUE;
313                         IF RwMat_K <= 0.0 THEN
314                             RwMat_K := MatK_Window;
315                             MatChangeCnt := 0;
316                         ELSIF (RwMatChangePct > 0.0)
317                             AND (ABS(MatK_Window - RwMat_K)
318                                 > RwMat_K * RwMatChangePct / 100.0) THEN
319                             (* v6s: far from the learned value: 1 window = ignored *)
320                             (* (outlier). 2 in a row = paper changed -> take it.  *)
321                             MatChangeCnt := MatChangeCnt + 1;
322                             IF MatChangeCnt >= 2 THEN
323                                 RwMat_K := MatK_Window;
324                                 MatChangeCnt := 0;
325                             END_IF;
326                         ELSE
327                             MatChangeCnt := 0;
328                             RwMat_K := RwMat_K + RwLearn_K * (MatK_Window - RwMat_K);
329                         END_IF;
330                     END_IF;
331                 END_IF;
332                 IF LearnWinOk THEN
333                     LearnFailCnt := 0;
334                 ELSIF LearnFailCnt < 1000 THEN
335                     LearnFailCnt := LearnFailCnt + 1;  (* no growth / implausible *)
336                 END_IF;
337             END_IF;
338             LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiamRev;  (* V9
    *)
339             LearnWinTicks := 0;
340             LearnWinRunning := O_MotorStart;
```

```
        END_IF;
      END_IF;
    END_IF;

(* ---- 5. Learn real stop deceleration on every stop ---------- *)
(* v6d: a run with jog / manual / slow button uses the 2nd ramp (O_AccDec2) *)
(*      -> its stop must not be learned. Latched for the whole run because  *)
(*      the jog button is already released on the stop edge.                *)
IF O_MotorStart AND NOT MotorRunMem THEN          (* start edge *)
    RunWasJog := FALSE;
END_IF;
IF O_MotorStart AND (ManMode OR JogRun) THEN
    RunWasJog := TRUE;
END_IF;

IF MotorRunMem AND NOT O_MotorStart THEN          (* stop command edge *)
    StopMeasActive := TRUE;
    StopStartPos := Encoder_Pos;
    StopStartSpd_Pps := SpdAct_Pps;
    StopStartDiam := DecelDiam;
    StandstillTicks := 0;
    (* STOP button and automatic stop take the same path (drive DI1 and    *)
    (* O_Brake -> T-one drop together, electrical plan sheets 1/9/15),     *)
    (* so both are learned.                                                *)
    StopLearnValid := ControlVoltage AND NOT RunWasJog AND NOT ManMode;
END_IF;
MotorRunMem := O_MotorStart;

IF O_MotorStart THEN
    StopMeasActive := FALSE;                  (* restarted: abort *)
ELSIF StopMeasActive AND Tick THEN
    IF NOT ControlVoltage THEN
        StopLearnValid := FALSE;              (* E-stop during the stop *)
    END_IF;
    IF SpdAct_Pps <= 20.0 THEN                (* <= 2 counts / 100 ms *)
        StandstillTicks := StandstillTicks + 1;
    ELSE
        StandstillTicks := 0;
    END_IF;
    IF StandstillTicks >= 3 THEN              (* standstill 300 ms *)
        StopMeasActive := FALSE;
        StopMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - StopStartPos));
        IF RwLearnStop AND StopLearnValid
          AND (StopStartSpd_Pps * MmPerPulse * 0.06 >= RwMinLearnSpd) THEN
            (* This stop SHOULD learn: result must be plausible      *)
            (* v6u: a stop shorter than the reaction distance, or    *)
            (* faster than RwDecelMax_mm, is NOT a fault: the web    *)
            (* stopped within a few mm, nothing to learn (trace      *)
            (* 10/28/2026: 23 m/min -> stopped in 102 pulses = 10 mm). *)
            (* Only a too SLOW stop (< RwDecelMin_mm) counts as fail. *)
            StopLearnOk := FALSE;
            IF StopMeas_Pls > StopStartSpd_Pps * RwTreact THEN
                StopDecelMeas := (StopStartSpd_Pps * StopStartSpd_Pps)
                              / (2.0 * (StopMeas_Pls - StopStartSpd_Pps * RwTreact));
                StopDecelMeas_mm := StopDecelMeas * MmPerPulse;  (* mm/s2 *)
                IF (StopDecelMeas_mm >= RwDecelMin_mm)
                  AND (StopDecelMeas_mm <= RwDecelMax_mm) THEN
                    StopLearnOk := TRUE;
                (* v6g: learn per mm of diameter *)
```

```
                (* v6g: learn per mm of diameter *)
                DecelK_Meas := StopDecelMeas_mm / ((StopStartDiam + DecelDiam) / 2.0);
                IF RwStopDecelK <= 0.0 THEN
                    RwStopDecelK := DecelK_Meas;
                ELSE
                    RwStopDecelK := RwStopDecelK + 0.3 * (DecelK_Meas - RwStopDecelK);
                END_IF;
                StopFailCnt := 0;
            ELSIF StopDecelMeas_mm < RwDecelMin_mm THEN
                IF StopFailCnt < 1000 THEN
                    StopFailCnt := StopFailCnt + 1;    (* stop too long *)
                END_IF;
            END_IF;                                    (* too fast: ignored *)
        END_IF;                                        (* too short: ignored *)
    END_IF;
  END_IF;
END_IF;

(* ---- 5b. v6f: learn real slow-down deceleration ----------------------- *)
(* From the SlowDown edge (at speed) until final speed is reached, or     *)
(* until the stop comes first. a = (v0^2 - v1^2) / (2 * distance).        *)
(* Not learned in jog / manual / slow button (2nd ramp).                  *)
IF SlowDown AND NOT SlowMem AND O_MotorStart
  AND NOT ManMode AND NOT JogRun
  AND (SpdAct_Pps > SpdFinal_Pps + SpdBand_Pps) THEN
    SlowMeasActive := TRUE;
    SlowStartPos := Encoder_Pos;
    SlowStartSpd_Pps := SpdAct_Pps;
    SlowStartDiam := DecelDiam;
END_IF;
SlowMem := SlowDown;

IF SlowMeasActive THEN
  IF NOT SlowDown OR ManMode OR JogRun THEN
    SlowMeasActive := FALSE;                   (* aborted *)
  ELSIF Tick AND ((SpdAct_Pps <= SpdFinal_Pps + 0.25 * SpdBand_Pps)
                  OR NOT O_MotorStart) THEN      (* final reached or stopped *)
    SlowMeasActive := FALSE;
    SlowMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - SlowStartPos));
    IF (SlowMeas_Pls > 0.0) AND (MmPerPulse > 0.0)
      AND ((SlowStartSpd_Pps - SpdAct_Pps) * MmPerPulse * 0.06 >= 10.0) THEN
      SlowDecelMeas_mm := (SlowStartSpd_Pps * SlowStartSpd_Pps
                           - SpdAct_Pps * SpdAct_Pps)
                          / (2.0 * SlowMeas_Pls) * MmPerPulse;  (* mm/s2 *)
      IF (SlowDecelMeas_mm >= RwDecelMin_mm)
        AND (SlowDecelMeas_mm <= RwDecelMax_mm) THEN
        (* v6g: learn per mm of diameter *)
        DecelK_Meas := SlowDecelMeas_mm / ((SlowStartDiam + DecelDiam) / 2.0);
        IF RwSlowDecelK <= 0.0 THEN
            RwSlowDecelK := DecelK_Meas;
        ELSE
            RwSlowDecelK := RwSlowDecelK + 0.5 * (DecelK_Meas - RwSlowDecelK);
        END_IF;
      END_IF;
    END_IF;
```

```
454     END_IF;
455 END_IF;
456
457 (* ---- 5c. v6g: live decels = learned K x current diameter ---------- *)
458 (* v6p: the one-time v6f seed is removed (it re-filled RwStopDecelK from    *)
459 (* RwStopDecel, so the learning could never be reset).                     *)
460 (* RwLearnReset (HMI / Watch tab, PLC resets it): clear the learned decels. *)
461 (* Both are re-learned at the next auto slow-down / stop >= RwMinLearnSpd.  *)
462 IF RwLearnReset THEN
463     RwStopDecelK := 0.0;
464     RwStopDecel := 0.0;
465     RwSlowDecelK := 0.0;
466     StopFailCnt := 0;
467     RwLearnReset := FALSE;
468 END_IF;
469 IF (RwStopDecelK > 0.0) AND (MmPerPulse > 0.0) THEN
470     RwStopDecel := RwStopDecelK * DecelDiam / MmPerPulse;     (* pulses/s2 *)
471 END_IF;
472 IF RwSlowDecelK > 0.0 THEN
473     RwSlowDecelLrn := RwSlowDecelK * DecelDiam;               (* mm/s2 *)
474 ELSE
475     RwSlowDecelLrn := 0.0;
476 END_IF;
477
```

1 Auto Stop Process 6-14

```
(*
 ============================================================
 ====================
   AutoStop_Control  V12  -  PART 2 of 2 (rung 1): sections 6 - 14c
   V12: END-OF-ROLL STOP RAMP: auto / diameter stops ramp down at
 FinalStopDecel
        (soft stop mechanism) instead of the drive DEC ramp (3 s = 20 Hz/s,
        up to ~8 m/s2 of web on a big roll): stop lengths use it (6, 11b).
   V11: 3-state unwinder display + stop command at UwStop_at_D_Val (section 8).
   (Sysmac: an inline ST rung holds max 1000 rows -> split in 2 rungs.
    Rung 0 = part 1 runs first, then this rung, every scan.)

 ============================================================
 ==================== *)

(* ---- 6. Lengths in pulses: stop, slow-down, remaining ------------------ *)
(* Stop length uses the ACTUAL speed (precision)                          *)

IF RwStopDecel > 0.0 THEN
   StopLen_Pls := SpdAct_Pps * RwTreact
                + (SpdAct_Pps * SpdAct_Pps) / (2.0 * RwStopDecel);
ELSE
   StopLen_Pls := SpdAct_Pps * (RwTreact + RwTstopDef / 2.0);
END_IF;

(* V12: end-of-roll stop ramp at FinalStopDecel (web mm/s2) + S-curve    *)

IF (FinalStopDecel > 0.0) AND FFEnable AND NOT ManMode AND (MmPerPulse >
 0.0) THEN
   StopLen_Pls := SpdAct_Pps * RwTreact
                + (SpdAct_Pps * SpdAct_Pps) / (2.0 * FinalStopDecel / MmPerPulse)
                + SpdAct_Pps * 0.5 * SCurveTime;
END_IF;

StopLen_Pls := StopLen_Pls * RwTrim;

(* Slow-down uses SpdRef = MAX(actual, work-speed setpoint):              *)
(* stable value, shown to the operator even before start                  *)

SlowLenBase_Pls := 0.0;  RwSlowLen_Pls := 0.0;  UwSlowLen_Pls := 0.0;

IF MmPerPulse > 0.0 THEN
   (* v6f: manual > learned > PLC ramp model                              *)
   (* V7: manual > feed-forward (one ramp at DecelWeb, real web mm/s2)  *)
   (*     > old loop with web ramps (coarse DecelWeb + trim tail)       *)
   (*     > learned > old PLC ramp model                                 *)

   IF RwSlowDecel > 0.0 THEN
      SlowDecel_Pps2 := RwSlowDecel / MmPerPulse;
      SlowOneRamp := TRUE;
   ELSIF FFEnable AND (DecelWeb > 0.0) AND (WebRatio > 0.0) AND NOT
 ManMode THEN
      SlowDecel_Pps2 := DecelWeb / MmPerPulse;
      SlowOneRamp := TRUE;
   ELSIF (DecelWeb > 0.0) AND (WebRatio > 0.0) AND NOT ManMode THEN  (* V7e
 *)
      SlowDecel_Pps2 := DecelWeb / MmPerPulse;
      SlowOneRamp := FALSE;
```

```
51    ELSIF RwSlowDecelLrn > 0.0 THEN
52       SlowDecel_Pps2 := RwSlowDecelLrn / MmPerPulse;  (* real, incl. trim tail *)
53       SlowOneRamp := TRUE;
54    ELSE
55       SlowDecel_Pps2 := DecelCoarse * 250.0 / MmPerPulse; (* 1 = 250 mm/s2 *)
56       SlowOneRamp := FALSE;
57    END_IF;
58
59    (* v6d: DecelTrim ramp used inside the +/- SpdDelta_Percent band *)
60
61    IF (DecelTrim > 0.0) AND (WebRatio > 0.0) AND (Vel_Max_mms > 0.0) THEN
62       (* V7: trim ramp (reference units) converted to the real web decel *)
63       TrimDecel_Pps2 := DecelTrim * 1000000.0 / (0.06 * WebRatio * Vel_Max_mms)
64                         / MmPerPulse;
65    ELSIF DecelTrim > 0.0 THEN
66       TrimDecel_Pps2 := DecelTrim * 250.0 / MmPerPulse;
67    ELSE
68       TrimDecel_Pps2 := SlowDecel_Pps2;
69    END_IF;
70
71    SpdBand_Pps := SpdDelta_Percent * 1000.0 / 60.0 / MmPerPulse;
72
73    (* v6d: speeds from the potentiometers, valid in auto AND ManMode     *)
74    (*      (m/min = Pot / Input_Ratio, same as Motor_Manager rung 8)     *)
75
76    IF (Input_Ratio > 0.0) AND ManMode THEN
77       (* v6i: ManMode drives the inverter with the raw pots (rung 17) *)
78       SpdFinal_Pps := INT_TO_REAL(Pot2_Slow_Velocity) / Input_Ratio * 1000.0 /
   60.0 / MmPerPulse;
79       SpdWork_Pps  := INT_TO_REAL(Pot1_Work_Velocity) / Input_Ratio * 1000.0 /
   60.0 / MmPerPulse;
80    ELSIF Input_Ratio > 0.0 THEN
81       SpdFinal_Pps := INT_TO_REAL(Pot2_Mem) / Input_Ratio * 1000.0 / 60.0 /
   MmPerPulse;
82       SpdWork_Pps  := INT_TO_REAL(Pot1_Mem) / Input_Ratio * 1000.0 / 60.0 /
   MmPerPulse;
83    ELSE
84       SpdFinal_Pps := LowSpd_Display * 1000.0 / 60.0 / MmPerPulse;
85       SpdWork_Pps  := Speed_Display  * 1000.0 / 60.0 / MmPerPulse;
86    END_IF;
87
88    (* v6y: rewinder AUTO stop -> work speed limited to RwAutoMaxSpdAct *)
89    (* (limit + value taken over only at standstill, Motor_Manager rung 5) *)
90
91    IF RwAutoLimAct AND NOT ManMode
92       AND (SpdWork_Pps > RwAutoMaxSpdAct * 1000.0 / 60.0 / MmPerPulse) THEN
93       SpdWork_Pps := RwAutoMaxSpdAct * 1000.0 / 60.0 / MmPerPulse;
94    END_IF;
95
96    SpdRef_Pps := MAX(SpdAct_Pps, SpdWork_Pps);
97
98    IF RwStopDecel > 0.0 THEN
99       StopLenFinal_Pls := SpdFinal_Pps * RwTreact
100                         + (SpdFinal_Pps * SpdFinal_Pps) / (2.0 * RwStopDecel);
101   ELSE
102      StopLenFinal_Pls := SpdFinal_Pps * (RwTreact + RwTstopDef / 2.0);
103   END_IF;
104
105   (* V12: final stop from the final speed on the end-of-roll ramp *)
```

```
106
107   IF (FinalStopDecel > 0.0) AND FFEnable AND NOT ManMode THEN
108      StopLenFinal_Pls := SpdFinal_Pps * RwTreact
109                        + (SpdFinal_Pps * SpdFinal_Pps) / (2.0 * FinalStopDecel /
      MmPerPulse)
110                        + SpdFinal_Pps * 0.5 * SCurveTime;
111   END_IF;
112
113   SlowLenBase_Pls := StopLenFinal_Pls * RwTrim;
114
115   IF (SpdRef_Pps > SpdFinal_Pps) AND (SlowDecel_Pps2 > 0.0) THEN
116
117      IF SlowOneRamp OR (TrimDecel_Pps2 <= 0.0) THEN
118         (* manual or learned decel: one constant ramp *)
119         SlowLenBase_Pls := SlowLenBase_Pls
120            + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
121             / (2.0 * SlowDecel_Pps2);
122      ELSE
123         (* auto: coarse ramp down to the band, trim ramp inside it *)
124         SpdKnee_Pps := SpdFinal_Pps + SpdBand_Pps;
125
126         IF SpdRef_Pps > SpdKnee_Pps THEN
127            SlowLenBase_Pls := SlowLenBase_Pls
128               + (SpdRef_Pps * SpdRef_Pps - SpdKnee_Pps * SpdKnee_Pps)
129                / (2.0 * SlowDecel_Pps2)
130               + (SpdKnee_Pps * SpdKnee_Pps - SpdFinal_Pps * SpdFinal_Pps)
131                / (2.0 * TrimDecel_Pps2);
132         ELSE
133            SlowLenBase_Pls := SlowLenBase_Pls
134               + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
135                / (2.0 * TrimDecel_Pps2);
136         END_IF;
137      END_IF;
138   END_IF;
139
140   (* V7f: S-curve rounding: the slow-down ends SCurveTime later ->      *)
141   (*      extra length = (SpdRef - SpdFinal) / 2 x SCurveTime          *)
142
143   IF FFEnable AND NOT ManMode AND (RwSlowDecel <= 0.0) AND (DecelWeb >
      0.0)
144      AND (WebRatio > 0.0) AND (SCurveTime > 0.0)
145      AND (SpdRef_Pps > SpdFinal_Pps) THEN
146      SlowLenBase_Pls := SlowLenBase_Pls
147                        + (SpdRef_Pps - SpdFinal_Pps) * 0.5 * SCurveTime;
148   END_IF;
149
150   RwSlowLen_Pls := SlowLenBase_Pls + RwSlowMargin_m * 1000.0 / MmPerPulse;
151
152   IF UwSlowMargin_m > 0.0 THEN
153      UwMarginUsed_m := UwSlowMargin_m;
154   ELSE
155      UwMarginUsed_m := RwSlowMargin_m;
156   END_IF;
157
158   UwSlowLen_Pls := SlowLenBase_Pls + UwMarginUsed_m * 1000.0 / MmPerPulse;
159 END_IF;
160
161 IF RwDiam >= RwTargetDiam THEN
```

```
ELSIF RwMat_K > 0.0 THEN
    RwRemain_Pls := (RwTargetDiam * RwTargetDiam - RwDiam * RwDiam)
                  / (2.0 * RwMat_K);
ELSE
    RwRemain_Pls := 1.0E9;                    (* K not learned: plain target *)
END_IF;

RwRemain_m  := RwRemain_Pls  * MmPerPulse / 1000.0;
RwStopLen_m := StopLen_Pls   * MmPerPulse / 1000.0;
RwSlowLen_m := RwSlowLen_Pls * MmPerPulse / 1000.0;

(* ---- 6b. Live operator displays ---------------------------------------- *)
(* Rewinder: diameter where slow-down / stop command will happen          *)

IF RwMat_K > 0.0 THEN
    SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
    IF SqrtArg > 0.0 THEN
        RwSlow_at_D_Val := SQRT(SqrtArg);
    ELSE
        RwSlow_at_D_Val := 0.0;
    END_IF;

    SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * StopLen_Pls;

    IF SqrtArg > 0.0 THEN
        RwStop_at_D_Val := SQRT(SqrtArg);
    ELSE
        RwStop_at_D_Val := 0.0;
    END_IF;
ELSE
    (* v6t: thickness not learned: fixed slow-down point before the target *)

    IF RwUseSlow AND (RwSlowFallbackMm > 0.0) THEN
        RwSlow_at_D_Val := RwTargetDiam - RwSlowFallbackMm;
    ELSE
        RwSlow_at_D_Val := RwTargetDiam;
    END_IF;
    RwStop_at_D_Val := RwTargetDiam;
END_IF;

(* Unwinder: meter count where the slow-down will start                   *)

IF MmPerPulse > 0.0 THEN
    UwSlow_at_M_Val := UwStop_at_M_Val - UwSlowLen_Pls * MmPerPulse / 1000.0;
ELSE
    UwSlow_at_M_Val := UwStop_at_M_Val;
END_IF;

IF UwSlow_at_M_Val < 0.0 THEN
    UwSlow_at_M_Val := 0.0;
END_IF;

(* Unwinder slow-down request, latched                                    *)

IF (Current_Metre < UwSlow_at_M_Val - 50.0) OR (UwStop_at_M_Val <= 0.0) THEN
    UwSlow_at_M_Req := FALSE;                 (* counter reset / longer roll *)
END_IF;
```

```
221 IF (UwStop_at_M_Val > 0.0) AND (Current_Metre >= UwSlow_at_M_Val) THEN
222     UwSlow_at_M_Req := TRUE;
223 END_IF;
224
225 (* ---- 6c. V7c: unwinder DIAMETER slow-down follows the speed ---------- *)
226 (* Works without AutoStop (only Measure_ON, like General rung 16).       *)
227 (* Paper left above the stop diameter, in pulses:                         *)
228 (*     (UwDiam_Act^2 - UwStopDiam_Limit^2) / (2 x RwMat_K)                *)
229 (* (same paper as the rewinder -> same learned K). Slow down when it is   *)
230 (* shorter than UwSlowLen_Pls = decel from the speed (DecelWeb) + final   *)
231 (* stop + UwSlowMargin_m. Solved for the diameter:                        *)
232 (*     UwSlowDiamCalc = SQRT(UwStopDiam_Limit^2 + 2 x RwMat_K x
    UwSlowLen_Pls)   *)
233 (* K not learned / UwSlow_at_D_Req OFF -> fixed UwSlow_at_D_Val as before. *)
234 (* Latched: once slowed down, the point is frozen (+10 mm hysteresis), so *)
235 (* the lower speed does not move it back and speed up again. Released by  *)
236 (* a new roll (diameter 30 mm above the frozen point).                    *)
237
238 IF UwSlow_at_D_Req AND (RwMat_K > 0.0) AND (UwStopDiam_Limit > 0.0)
239    AND (UwSlowLen_Pls > 0.0) THEN
240     SqrtArg := UwStopDiam_Limit * UwStopDiam_Limit + 2.0 * RwMat_K *
    UwSlowLen_Pls;
241     UwSlowDiamCalc := SQRT(SqrtArg);            (* same pattern as 6b *)
242 ELSE
243     UwSlowDiamCalc := UwSlow_at_D_Val;
244 END_IF;
245
246 IF UwSlowDiamCalc < UwStopDiam_Limit + 10.0 THEN
247     UwSlowDiamCalc := UwStopDiam_Limit + 10.0;
248 ELSIF UwSlowDiamCalc > 1500.0 THEN
249     UwSlowDiamCalc := 1500.0;
250 END_IF;
251
252 IF UwSlowDiamLatch AND (UwDiam_Act > UwSlowDiamAct + 30.0) THEN
253     UwSlowDiamLatch := FALSE;                   (* new roll *)
254 END_IF;
255
256 IF NOT UwSlowDiamLatch THEN
257     UwSlowDiamAct := UwSlowDiamCalc;            (* live point, HMI *)
258     IF Measure_ON AND (UwDiam_Act > 0.0) AND (UwDiam_Act <=
    UwSlowDiamCalc) THEN
259         UwSlowDiamLatch := TRUE;
260         UwSlowDiamAct := UwSlowDiamCalc + 10.0;     (* frozen + hysteresis *)
261     END_IF;
262 END_IF;
263
264 (* ---- 7. Rewinder requests: latched, reset when a new core is loaded ---- *)
265
266 IF NOT RwStopEnable
267    OR ((RwDiam < RwTargetDiam - RwResetHyst) AND NOT RwSensFault) THEN
268     RwStop_at_D_Req := FALSE;
269     RwSlow_at_D_Req := FALSE;
270 END_IF;
271
272 IF RwStopEnable THEN
273     IF (RwRemain_Pls <= StopLen_Pls) OR (RwSensFault AND RwStopOnFault) THEN
274         RwStop_at_D_Req := TRUE;
275     END_IF;
276     IF RwUseSlow AND (RwRemain_Pls <= RwSlowLen_Pls) THEN
```

```
277         RwSlow_at_D_Req := TRUE;
278     END_IF;
279
280     (* v6t: thickness not learned (RwMat_K = 0): slow down at a fixed      *)
281     (* distance before the target instead of stopping from full speed.    *)
282
283     IF RwUseSlow AND (RwMat_K <= 0.0) AND (RwSlowFallbackMm > 0.0)
284       AND NOT RwSensFault
285       AND (RwDiam >= RwTargetDiam - RwSlowFallbackMm) THEN
286         RwSlow_at_D_Req := TRUE;
287     END_IF;
288 END_IF;
289
290 (* ---- 8. One AUTO at a time + single "SLOW DOWN AT" display ---------- *)
291
292 IF UwStopEnable AND NOT UwEnMem THEN              (* unwinder AUTO switched
    ON *)
293     RwStopEnable := FALSE;
294 END_IF;
295
296 IF RwStopEnable AND NOT RwEnMem THEN              (* rewinder AUTO switched
    ON *)
297     UwStopEnable := FALSE;
298 END_IF;
299
300 UwEnMem := UwStopEnable;
301 RwEnMem := RwStopEnable;
302
303 IF RwStopEnable THEN
304     SlowAtDisp := RwSlow_at_D_Val;                (* mm *)
305     SlowAtMode := 2;
306 ELSIF UwStopEnable THEN
307     SlowAtDisp := UwSlow_at_M_Val;                (* m  *)
308     SlowAtMode := 1;
309 ELSE
310     SlowAtDisp := 0.0;
311     SlowAtMode := 0;                              (* hide display *)
312 END_IF;
313
314 SlowAtShow := (SlowAtMode <> 0);
315
316 (* V11: unwinder column, same place, 3 states:                          *)
317 (*   AUTO STOP UNWINDER (12SA5 + unwinder AUTO) -> STOP LENGTH [m]      *)
318 (*   normal run, before the slow-down          -> SLOW AT Ø [mm]        *)
319 (*   slow-down reached (decel / final speed)   -> STOP AT Ø [mm]        *)
320 (* V11: stop by diameter EARLIER, so the web is stopped AT UwStopDiam_Limit:  *)
321 (* the stop command comes at UwStop_at_D_Val = UwStopDiam_Limit + the
    diameter   *)
322 (* of the paper unwound during the stop from the ACTUAL speed           *)
323 (* (StopLen_Pls, section 6, learned stop decel):                        *)
324 (*   UwStop_at_D_Val = SQRT(UwStopDiam_Limit^2 + 2 x RwMat_K x StopLen_Pls)
    *)
325 (* General rung 18 (UwStopByDiam) compares with UwStop_at_D_Val.        *)
326 (* K not learned -> UwStopDiam_Limit (as before). Max UwStopDiam_Limit + 100
    mm.    *)
327
328 UwStopLenShow  := I_AutoStop_OS AND UwStopEnable;
329 UwSlowDiamShow := NOT UwStopLenShow AND Measure_ON AND NOT
    UwSlowDiamLatch;
```

```
330  UwStopDiamShow := NOT UwStopLenShow AND Measure_ON AND
     UwSlowDiamLatch;
331
332  IF (RwMat_K > 0.0) AND (StopLen_Pls > 0.0) THEN
333      SqrtArg := UwStopDiam_Limit * UwStopDiam_Limit + 2.0 * RwMat_K *
     StopLen_Pls;
334      UwStop_at_D_Val := SQRT(SqrtArg);
335  ELSE
336      UwStop_at_D_Val := UwStopDiam_Limit;
337  END_IF;
338
339  IF UwStop_at_D_Val > UwStopDiam_Limit + 100.0 THEN
340      UwStop_at_D_Val := UwStopDiam_Limit + 100.0;
341  ELSIF UwStop_at_D_Val < UwStopDiam_Limit THEN
342      UwStop_at_D_Val := UwStopDiam_Limit;
343  END_IF;
344
345  (* ---- 9. Automatic stop master = selector 12SA5 (I_AutoStop_OS) -------- *)
346
347  AutoSlowCmd := I_AutoStop_OS AND
348              ((UwStopEnable AND UwSlow_at_M_Req) OR (RwStopEnable AND
     RwSlow_at_D_Req));
349
350  AutoStopCmd := I_AutoStop_OS AND
351              ((UwStopEnable AND (UwStop_at_M_Val > 0.0)
352                AND (Current_Metre >= UwStop_at_M_Val))
353               OR (RwStopEnable AND RwStop_at_D_Req));
354
355  (* ---- 10. Unwinder roll empty (for alarm 15) ---------------------------- *)
356
357  IF UwDiam_Act > (UwStopDiam_Limit + UwEmptyHyst) THEN
358      UnWinder_Empty := FALSE;                    (* new roll loaded *)
359  END_IF;
360
361  IF (UwDiam_Act > 0.0) AND (UwDiam_Act <= UwStopDiam_Limit) THEN
362      UnWinder_Empty := TRUE;                     (* roll empty *)
363  END_IF;
364
365  (* v6x: new parent roll loaded (empty -> not empty) -> HMI popup        *)
366
367  (* "DID YOU CHANGE THE PAPER (GSM)?". YES = forget the learned thickness *)
368  (* (same as NEW PAPER). NO = the HMI resets GsmAsk.                      *)
369  (* V9: ask only ONCE per REAL new roll: the roll must have been empty for *)
370  (* 3 s (arms the question), then the new roll must read more than        *)
371  (* UwStopDiam_Limit + 100 mm for 3 s. The sonar bouncing around
     UwStopDiam_Limit     *)
372  (* (empty core, hands, arms) no longer re-opens the popup.               *)
373
374  IF Tick THEN
375      IF UnWinder_Empty THEN
376
377          IF UwEmptyTicks < 1000 THEN
378              UwEmptyTicks := UwEmptyTicks + 1;
379          END_IF;
380
381          IF UwEmptyTicks >= 30 THEN
382              GsmArmed := TRUE;
383          END_IF;
384
```

```
385         UwFullTicks := 0;
386       ELSE
387         UwEmptyTicks := 0;
388
389         IF GsmArmed AND Measure_ON AND (UwDiam_Act > UwStopDiam_Limit +
    100.0) THEN
390           UwFullTicks := UwFullTicks + 1;
391
392           IF UwFullTicks >= 30 THEN
393             GsmAsk := TRUE;
394             GsmArmed := FALSE;
395             UwFullTicks := 0;
396           END_IF;
397         ELSE
398           UwFullTicks := 0;
399         END_IF;
400       END_IF;
401     END_IF;
402
403     UwEmptyMem := UnWinder_Empty;
404
405   IF GsmAskYes THEN
406       RwMat_K := 0.0;
407       RwMatLocked := FALSE;                    (* V9 *)
408       LongActive := FALSE;
409       LearnFailCnt := 0;
410       MatChangeCnt := 0;
411       GsmAskYes := FALSE;
412       GsmAsk := FALSE;
413   END_IF;
414
415   (* V10: HMI popup "UNWINDER DIAMETER SENSOR SONAR IS OFF ! DO YOU WANT
    TO  *)
416   (* TURN IT ON ?" - only at standstill: at power-up and after every stop    *)
417   (* while Measure_ON = FALSE. YES (SonarAskYes, PLC resets it) = sonar ON:  *)
418   (* NO = the HMI resets SonarAsk (asked again after the next stop).        *)
419
420   IF Measure_ON THEN
421       SonarAsk := FALSE;                       (* sonar ON: no popup *)
422   ELSIF NOT StartMotor_Cmd AND (SonarRunMem OR NOT SonarInit) THEN
423       SonarAsk := TRUE;                        (* stopped / power-up *)
424   END_IF;
425
426     SonarInit := TRUE;
427     SonarRunMem := StartMotor_Cmd;
428
429   IF StartMotor_Cmd THEN
430       SonarAsk := FALSE;                       (* never while running *)
431   END_IF;
432
433   IF SonarAskYes THEN
434       Measure_ON := TRUE;
435       SonarAsk := FALSE;
436       SonarAskYes := FALSE;
437   END_IF;
438
439   (* ---- 11. Rewinder max diameter protection - ALWAYS active ---------- *)
440   (* Independent of selector 12SA5 and of the AUTO buttons.                 *)
441   (* Mechanical limit 680 mm radius = 1360 mm                               *)
```

```
442
443 IF NOT RwSensFault THEN
444     RwLastValidDiam := RwDiam;
445 END_IF;
446
447 IF RwDiam >= RwMaxDiam THEN
448     RwRemainMax_Pls := 0.0;
449 ELSIF RwMat_K > 0.0 THEN
450     RwRemainMax_Pls := (RwMaxDiam * RwMaxDiam - RwDiam * RwDiam)
451                        / (2.0 * RwMat_K);
452 ELSE
453     RwRemainMax_Pls := 1.0E9;                    (* K not learned: plain limit *)
454 END_IF;
455
456 (* Display: diameter where the max-protection slow-down will start        *)
457
458 IF RwMat_K > 0.0 THEN
459     SqrtArg := RwMaxDiam * RwMaxDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
460     IF SqrtArg > 0.0 THEN
461         RwMaxSlowAtDiam := SQRT(SqrtArg);
462     ELSE
463         RwMaxSlowAtDiam := 0.0;
464     END_IF;
465 ELSIF RwMaxSlowFallback > 0.0 THEN
466     RwMaxSlowAtDiam := RwMaxSlowFallback;        (* fallback in use *)
467 ELSE
468     RwMaxSlowAtDiam := RwMaxDiam;
469 END_IF;
470
471 (* Reset only after the roll is removed (diameter well below the limit)    *)
472
473 IF (RwDiam < RwMaxDiam - RwResetHyst) AND NOT RwSensFault THEN
474     RwMaxStop := FALSE;
475     RwMaxSlow := FALSE;
476 END_IF;
477
478 IF NOT RwSensFault THEN
479
480     IF RwRemainMax_Pls <= StopLen_Pls THEN
481         RwMaxStop := TRUE;                       (* speed-compensated stop *)
482     END_IF;
483
484     IF RwRemainMax_Pls <= RwSlowLen_Pls THEN
485         RwMaxSlow := TRUE;                       (* warning slow-down + horn *)
486     END_IF;
487
488     IF (RwMat_K <= 0.0) AND (RwMaxSlowFallback > 0.0)
489       AND (RwDiam >= RwMaxSlowFallback) THEN
490         RwMaxSlow := TRUE;                       (* fallback while K not learned *)
491     END_IF;
492
493 ELSIF (AI_Diam_Rewinder < RwRawMin) AND (RwLastValidDiam > RwMaxDiam -
    100.0) THEN
494     RwMaxStop := TRUE;                           (* roll grew into the DT35 near zone *)
495 END_IF;
496
497 (* Max diameter protection added OUTSIDE the 12SA5 master (section 9)      *)
498
499 AutoSlowCmd := AutoSlowCmd OR RwMaxSlow;
```

```
500  AutoStopCmd := AutoStopCmd OR RwMaxStop;
501
502 (* ---- 11b. V12: END-OF-ROLL STOP RAMP ---------------------------------- *)
503 (* Normal end-of-roll stops (AutoStopCmd: unwinder metres, rewinder target,*)
504 (* rewinder max diameter; UwStopByDiam: unwinder diameter) no longer drop  *)
505 (* the run command at once (drive DEC ramp 3 s = 20 Hz/s -> up to 8 m/s2  *)
506 (* of web on a big roll, hard on the rewinder gearbox). With the feed-    *)
507 (* forward in AUTO they start the SOFT STOP at FinalStopDecel (General    *)
508 (* rung 10); the run command drops below SoftStopEndSpd. START cannot     *)
509 (* cancel it. The stop lengths (section 6) use the same ramp, so the web  *)
510 (* still stops at the target / UwStopDiam_Limit.                          *)
511 (* Immediate stop (EndStopNow -> Motor_Manager rung 4 Stop):              *)
512 (*   ramp not usable (FinalStopDecel 0, FFEnable OFF, ManMode, JOG, the   *)
513 (*   motor stopped = start blocked as before), or safety net: unwinder    *)
514 (*   10 mm below UwStopDiam_Limit / rewinder at RwMaxDiam during the ramp.*)
515
516 IF FinalStopDecel < 0.0 THEN
517     FinalStopDecel := 0.0;
518 ELSIF (FinalStopDecel > 0.0) AND (FinalStopDecel < 50.0) THEN
519     FinalStopDecel := 50.0;
520 ELSIF FinalStopDecel > 2000.0 THEN
521     FinalStopDecel := 2000.0;
522 END_IF;
523
524 EndStopReq := AutoStopCmd OR (Measure_ON AND UwStopByDiam);
525 EndRampOK := (FinalStopDecel > 0.0) AND FFActive AND StartMotor_Cmd;
526
527 IF EndStopReq AND EndRampOK THEN
528     FinalStopRamp := TRUE;                    (* -> soft stop, General 10 *)
529 END_IF;
530
531 IF NOT StartMotor_Cmd THEN
532     FinalStopRamp := FALSE;
533 END_IF;
534
535 EndRampHard := FinalStopRamp
536             AND ((Measure_ON AND (UwDiam_Act > 0.0)
537                 AND (UwDiam_Act < UwStopDiam_Limit - 10.0))
538                 OR ((RwDiam >= RwMaxDiam) AND NOT RwSensFault));
539
540 EndStopNow := (EndStopReq AND NOT FinalStopRamp) OR EndRampHard;
541
542 (* ---- 12. Learning supervision -> alarm 16 (section Alarms) ------------ *)
543 (* Learned paper thickness, for display / plausibility check              *)
544
545 IF MmPerPulse > 0.0 THEN
546     RwThick_mm := RwMat_K * 3.14159 / (2.0 * MmPerPulse);
547 ELSE
548     RwThick_mm := 0.0;
549 END_IF;
550
551 (* Fault: too many consecutive windows / stops that should have           *)
552 (* learned but did not. Clears by itself on the next good result.         *)
553 (* v6w: ALARM RESET (panel button or HMI) also clears the fail counters, *)
554 (* so alarm 16 resets in one press. It comes back only if the learning  *)
555 (* fails again RwLearnFailMax / RwStopFailMax times in a row.             *)
556
557 IF I_RsetAlarm OR Alarm_Reset THEN
558     LearnFailCnt := 0;
```

```
559     StopFailCnt := 0;
560   END_IF;
561
562   GSM_LearnFault := (LearnFailCnt >= RwLearnFailMax)
563             OR (StopFailCnt >= RwStopFailMax);      (* -> alarm 16 *)
564
565 (* v6e: learning status for the HMI. FALSE until learned.                *)
566
567   GSM_Learned_OK := (RwMat_K > 0.0)
568                 AND ((RwStopDecel > 0.0) OR NOT RwLearnStop)
569                 AND NOT GSM_LearnFault;
570
571 (* ---- 13. v6h: keep T-one in automatic tension control during the stop - *)
572 (* While running: BrakeHold = Brake_Cmd (no change in running behaviour).  *)
573 (* After the stop: stays ON until the web is below 5 m/min, after 15 s,   *)
574 (* or on E-stop. Already TRUE on the stop scan (Output_Manager runs before *)
575 (* this section), so O_Brake never drops for one scan.                    *)
576 (* Output_Manager rung 2:  O_Brake := Brake_Cmd OR BrakeHold              *)
577
578 IF O_MotorStart THEN
579     BrakeHold := Brake_Cmd AND ControlVoltage;
580     BrakeHoldTicks := 0;
581   ELSIF BrakeHold THEN
582
583     IF Tick THEN
584       BrakeHoldTicks := BrakeHoldTicks + 1;
585     END_IF;
586
587     (* v6v: signed web speed: a web pulled BACK (negative speed) also     *)
588     (* releases the T-one (trace 10/28 15:00: -6.6 m/min, about 20 cm).  *)
589
590     IF NOT ControlVoltage OR (Velocity_M_min < 5.0) OR (BrakeHoldTicks >= 150)
    THEN
591       BrakeHold := FALSE;
592     END_IF;
593   END_IF;
594
595 (* ---- 14. v6j: TEACH CORE - automatic offset with an empty core -------- *)
596 (* HMI: set RwTeachCore (button, Set ON). Conditions: motor stopped, web  *)
597 (* not moving, DT35 OK. After 2 s stable (filter settled):                *)
598 (*    RwAxisDist := RwDist + RwCoreDiameter / 2   -> RwDiam = RwCoreDiameter
    *)
599 (* RwTeachState: 0 idle, 1 teaching (wait 2 s), 2 done OK,                *)
600 (*               3 refused (motor running / web moving / DT35 fault).     *)
601 (* Only the offset is taught: the scale is RwDist10V - RwDist0V           *)
602 (* (= real distance between the two DT35 taught points).                  *)
603
604 IF RwTeachCore THEN
605     IF O_MotorStart OR (RwSpeed > 1.0) OR RwSensFault
606       OR NOT RwFiltInit OR (RwCoreDiameter <= 0.0) THEN
607       RwTeachState := 3;                    (* refused *)
608       RwTeachCore := FALSE;
609       TeachTicks := 0;
610     ELSE
611       RwTeachState := 1;                    (* teaching *)
612
613       IF Tick THEN
614         TeachTicks := TeachTicks + 1;
615       END_IF;
```

```
616
617       IF TeachTicks >= 20 THEN                (* 2 s stable *)
618          RwAxisDist_Prev := RwAxisDist;       (* v6l: keep previous *)
619          RwAxisDist := RwDist + RwCoreDiameter / 2.0;
620          RwTeachState := 2;                   (* done *)
621          RwTeachCore := FALSE;
622          TeachTicks := 0;
623       END_IF;
624    END_IF;
625 ELSE
626    TeachTicks := 0;
627
628    IF O_MotorStart THEN
629       RwTeachState := 0;                      (* clear status on next run *)
630    END_IF;
631 END_IF;
632
633 (* ---- 14b. v6k: unwinder TEACH CORE (HMI button Core_Measure) ---------- *)
634 (* Same behaviour as section 14. Sonar is not filtered, so the 2 s are   *)
635 (* averaged:  UwCoreDistance := average(UwDiam_Measure)  (sonar -> core   *)
636 (* surface). UwDiam_Act = (UwCoreDistance - UwDiam_Measure) * 2          *)
637 (*                      + UwCoreDiameter   (General rung 14, unchanged). *)
638 (* UwTeachState: 0 idle, 1 teaching, 2 done OK, 3 refused.               *)
639
640 IF Core_Measure THEN
641
642    IF O_MotorStart OR (RwSpeed > 1.0) OR (UwCoreDiameter <= 0.0)
643       OR (UwDiam_Measure <= 0.0) THEN
644       UwTeachState := 3;                      (* refused *)
645       Core_Measure := FALSE;
646       UwTeachTicks := 0;
647       UwTeachSum := 0.0;
648    ELSE
649       UwTeachState := 1;                      (* teaching *)
650
651       IF Tick THEN
652          UwTeachTicks := UwTeachTicks + 1;
653          UwTeachSum := UwTeachSum + UwDiam_Measure;
654       END_IF;
655
656       IF UwTeachTicks >= 20 THEN              (* 2 s, 20 samples *)
657          UwCoreDist_Prev := UwCoreDistance;   (* v6l: keep previous *)
658          UwCoreDistance := UwTeachSum / INT_TO_REAL(UwTeachTicks);
659          UwTeachState := 2;                   (* done *)
660          Core_Measure := FALSE;
661          UwTeachTicks := 0;
662          UwTeachSum := 0.0;
663       END_IF;
664    END_IF;
665 ELSE
666    UwTeachTicks := 0;
667    UwTeachSum := 0.0;
668
669    IF O_MotorStart THEN
670       UwTeachState := 0;                      (* clear status on next run *)
671    END_IF;
672 END_IF;
673
674 (*    14c. v6l: UNDO teach = swap current <-> previous (machine stopped) *)
```

```
675 (* Pressing UNDO twice goes back to the taught value. State 4 = RESTORED. *)
676
677 IF RwTeachUndo THEN
678
679     IF NOT O_MotorStart AND (RwAxisDist_Prev > 0.0) THEN
680         TeachSwap := RwAxisDist;
681         RwAxisDist := RwAxisDist_Prev;
682         RwAxisDist_Prev := TeachSwap;
683         RwTeachState := 4;                    (* restored *)
684     ELSE
685         RwTeachState := 3;                    (* refused *)
686     END_IF;
687     RwTeachUndo := FALSE;
688 END_IF;
689
690 IF UwTeachUndo THEN
691
692     IF NOT O_MotorStart AND (UwCoreDist_Prev > 0.0) THEN
693         TeachSwap := UwCoreDistance;
694         UwCoreDistance := UwCoreDist_Prev;
695         UwCoreDist_Prev := TeachSwap;
696         UwTeachState := 4;                    (* restored *)
697     ELSE
698         UwTeachState := 3;                    (* refused *)
699     END_IF;
700     UwTeachUndo := FALSE;
701 END_IF;
```

1-8-1-1-9.Tension_Helper

0

```
// TENSION HELPER: recommended T-one tension from paper GSM and roll width.
// Rule of thumb (paper): 0.54 N per mm of width per 100 g/m2 (TensFactor).
// Button TENSION UNITS (momentary) cycles kg -> lb -> N.

IF PaperGSM < REAL#10 THEN
    PaperGSM := REAL#10;
ELSIF PaperGSM > REAL#500 THEN
    PaperGSM := REAL#500;
END_IF;

IF WebWidth_mm < REAL#100 THEN
    WebWidth_mm := REAL#100;
ELSIF WebWidth_mm > REAL#3000 THEN
    WebWidth_mm := REAL#3000;
END_IF;

IF (TensFactor < REAL#0.1) OR (TensFactor > REAL#2) THEN
    TensFactor := REAL#0.54;
END_IF;

// Unit button: one press = next unit

IF TensUnitPB AND NOT TensUnitMem THEN
    TensUnit := TensUnit + 1;
    IF TensUnit > 2 THEN
        TensUnit := 0;
    END_IF;
END_IF;

TensUnitMem := TensUnitPB;

IF (TensUnit < 0) OR (TensUnit > 2) THEN
    TensUnit := 0;
END_IF;

TensIdeal_N := PaperGSM / REAL#100 * WebWidth_mm * TensFactor;

CASE TensUnit OF
    1:  TensIdeal_Disp := TensIdeal_N / REAL#9.81 * REAL#2.20462;   // lb
    2:  TensIdeal_Disp := TensIdeal_N;                              // N
ELSE
        TensIdeal_Disp := TensIdeal_N / REAL#9.81;                  // kg
END_CASE;

TensShowKg := (TensUnit = 0);
TensShowLb := (TensUnit = 1);
TensShowN  := (TensUnit = 2);
```

1-9.Data

1-9-1.Data Types

Name	Type	Comment

1-9-2.Global Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Network Publish	Comment
VAR_GLOBAL							
Pot1_Work_Velocity	INT		BuiltInIO://opt/#0/Ch1_Analog_Input_Value	False	False	Publish Only	Potentiometer 1 value - Work speed -
InvRef	INT		BuiltInIO://opt/#0/Ch2_Analog_Output_Value	False	False	Publish Only	Inverter speed reference
Fix10Vdc	INT		BuiltInIO://opt/#0/Ch1_Analog_Output_Value	False	False	Publish Only	Fixed 10 Vdc generator
VelMax	REAL	800		True	False	Publish Only	Maximum speed
VelSet	REAL			False	False	Publish Only	Set speed
RwSlowDeceIK	REAL	0.15		True	False	Publish Only	Learned slow-down decel per mm of rewinder diameter (mm/s2 per mm)
RwMaxSlowFallback	REAL	1360.0		True	False	Publish Only	Rewinder max: slow-down diameter while K not learned (mm)
PaperDensity	REAL			False	False	Publish Only	
Language	DINT			True	False	Publish Only	Language
RwThickMin_mm	REAL	0.02		True	False	Publish Only	Learning: min plausible paper thickness (mm)
PulseRate	UDINT		IOBus://unit#3/Ch1 Pulse Rate	False	False	Publish Only	Pulses per 100 ms
RwThickMax_mm	REAL	2.0		True	False	Publish Only	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL	20.0		True	False	Publish Only	Learning: min plausible stop decel (mm/s2)
RwDecelMax_mm	REAL	5000.0		True	False	Publish Only	Learning: max plausible stop decel (mm/s2)
RwLearnFailMax	INT	12		True	False	Publish Only	Learning: failed windows before alarm (12 = 2 min)
RwStopFailMax	INT	3		True	False	Publish Only	Learning: failed stops before alarm
RwThick_mm	REAL			True	False	Publish Only	Learned paper thickness (mm)
OverSpeed	BOOL			False	False	Publish Only	Speed too high
WorkSpd_Max	REAL	400.0		True	False	Publish Only	Work speed pot full scale (m/min), 50..VelMax
WorkSpd_Max_Set	REAL	400.0		True	False	Publish Only	HMI request: work pot full scale (m/min), applied only with motor stopped
UwRawMin	INT	10		True	False	Publish Only	Unwinder sonar raw min (below = sensor fault, alarm 17)
UwRawMax	INT	3990		True	False	Publish Only	Unwinder sonar raw max (above = sensor fault / no echo, alarm 17)
TensCheckEnable	BOOL	False		True	False	Publish Only	Tension check ON (HMI switch); OFF = start as before
TensWaitSpd	REAL	15.0		True	False	Publish Only	Speed held after START until tension established (m/min)
TensionOK	BOOL	False		False	False	Publish Only	Tension established (deboun

							ced T-one Alarm 0)
TensWait	BOOL	False		False	False	Publish Only	Waiting for tension (speed held at TensWaitSpd, HMI lamp)
SonarAsk	BOOL	False		False	False	Publish Only	V10 HMI popup: sonar is OFF, turn it on? (HMI NO resets it)
UwSlow_at_D_Req	BOOL	True		True	False	Publish Only	V7c: unwinder diameter slow-down follows the speed (FALSE = fixed UwSlowDown_Diam)
UwSlowDiamAct	REAL	0.0		False	False	Publish Only	V7c: unwinder slows down at this diameter (mm), HMI display
FinalStopDecel	REAL	500.0		True	False	Publish Only	V12: end-of-roll stop ramp, web decel (mm/s2), 0 = drive DEC ramp as before, 50..2000
FinalStopRamp	BOOL	False		False	False	Publish Only	V12: end-of-roll stop ramp running (HMI lamp)
RwJogMaxDiam	REAL	400.0		True	False	Publish Only	V7d: jog blocked above this rewinder diameter (mm), 0 = no limit
JogInhibit	BOOL	False		False	False	Publish Only	V7d: jog blocked (rewinder roll too big), HMI lamp
SCurveTime	REAL	0.5		True	False	Publish Only	V7f: S-curve rounding of the AUTO ramps (s), 0 = off, max 2
TOneDecel	BOOL	False		False	False	Publish Only	V7f: line decelerating (for a T-one stop/boost input)
HornStartBeep_s	REAL	1.0		True	False	Publish Only	V9 horn: beep when the slow-down starts (s), 0 = off, max 5
HornBeepLen_s	REAL	0.2		True	False	Publish Only	V9 horn: length of the short beeps during slow-down (s), 0 = off, max 2
HornBeepPeriod	REAL	5.0		True	False	Publish Only	V9 horn: one short beep every ... s during slow-down, 0 = off, max 30
HornStopBeep_s	REAL	3.0		True	False	Publish Only	V9 horn: beep when stopped after a slow-down (s), 0 = off, max 10
HornTest	BOOL	False		False	False	Publish Only	V9 horn: HMI test button (momentary), machine stopped only
RwLockDiam	REAL	300.0		True	False	Publish Only	V9: paper thickness LOCKED at this rewinder diameter (mm), 0 = never lock
RwLockMinGrow_mm	REAL	50.0		True	False	Publish Only	V9: min diameter growth since the start of learning before locking (mm)
RwMatLocked	BOOL	False		True	False	Publish Only	V9: paper thickness locked (HMI lamp), NEW PAPER / GSM YES unlocks
LongThick_mm	REAL	0.0		False	False	Publish Only	V9: thickness from the long baseline (mm), HMI display
RwDiamRev	REAL	0.0		False	False	Publish Only	V9: rewinder diameter averaged over one revolution (mm)
RwWave_mm	REAL	0.0		False	False	Publish Only	V9: diameter wave peak-pe

							ak in the last revolution (mm), HMI
UwStopLenShow	BOOL	False		False	False	Publish Only	V9 HMI: show unwinder STOP LENGTH (AUTO STOP UNWINDER active)
UwSlowDiamShow	BOOL	False		False	False	Publish Only	V9 HMI: show unwinder SLOW AT diameter (normal operation, sonar ON)
SoftCancelAccel	REAL	100.0		True	False	Publish Only	V9: then ramp up at this web accel (mm/s2), 0 = AccelWeb
RwMatReset	BOOL	False		False	False	Publish Only	HMI NEW PAPER: forget the learned paper thickness RwMat_K (PLC resets it)
FinalSpd_Max_Set	REAL	40.0		True	False	Publish Only	HMI request: final pot full scale (m/min), applied only with motor stopped
FinalSpd_Max	REAL	40.0		True	False	Publish Only	Final speed pot full scale (m/min), 5..WorkSpd_Max
UnderSpeed	BOOL			False	False	Publish Only	Speed too low
RwLearnReset	BOOL	False		False	False	Publish Only	HMI / Watch: clear learned decels RwStopDecelK, RwStopDecel, RwSlowDecelK (PLC resets it)
RwCoreDiameter	REAL	100.0		True	False	Publish Only	Rewinder core outside diameter for TEACH CORE (mm)
RwTeachCore	BOOL	False		False	False	Publish Only	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT	0		False	False	Publish Only	TEACH CORE: 0 idle 1 teaching 2 done 3 refused
SpdDelta_Percent	REAL			True	False	Publish Only	Speed difference percentage
Encoder_Pos	DINT		IOBus://unit#3/Ch1 Encoder Present Position	False	False	Publish Only	Encoder counter
UwTeachState	INT	0		False	False	Publish Only	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
Encoder_On	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Counter Enable	False	False	Publish Only	Encoder enable
Encoder_Reset	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Internal Reset Execution	False	False	Publish Only	Encoder reset
HMI_Screen	ARRAY[0..1] OF DINT			False	False	Publish Only	HMI panel Screen
Velocity_M_	REAL			True	False	Publish Only	Speed in m/min

min							
Current_Metre	REAL			True	False	Publish Only	Current metres
I_RsetCnt	BOOL			False	False	Publish Only	Metre counter reset button
Speed_Display	REAL			True	False	Publish Only	Set speed (displayed)
I_ControlVoltage	BOOL		BuiltInIO://cpu/#0/Input_Bit_00	False	False	Publish Only	In_0 ControlVoltageON
I_Start_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_01	False	False	Publish Only	In_1 Start button
I_Stop_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_02	False	False	Publish Only	In_2 Stop button
I_Motor_Thermal	BOOL		BuiltInIO://cpu/#0/Input_Bit_03	False	False	Publish Only	In_3 Motor thermal switch
I_Prot_ByPass	BOOL		BuiltInIO://cpu/#0/Input_Bit_04	False	False	Publish Only	In_4 Guards-bypassed selector switch - Under Panel
I_Hub_Position	BOOL		BuiltInIO://cpu/#0/Input_Bit_05	False	False	Publish Only	In_5 Hub/chuck in extraction position
I_FrontGuard_Closed	BOOL		BuiltInIO://cpu/#0/Input_Bit_06	False	False	Publish Only	In_6 Front guard closed
I_Jog_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_07	False	False	Publish Only	In_7 Jog Push button
I_VFD_Alarm	BOOL		BuiltInIO://cpu/#0/Input_Bit_08	False	False	Publish Only	In_8 Inverter in alarm
I_SafeGuard_Closed	BOOL			False	False	Publish Only	Side doors closed
I_OverRider_PH	BOOL		BuiltInIO://cpu/#0/Input_Bit_13	False	False	Publish Only	In_13 Roll diameter photocell
I_OR_Position_Sens	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN01 (Pin 1.1) _6000_01	False	False	Publish Only	OverRider position sens
I_OR_Pressure_Sens	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN02 (Pin 2.1) _6000_02	False	False	Publish Only	OverRider pressure switch
I_OR_Auto_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN03 (Pin 1.2) _6000_03	False	False	Publish Only	OverRider in automatic
I_OR_Cont_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-	False	False	Publish Only	OverRider in continuous

			ECO Input-_IN04 (Pin 2.2) _6000_04				
O_MotorStart	BOOL		BuiltInIO://cpu/#0/Output_Bit_00	False	False	Publish Only	Q_0 Motor start
I_FanTemp_Ok	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN06 (Pin 2.3) _6000_06	False	False	Publish Only	Motor (fan) thermal OK
O_Alarm_Array	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 01 (Pin 1.1) _7000_01	False	False	Publish Only	Alarm lamp
O_Prot_ByPass_Lamp	BOOL			False	False	Publish Only	Guards bypassed lamp
O_Motor_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_01	False	False	Publish Only	Q_1 Motor ON lamp
O_RwUnload	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 03 (Pin 1.2) _7000_03	False	False	Publish Only	Unloaders enable - opening
O_FrontGuard_ByPass	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 04 (Pin 2.2) _7000_04	False	False	Publish Only	Front guard bypass
O_Brake	BOOL		BuiltInIO://cpu/#0/Output_Bit_02	False	False	Publish Only	Q_2 Brake command
O_OR_Auto_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_06	False	False	Publish Only	Q_6 OverRider auto/continuous lamp
O_OverRider_Up	BOOL		BuiltInIO://cpu/#0/Output_Bit_09	False	False	Publish Only	Q_9 Raise OverRider ~10mm Over Rewinder roll
O_OverRider_Hold	BOOL		BuiltInIO://cpu/#0/Output_Bit_04	False	False	Publish Only	Q_4 OverRider Lock
O_CoolingFan	BOOL		BuiltInIO://cpu/#0/Output_Bit_03	False	False	Publish Only	Q_3 Brake and motor fans
Fan_Timeout	REAL	0.5		True	False	Publish Only	Fans stop timeout [min]
OverRider_TimeOFF	REAL	0.01		True	False	Publish Only	OverRider photocell OFF timeout (panel)
OverRider_TimeON	REAL	0.05		True	False	Publish Only	OverRider photocell ON timeout (panel)
AlarmOn	BOOL			False	False	Publish Only	Alarm active
BuzzerOn	BOOL			False	False	Publish Only	Panel buzzer command

Msg	INT			False	False	Publish Only	Message
Index_B	INT			False	False	Publish Only	Alarm index
MotStop	BOOL			False	False	Publish Only	Motor stopped
Clock1s	BOOL			False	False	Publish Only	1 s clock
ManMode	BOOL			True	False	Publish Only	Manual mode
Counter_Reset	BOOL			False	False	Publish Only	Counter reset from panel
I_FrontGuard_Open	BOOL		BuiltInIO://cpu/#0/Input_Bit_09	False	False	Publish Only	In_9 Front guard open
O_RwLoading_Enable	BOOL		BuiltInIO://cpu/#0/Output_Bit_07	False	False	Publish Only	Q_7 Unloaders Load enable - closing
JogSpd_SP	REAL	10.0		True	False	Publish Only	Speed in manual
O_AccDec2	BOOL		BuiltInIO://cpu/#0/Output_Bit_05	False	False	Publish Only	Q_5 Second acc/dec ramp
AccelTrim	REAL			True	False	Publish Only	Acceleration coefficient in run
DecelTrim	REAL			True	False	Publish Only	Deceleration coefficient in run
DecelCoarse	REAL			True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL			True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests accel)
UwStopDiamShow	BOOL	False		False	False	Publish Only	V11 HMI: show unwinder STOP AT diameter (slow-down reached, decel / final speed)
O_AcousticSignal	BOOL		BuiltInIO://cpu/#0/Output_Bit_08	False	False	Publish Only	Q_8 Horn / Acoustic signal
Pot2_Slow_Velocity	INT		BuiltInIO://opt/#0/Ch2_Analog_Input_Value	False	False	Publish Only	Potentiometer 2 value - Final speed -
I_AutoStop_OS	BOOL		BuiltInIO://cpu/#0/Input_Bit_10	False	False	Publish Only	In_10 Stop on metre counter
I_Input_11_Spare_1255	BOOL		BuiltInIO://cpu/#0/Input_Bit_11	False	False	Publish Only	In_11 SPARE (wire 12.55) Under Panel
I_WebBreak_Monitor	BOOL		BuiltInIO://cpu/#0/Input_Bit_12	False	False	Publish Only	In_12 Paper break monitoring ON
I_Vacuum_OK	BOOL		IOBus://unit#3/Ch1 Reset/External Input Status/Ch1 External Input 0 Status	False	False	Publish Only	Extractor fan thermal OK
I_T_one_TensionOK	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN05 (Pin 1.3) _6000_05	False	False	Publish Only	T-one Alarm 0 (15-16): tension established (replaces I_SlowSens_PH)

I_WebBreak_PH	BOOL		ECAT://node #[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN07 (Pin 1.4) _6000_07	False	False	Publish Only	Paper break photocell
UwStop_at_M_Val	REAL	5100		True	False	Publish Only	Unwinder Stop at metres
UwSlowMargin_m	REAL	10		True	False	Publish Only	Unwinder Slow-down metres
LowSpd_Display	REAL			True	False	Publish Only	Slow-down speed (displayed)
RwMaxDiam	REAL	1380.0		True	False	Publish Only	Rewinder max diameter protection (mm)
RwMaxStop	BOOL			False	False	Publish Only	Rewinder max diameter STOP (latched)
Encoder_Pulse	INT	1000		True	False	Publish Only	Encoder pulses/Rotation
RwSlowDecelLrn	REAL	0.0		True	False	Publish Only	Learned real slow-down decel (mm/s2) - 0 = not learned
CountReset_Req	BOOL			False	False	Publish Only	Counter reset requested
Psw	INT			False	False	Publish Only	Password Entry
I_RsetAlarm	BOOL		ECAT://node #[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN08 (Pin 2.4) _6000_08	False	False	Publish Only	Alarm reset
EncoderWheel_Circumf	REAL	400		True	False	Publish Only	Encoder wheel (Cylinder / Roller) Circumference [mm]
AlarmCodeSV	ARRAY[0..32] OF BOOL			False	False	Publish Only	Panel alarms
SafeGuard_ByPass	BOOL			False	False	Publish Only	Guards bypassed
FrontGuard_Closed	BOOL			False	False	Publish Only	Front guard closed
SoftStop	BOOL	False		False	False	Publish Only	Soft stop in progress (HMI)
SoftStopEnable	BOOL	True		True	False	Publish Only	STOP button = soft stop (False = immediate as before)
SoftStopEndSpd	REAL	5.0		True	False	Publish Only	Soft stop: run command drops below this speed (m/min)
UwDiam_Measure	REAL			True	False	Publish Only	Measured distance
UwCoreDiameter	REAL	100		True	False	Publish Only	Core Outside diameter [mm]
Core_Measure	BOOL	False		True	False	Publish Only	Stored core value
UwCoreDistance	REAL	785.56		True	False	Publish Only	Sonar distance to empty core. [mm]
AI_Diam_Unwinder	INT		IOBus://unit#2/Ch1 Analog Input Value	False	False	Publish Only	Analog value - UnWinder Diameter
Measure_ON	BOOL	True		True	False	Publish Only	Measurement enabled
UwDiam_Act	REAL			True	False	Publish Only	Measured roll diameter

UwSlow_at_D_Val	REAL	200		True	False	Publish Only	Unwinder Slow-down diameter [mm]
UwStopDiam_Limit	REAL	110		True	False	Publish Only	Stop diameter
Alarm_Reset	BOOL			False	False	Publish Only	Alarm reset from panel
Psw22	BOOL			False	False	Publish Only	Password 22 for Popup 22
Psw44	BOOL			False	False	Publish Only	Password 44 for Popup 44
I_Roll_OK	BOOL			False	False	Publish Only	Roll OK / Broken Holder Joint
AI_Diam_Rewinder	INT		IOBus://unit#2/Ch2 Analog Input Value	False	False	Publish Only	Analog value - ReWinder Diameter
RwStopEnable	BOOL	False		True	False	Publish Only	Rewinder diameter stop enabled
RwTargetDiam	REAL	300.0		True	False	Publish Only	Rewinder final diameter (mm)
RwUseSlow	BOOL	True		True	False	Publish Only	Slow down before target
RwSlowMargin_m	REAL	10.0		True	False	Publish Only	Rewinder Metres at slow speed before stop
RwSlowDecel	REAL	0.0		True	False	Publish Only	Rewinder Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL	100.0		True	False	Publish Only	Reset requests below target - this [mm]
RwDist0V	REAL	0.0		True	False	Publish Only	DT35 distance at 0 V (mm)
RwDist10V	REAL	750.0		True	False	Publish Only	DT35 distance at 10 V (mm)
RwRawMin	INT	10		True	False	Publish Only	Raw below = sensor fault
RwRawMax	INT	3900		True	False	Publish Only	Raw above = sensor fault
RwStopOnFault	BOOL	True		False	False	Publish Only	Stop on sensor fault
RwAxisDist	REAL	675.3		True	False	Publish Only	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL	0.2		True	False	Publish Only	Distance filter per 100 ms (0..1)
RwMinPulses	DINT	1000		True	False	Publish Only	Min pulses in window to learn
RwLearn_K	REAL	0.3		True	False	Publish Only	Material factor filter weight
RwMat_K	REAL	0.0		True	False	Publish Only	Learned material factor
RwLearnStop	BOOL	True		True	False	Publish Only	Learn stop decel on every stop
RwMinLearnSpd	REAL	50.0		True	False	Publish Only	Min speed to learn a stop (m/min)
RwStopDecel	REAL	0.0		True	False	Publish Only	Learned stop decel (pulses/s2)
RwTreact	REAL	0.1		True	False	Publish Only	Stop reaction time (s)
RwTstopDef	REAL	3.0		True	False	Publish Only	Stop time until decel learned (s)
RwTrim	REAL	1.0		True	False	Publish Only	Stop length trim (>1 = earlier)
RwDist	REAL			False	False	Publish Only	Filtered distance (mm)
RwDiam	REAL			False	False	Publish Only	Rewinder diameter (mm)
RwSpeed	REAL			False	False	Publish Only	Line speed from encoder (m/min)
RwRemain_m	REAL			False	False	Publish Only	Metres to target diameter
RwStopLen_m	REAL			False	False	Publish Only	Stopping length now (m)
RwSlowLen_m	REAL			False	False	Publish Only	Rewinder Slow-down length [m]

RwStop_at_D_Req	BOOL			False	False	Publish Only	Rewinder diameter STOP request
RwSlow_at_D_Req	BOOL			False	False	Publish Only	Rewinder diameter SLOW request
RwSensFault	BOOL			False	False	Publish Only	Rewinder DT35 sensor fault
UwStopEnable	BOOL	False		True	False	Publish Only	Unwinder Auto Stop - Enable
SlowAtDisp	REAL			False	False	Publish Only	HMI: slow-down starts at (m or mm, see SlowAtMode)
SlowAtMode	INT			False	False	Publish Only	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL			False	False	Publish Only	HMI: show slow-down display
UwSlow_at_M_Val	REAL			False	False	Publish Only	Unwinder: slow-down starts at [m]
UwSlow_at_M_Req	BOOL			False	False	Publish Only	Unwinder metre slow-down request (latched)
RwSlow_at_D_Val	REAL			False	False	Publish Only	Rewinder slow-down starts at diameter [mm]
RwStop_at_D_Val	REAL			False	False	Publish Only	Rewinder: stop command at diameter (mm)
RwMaxSlow	BOOL			False	False	Publish Only	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL	30.0		True	False	Publish Only	Unwinder Empty Hysteresis
GSM_Learned_OK	BOOL			False	False	Publish Only	Paper [Grams / Square Meter] FALSE until GSM learned
RwMaxSlowAtDiam	REAL	1370.0		False	False	Publish Only	Rewinder max: slow-down starts at diameter (mm)
RwStopDecelK	REAL	0.0		True	False	Publish Only	Learned stop decel per mm of rewinder diameter (mm/s2 per mm)
SoftStopDecel	REAL	0.0		True	False	Publish Only	Soft stop decel (mm/s2), 0 = DecelCoarse
SpdMax_Pending	BOOL	False		False	False	Publish Only	New pot range waiting for the motor to stop
RwAxisDist_Prev	REAL	675.3		True	False	Publish Only	Rewinder: RwAxisDist before the last TEACH (for UNDO)
UwCoreDist_Prev	REAL	785.56		True	False	Publish Only	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
RwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO rewinder TEACH (PLC resets it)
UwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO unwinder TEACH (PLC resets it)
Psw33	BOOL			False	False	Publish Only	Password 33 for Popup 33
RwMatChangePct	REAL	35.0		True	False	Publish Only	Paper change detection: window deviation (%), 2 windows in a row = new paper, 0 = off
RwSlowFallbackMm	REAL	30.0		True	False	Publish Only	Rewinder: slow-down this many mm before the target while the paper thickness is not learned (0 = off)
RwAutoMaxSpd	REAL	500.0		True	False	Publish Only	Rewinder AUTO stop ON: maximum work speed (m/min), 0 = no limit
RwWinLen_	REAL	60.0		True	False	Publish Only	Paper thickness learning: M

m							AX window length (m of paper)
RwWinGrow_mm	REAL	5		True	False	Publish Only	Paper thickness learning: window = this diameter growth (mm), 4 m .. RwWinLen_m
GsmAsk	BOOL	False		False	False	Publish Only	HMI popup: new parent roll, did you change the paper? (HMI NO resets it)
GsmAskYes	BOOL	False		False	False	Publish Only	HMI popup YES: forget the learned thickness (PLC resets it)
DecelWeb	REAL	250.0		True	False	Publish Only	V7: AUTO decel ramp, real web mm/s2 (0 = DecelCoarse)
AccelWeb	REAL	200.0		True	False	Publish Only	AUTO accel ramp, real web mm/s2 (0 = AccelCoarse)
RwRatioD	REAL	2900.0		True	False	Publish Only	V7: drive counts per m/min x rewinder diameter (mm), adapted while running
FFEnable	BOOL	True		True	False	Publish Only	V7: feed-forward speed loop ON (FALSE = old loop of rung 16)
SpdHold	BOOL	False		False	False	Publish Only	V9: speed held after START during a soft stop, START again = ramp up (HMI lamp)
SonarAskYes	BOOL	False		False	False	Publish Only	V10 HMI popup YES: sonar ON (PLC resets it)
UwStop_at_D_Val	REAL	0.0		False	False	Publish Only	V11: unwinder stop command diameter (mm) so the web stops at UwDiamStop, HMI "STOP AT"
Clock10ms	BOOL			False	False	Publish Only	10mS clock
Clock100ms	bool			False	False	Publish Only	100mS clock
Sec_to_Min	REAL			False	False	Do not publish	
Psw48	BOOL			False	False	Publish Only	Password 48 for Popup 48
JogRun	BOOL			False	False	Publish Only	V10: JOG really running (button, not a normal run, not a soft stop)
GuardStopOK	BOOL			False	False	Publish Only	V11: stopped for real -> guard bypass allowed (Output 13)
SlowDown	BOOL			False	False	Publish Only	Slow down
WarningOn	bool			False	False	Publish Only	HMI Warning Display
AlarmSilenced	BOOL			False	False	Publish Only	
PaperGSM	REAL			False	False	Publish Only	
WebWidth_mm	REAL			False	False	Publish Only	
TensFactor	REAL			True	False	Publish Only	Tension Helper
TensShowKg	BOOL			True	False	Publish Only	Tension Helper
TensShowLb	BOOL			True	False	Publish Only	Tension Helper
TensShowN	BOOL			True	False	Publish Only	Tension Helper
TensIdeal_Disp	REAL			True	False	Publish Only	Tension Helper
TensUnit	INT			True	False	Publish Only	Tension Helper
TensUnitPB	BOOL			False	False	Publish Only	Tension Helper
TensLostWarn	BOOL	False		False	False	Publish Only	Tension lost while running (warning)

	UwSensTest	BOOL	False		False	False	Publish Only	Unwinder sonar fault (raw out of range 1 s)
	UwSensFault	BOOL			False	False	Publish Only	UnWinder Sensor Fault
	Thickness_Fault	BOOL			False	False	Publish Only	Thickness Not Learned !